

《Web 应用开发项目实践》课程设计报告

(2025/2026 学年第二学期)

题目： 物资库存管理系统

学号： 2024337621070

姓名： 王泓璇

班级： 24 计科 3 班

指导教师： 何晓玉、陶灵兵

浙江理工大学计算机科学与技术(人工智能)学院

一、 选题背景与意义

1.1 选题背景

物资库存管理是学校实验室、学院办公室、企业仓库和行政后勤部门中常见的管理场景。物资通常具有种类多、规格复杂、库存变化频繁和领用记录需要追溯等特点。传统的纸质台账或 Excel 表格虽然使用门槛低，但在多人协同、库存实时更新、历史记录查询和统计分析方面存在明显不足。一旦出现漏登、重复登记或库存数据滞后，就可能造成库存盘点困难、重复采购、物资短缺或账实不符等问题。

选择“物资库存管理系统”作为题目，围绕物资基础资料维护、入库登记、出库登记和统计报表四类核心需求进行系统设计。通过 Web 应用的方式，可以将物资管理流程标准化、数据电子化和统计可视化，使管理员能够快速掌握库存总量、低库存物资、分类分布和出入库变化趋势。

1.2 选题意义

本课题选择“物资库存管理系统”作为 Web 应用开发项目，具有较强的实践价值、技术训练价值和综合应用意义。物资库存管理是学校、企业、实验室、仓库、办公室和后勤管理部门日常工作中的重要内容，传统人工记录方式容易出现数据遗漏、库存统计不及时、出入库记录混乱、库存数量计算错误、物资重复采购或账实不符等问题。通过开发物资库存管理系统，可以实现物资信息、入库记录、出库记录和库存统计的统一管理，使物资流转过程更加规范、透明和可追溯，从而提高物资管理效率和库存数据准确性。

1.实践意义：通过本系统的设计与实现，可以将前端页面设计、后端接口开发、数据库设计、用户登录认证、角色权限控制、库存业务逻辑处理、文件上传和统计图表展示等知识综合运用到实际项目中。本系统采用前后端分离模式，前端负责页面展示、用户交互、路由跳转和图表渲染，后端负责接口服务、业务处理、权限校验、数据库访问和库存计算。通过完整开发该系统，可以帮助开发者熟悉 Web 应用从需求分析、总体设计、数据库建模、详细设计、编码实现到测试运行的完整流程，提高解决实际业务问题的能力。

2.技术意义：本系统前端使用 Vue3、Vite、Element Plus、Vue Router、Axios 和 ECharts 等技术实现后台管理界面、页面路由、接口请求和统计图表展示；后端使用 SpringBoot 3.3.5 构建 RESTful API 服务，使用 Spring Security+JWT 实现登录认证、接口鉴权和角色权限控制，使用 BCrypt 对用户密码进行加密存储；数据库持久化方面使用 Spring Data JPA+Hibernate 完成实体映射、增删改查和分页查询，针对库存统计报表中的复杂聚合查询辅助使用 JdbcTemplate；数据库采用 MySQL 8.0 保存用户、物资、入库记录和出库记录等核心业务数据；接口文档方面使用 Springdoc OpenAPI+Swagger UI 生成接口说明，便于接口测试、前后端联调和答辩演示。通过该项目，可以加深对主流前后端开发框架、数据库持久化、安全认证、RESTful 接口规范和图表可视化技术的理解。

3.应用意义：物资库存管理系统能够实现物资基本信息维护、物资照片上传、物资入库、物资出库、库存自动计算、负库存拦截、低库存预警和库存报表统计等功能。系统中

的每种物资通过唯一编号进行标识，可以有效避免物资重复录入；入库操作完成后系统自动增加库存，管理员出库直接扣除，操作员出库申请提交后，系统先进行库存校验并生成待审核记录，管理员审核通过后系统自动扣减库存；若审核驳回，则库存数量不发生变化，当出库数量超过当前库存时系统会进行拦截并提示库存不足，从而减少人工计算库存带来的错误。库存统计模块能够根据物资数据和出入库记录生成分类库存、出入库趋势、库存排名和低库存预警等图表，为管理员了解库存状态和制定补货计划提供数据支持。对于中小型仓库、实验室耗材管理、办公用品管理等场景，具有一定的实际应用价值。同时，系统在出库业务中增加管理员审核机制，操作员提交出库申请后需要管理员审核通过才会正式扣减库存，使物资领用流程更加规范，进一步提高库存管理的安全性和可追溯性。

4.综合意义：本课题涵盖需求分析、技术选型、总体设计、数据库设计、详细设计、前后端编码实现、接口联调、系统测试、问题分析和文档撰写等完整软件流程。通过完成该系统，可以提升需求分析能力、模块划分能力、数据库建模能力、接口设计能力、前后端联调能力、代码调试能力和项目文档撰写能力。课程任务书中选题3要求完成系统用户管理、物资基本信息管理、物资入库管理、物资出库管理以及相应统计图表功能，本系统的设计与实现正是围绕这些要求展开。同时，系统在基本功能基础上进一步实现了登录鉴权、角色权限、物资照片上传、低库存预警、分页查询、条件查询和数据可视化展示等扩展内容，使课程设计成果更加完整，也为后续学习企业级 Web 系统开发打下基础。同时，系统在基础库存管理功能之外增加了 AI 智能助手功能。用户可以通过智能助手咨询系统操作方法、库存管理规则、出库审核流程、低库存预警说明和统计报表查看方式等内容。智能助手由前端聊天窗口发起请求，后端统一调用智谱 API 获取回答，并将结果返回前端展示。该功能提高了系统的人机交互能力和使用便捷性，使系统在完成基础库存管理的同时，具备一定的智能辅助服务能力。

二、 技术选型介绍

本物资库存管理系统面向实验室、仓库、办公物资等典型库存管理场景，系统不仅需要完成用户登录认证、物资基础信息维护、照片上传、入库登记、出库登记、库存自动计算、负库存拦截等基础业务，还需要支持分页查询、条件检索、库存统计分析、图表展示以及接口文档生成等功能。结合课程设计要求、系统功能需求和实际代码实现情况，本系统采用 Vue3+SpringBoot+MySQL 的前后端分离技术路线，并结合 Element Plus、Axios、ECharts、Spring Security、JWT、Spring Data JPA、JdbcTemplate 和 Swagger UI 等技术完成系统开发。SpringBoot 与 Vue 技术组合在 Web 管理系统开发中应用较为广泛，能够较好地兼顾前端交互体验和后端业务处理能力^[1]。

2.1 总体技术路线

本系统采用前后端分离架构进行设计与实现。前端项目位于 frontend 目录，主要负责页面布局、交互逻辑、路由管理、表单校验、数据展示以及图表可视化；后端项目位于 backend

目录，主要负责接口设计、业务处理、权限控制、库存计算、数据库访问、文件上传及统计报表生成。前后端通过 HTTP 协议进行通信，前端利用 Axios 调用后端 RESTful API，后端统一返回 JSON 格式数据，由前端完成最终渲染与交互反馈。前后端分离模式能够降低页面展示层与业务逻辑层之间的耦合度，提高系统的可维护性和扩展性。

系统整体技术架构如图 1 所示。

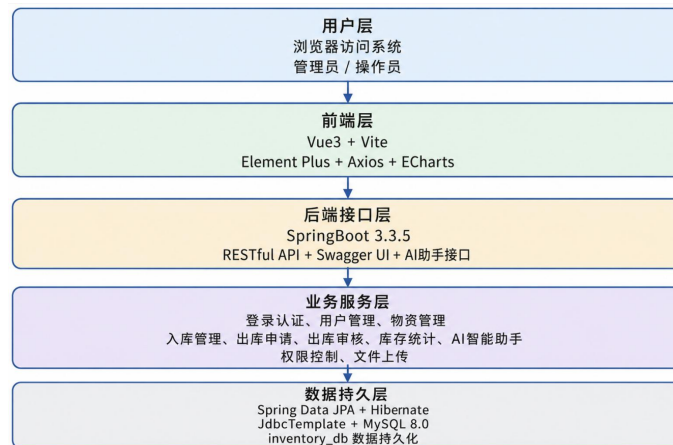


图 1 物资库存管理系统技术架构图

由图 1 可知，本系统整体分为用户层、前端层、后端接口层、业务服务层和数据持久层五个部分。用户层主要面向管理员和操作员两类用户，用户通过浏览器访问系统并完成相应操作。管理员主要负责用户管理、物资数据维护和系统数据查看；操作员主要负责物资入库、物资出库和库存信息查询等日常业务。

前端层采用 Vue3 + Vite 技术进行开发。Vue3 具有响应式数据绑定和组件化开发特点，适合构建单页 Web 应用系统^[2]。系统使用 Element Plus 实现表格、表单、按钮、弹窗、分页、菜单和上传组件等常用后台管理界面元素，使系统界面风格统一、操作友好。Axios 用于前后端接口通信，前端在请求中携带登录后的 JWT Token，实现登录状态保持和接口访问控制。ECharts 用于库存统计图表展示，能够支持柱状图、折线图、饼图等多种图表形式，适合库存分类统计、出入库趋势和低库存预警等数据可视化场景^[7]。

后端接口层采用 SpringBoot3.3.5 构建 RESTful API 服务。SpringBoot 可以简化 Spring 应用开发过程，方便整合 Web、数据库访问、安全认证、参数校验和接口文档等常用模块^[3]。系统接口按照 RESTful 风格设计，例如物资管理、入库管理、出库管理、用户管理和统计报表等模块均通过统一的 API 路径对外提供服务。RESTful API 通过资源路径和 HTTP 请求方法描述业务操作，具有结构清晰、易于维护和便于前后端协作的特点^[8]。

业务服务层是系统业务逻辑的核心部分，主要包括登录认证、用户管理、物资管理、入库管理、出库申请、出库审核、库存统计、权限控制、文件上传和 AI 智能助手等功能。其中，登录认证用于校验用户身份并生成 JWT Token；用户管理用于维护系统用户信息和账号状态；物资管理用于维护物资编号、名称、分类、规格、单位、供应商、库存数量、安全库存和物资照片等信息；入库管理用于登记入库记录并自动增加库存；出库管理用于提交出库申请并记录出库业务信息，系统在提交申请时进行库存校验；管理员审核通过后，

系统才正式扣减库存；若库存不足或审核驳回，则不会改变库存数量；库存统计用于统计库存总量、分类库存、出入库趋势和低库存预警等数据，AI 智能助手用于接收用户提出的系统操作或库存管理相关问题，后端通过统一接口调用智谱 API 获取回答，并返回给前端聊天窗口展示。为了保证接口安全，智谱 API Key 不直接暴露在前端，而是由后端统一读取配置并代理调用。。

数据持久层采用 Spring Data JPA+Hibernate 完成实体类与数据库表之间的映射，实现用户、物资、入库记录和出库记录等基础数据的增删改查、分页查询和条件查询。Hibernate 是常见的对象关系映射框架，能够将 Java 实体对象与关系型数据库表进行映射，从而减少重复的数据访问代码^[5]。对于库存统计、分类汇总、趋势分析等复杂 SQL 查询，系统使用 JdbcTemplate 进行辅助实现，提高统计报表查询的灵活性。数据库采用 MySQL 8.0，数据库名为 inventory_db，主要保存用户信息、物资信息、入库记录和出库记录等核心业务数据。MySQL 作为常见关系型数据库，具有稳定性高、使用广泛、事务支持较好等特点，适合保存库存系统中的结构化业务数据^[6]。

整体来看，本系统的技术路线具有层次清晰、职责明确、便于维护和扩展的特点。前端主要负责页面展示和用户交互，后端主要负责业务处理和接口服务，数据库主要负责数据持久化存储。前后端之间通过 RESTful API 进行通信，既降低了系统耦合度，也提高了系统的可维护性和扩展性。

2.2 系统主要技术选型

为保证系统功能完整、界面友好、数据逻辑正确并具备较好的可维护性，本系统结合课程设计要求和实际开发需要，选取了如下技术方案。系统主要技术选型如表 1 所示。

表 1 系统主要技术选型

层次	技术	作用	选型理由
前端框架	Vue3 + Vite	构建单页应用和开发环境	Vue3 支持组件化开发，适合将登录页、物资管理页、入库管理页、出库管理页和统计报表页拆分为独立组件； Vite 启动速度快，热更新效率高，便于课程项目快速开发和调试 ^[2] 。
前端路由	Vue Router	实现页面跳转和路由管理	系统包含多个功能页面，使用 Vue Router 可以统一管理页面路径，并可结合路由守卫实现未登录拦截和权限控制。
UI 组件库	Element Plus	实现表格、表单、按钮、菜单、弹窗、分页、上传等界面组件	Element Plus 与 Vue3 兼容性好，组件丰富、风格统一，适合后台管理系统开发，能够提高界面美观性和开发效率。

接口通信	Axios	实现前端与后端之间的 HTTP 数据交互	Axios 使用简单，支持请求拦截器和响应拦截器，可以统一携带 JWT Token，并统一处理接口异常和登录失效等情况。
图表展示	ECharts	实现库存统计图表和 数据可视化	ECharts 图表类型丰富， 适合展示库存分类统计、出入库趋势、 库存排名和低库存预警等数据， 能够提升系统的数据分析能力 ^[7] 。
后端框架	SpringBoot 3.3.5	构建后端 RESTful API 服务	SpringBoot 开发效率高，生态成熟，能够快速整合 Web、数据库、安全认证、参数校验和接口文档等模块， 适合开发管理类 Web 系统。
认证机制	Spring Security + JWT	实现用户登录认证、接口鉴 权和角色权限控制	Spring Security 提供完善的安全控制机制， JWT 适合前后端分离系统的无状态认证， 可以避免传统 Session 依赖， 提高接口访问安全性 ^[4] 。
接口文档	Springdoc OpenAPI+ Swagger UI	自动生成接口文档并提供 在线测试页面	能够展示接口地址、请求方式、参数说明 和返回结果，便于接口测试、 前后端联调和答辩演示
数据持久 化框架	Spring Data JPA+Hibernate	实现实体映射、增删改查、 分页查询和关系维护	Spring Data JPA 能够减少重复 SQL 编写， 适合完成用户、物资、入库记录和出库记录等基础数据管理；Hibernate 负责对象关系映射，能够提高开发效率 ^[5] 。
密码加密	BCrypt	对用户密码进行 加密存储和校验	BCrypt 可以对密码进行不可逆哈希处理， 避免数据库中存储明文密码， 提高系统安全性。
数据库	MySQL 8.0	持久化保存系统业务数据	MySQL 稳定性高、使用广泛、工具支持完善，适合保存用户、物资、入库记录、出库记录等结构化业务数据 ^[6] 。
接口规范	RESTful API	规范前后端接口路径 和请求方式	RESTful 风格结构清晰， 使用 GET、POST、PUT、DELETE 分别表示查询、新增、修改和删除操作， 便于前后端联调和后期维护 ^[8] 。

AI 服务	智谱 API	实现系统智能问答和操作辅助	智谱 API 可以通过 HTTP 接口调用大语言模型能力，适合实现系统操作说明、库存管理规则问答和出库审核流程解释等智能辅助功能。系统由后端统一调用智谱 API，避免前端暴露 API Key，提高接口调用的安全性。
-------	--------	---------------	---

从表 1 可以看出，本系统的技术选型围绕“前后端分离、界面友好、安全认证、数据持久化、接口规范和统计图表展示”展开。前端技术栈主要保证页面开发效率和用户体验；后端技术栈主要保证接口服务、业务逻辑和系统安全；数据库与持久化技术主要保证数据存储的完整性和可追溯性；图表和接口文档技术则进一步增强了系统的展示效果和开发规范性。

2.3 技术框架优缺点分析

本系统采用的 Vue3+SpringBoot+MySQL 技术组合较适合物资库存管理系统这类管理信息系统。该组合既能保证系统具备较好的界面展示效果，又能保证后端业务逻辑清晰、数据处理规范和系统安全性较高。

从前端角度来看，Vue3 的组件化开发方式能够将不同页面拆分为独立模块，使页面结构更加清晰，便于维护和复用。Element Plus 提供了大量成熟的后台管理组件，可以快速完成统一风格的表格、表单、弹窗、菜单和分页设计，使系统界面更加规范、美观。Axios 能够统一处理接口请求，并结合 Token 实现登录状态保持。ECharts 可以将库存数据以图表形式展示出来，使管理员能够更加直观地了解库存数量、分类分布和出入库变化趋势^[7]。但是，Vue3、Element Plus 和 ECharts 都需要一定的配置和组件封装能力，如果页面结构设计不合理，容易造成组件之间数据传递复杂。

从后端角度来看，SpringBoot 的优势在于开发效率高、生态成熟、整合能力强。系统可以通过 SpringBoot 快速整合 Web 接口、数据库访问、安全认证、参数校验和接口文档等功能。Spring Security + JWT 能够较好地适应前后端分离系统的认证方式，登录成功后由后端生成 Token，前端后续请求携带 Token，后端解析 Token 后完成用户身份识别和权限控制^[4]。BCrypt 对密码进行加密存储，增强了系统账号安全性。但 Spring Security 配置相对复杂，对初学者有一定难度，需要理解过滤器、认证对象、权限配置等内容。

从数据持久化角度来看，Spring Data JPA 能够通过实体类和 Repository 接口快速完成常规数据库操作，适合用户、物资、入库记录和出库记录等基础模块的开发。JPA 的分页查询和实体映射机制可以减少大量重复 SQL 编写，提高开发效率。Hibernate 作为对象关系映射框架，能够将 Java 对象和数据库表结构对应起来，使数据访问层更加清晰^[5]。但对于库存统计报表这种涉及分组、聚合、时间趋势的复杂查询，如果全部使用 JPA 实现会比较繁琐。因此，本系统结合 JdbcTemplate 编写统计 SQL，使报表查询更加灵活，较好地平衡了开发效率和查询灵活性。

从数据库角度来看, MySQL 8.0 稳定性较高, 能够保存结构化业务数据, 并通过主键、唯一约束和外键关系保证数据完整性^[6]。本系统中, materials 表保存物资基础信息, users 表保存系统用户信息, inbound_records 表保存入库流水, outbound_records 表保存出库流水。物资表中的 code 字段作为唯一标识, 可以避免物资重复录入; 入库记录和出库记录通过 material_id 关联物资表, 通过 created_by 关联用户表, 可以保证每次库存变动都有明确的物资来源和操作人员, 实现库存流转的可追溯管理。

综上所述, 本系统采用的技术框架能够较好地满足课程设计对功能完整性、界面友好性、数据准确性、安全认证、接口规范和统计图表展示的要求。该架构分层清晰, 前后端职责明确, 系统具有较好的可维护性和扩展性。

三、 需求分析

主要对“物资库存管理系统”的功能性需求和非功能性需求进行分析。本系统需要设计并实现一个基于 Web 的物资库存管理系统, 系统应包含基础的系统用户管理、物资基本信息管理、物资入库管理、物资出库管理以及相应的库存统计图表功能。同时, 系统界面应清晰、友好、美观、易用; 物资基本信息应完整, 并支持照片上传; 每种物资应具有唯一标识; 入库和出库操作应能够自动计算库存; 当出库数量超过当前库存时, 系统应进行负库存拦截; 系统还应能够根据出入库数据统计库存报表并生成图表。在完成基本功能的基础上, 系统还可以进一步扩展低库存预警、角色权限控制、多条件查询、分页显示、异常提示等功能。

系统主要面向管理员和操作员两类用户, 核心功能包括登录认证、用户管理、物资管理、物资照片上传、入库管理、出库管理、库存统计、低库存预警等。系统采用前后端分离模式, 前端负责页面展示和交互操作, 后端负责接口服务、业务逻辑、权限校验和数据库访问, 数据库保存用户、物资、入库记录和出库记录等核心数据。通过该系统, 可以实现物资库存管理过程的信息化、规范化和可视化。

3.1 需求背景分析

物资库存管理是学校实验室、学院办公室、企业仓库、行政后勤部门等场景中常见的管理工作。物资通常具有种类多、规格型号复杂、库存变化频繁、领用记录需要追溯等特点。传统管理方式多依赖纸质台账或 Excel 表格, 虽然使用门槛较低, 但在多人协作、库存实时更新、历史记录查询、库存统计分析等方面存在明显不足。

传统物资管理方式主要存在以下问题:

- (1) 物资编号不统一, 容易造成同一种物资重复录入, 或者不同物资之间信息混淆。
- (2) 入库和出库记录分散, 库存数量需要人工计算, 容易出现计算错误。
- (3) 出库时缺少库存校验, 可能出现库存不足仍然出库的情况, 导致负库存问题。
- (4) 缺少低库存预警机制, 管理员无法及时发现需要补充采购的物资。
- (5) 历史出入库记录查询不方便, 难以追溯物资流转过程、操作人员和业务原因。

（6）缺少统计图表功能，管理人员无法直观了解库存总量、分类分布和出入库变化趋势。

针对以上问题，本系统围绕“物资信息维护—物资入库—物资出库—库存自动更新—库存统计分析”这一业务主线进行设计。系统通过信息化方式对物资基础信息、入库记录、出库记录和库存统计数据进行统一管理，提高库存数据的准确性、实时性和可追溯性。

3.2 系统需求总体分析

本系统总体需求可以概括为以下几个方面：

（1）界面友好需求：系统界面应清晰、美观、易用，采用统一的后台管理风格，菜单结构清晰，表格、表单、按钮、弹窗、分页和图表布局合理，使用户能够快速完成物资查询、新增、修改、删除、入库、出库和统计查看等操作。

（2）物资管理需求：系统应支持物资基本信息管理，每种物资具有唯一编号，物资信息应包括物资编号、物资名称、分类、规格型号、单位、供应商、库位、安全库存、当前库存和物资照片等内容。系统应支持物资照片上传和展示，便于用户识别物资。

（3）入库管理需求：系统应支持物资入库登记，操作人员选择已有物资并填写入库数量、入库单价、来源和备注后，系统自动增加该物资的当前库存，同时保存入库记录。

（4）出库管理需求：系统应支持物资出库申请和管理员审核。操作员选择已有物资并填写出库数量、领用人、用途和备注后，系统先校验库存是否充足，并生成待审核出库记录。管理员审核通过后，系统自动扣减该物资的当前库存；管理员审核驳回时，出库记录状态变为已驳回，库存数量不发生变化。当出库数量大于当前库存时，系统应拒绝提交出库申请并提示库存不足。

（5）库存统计需求：系统应能够根据物资信息、入库记录和出库记录生成库存报表，并使用图表展示库存总量、分类库存、出入库趋势、库存排名和低库存预警等内容。

（6）功能扩展需求：在基本功能基础上，系统进一步实现低库存预警、角色权限控制、分页查询、条件查询、图片上传、异常提示等扩展功能，使系统更加完整、实用和规范。

（7）AI 智能助手需求：系统应提供 AI 智能助手功能，用户可以在系统页面中打开智能助手窗口，输入与系统使用、库存管理、出库审核、低库存预警和统计报表相关的问题。系统接收到用户问题后，由后端调用智谱 API 获取回答，并将回答内容返回前端展示，从而为用户提供操作引导和智能辅助服务。

系统需求分析结构如图 2 所示。

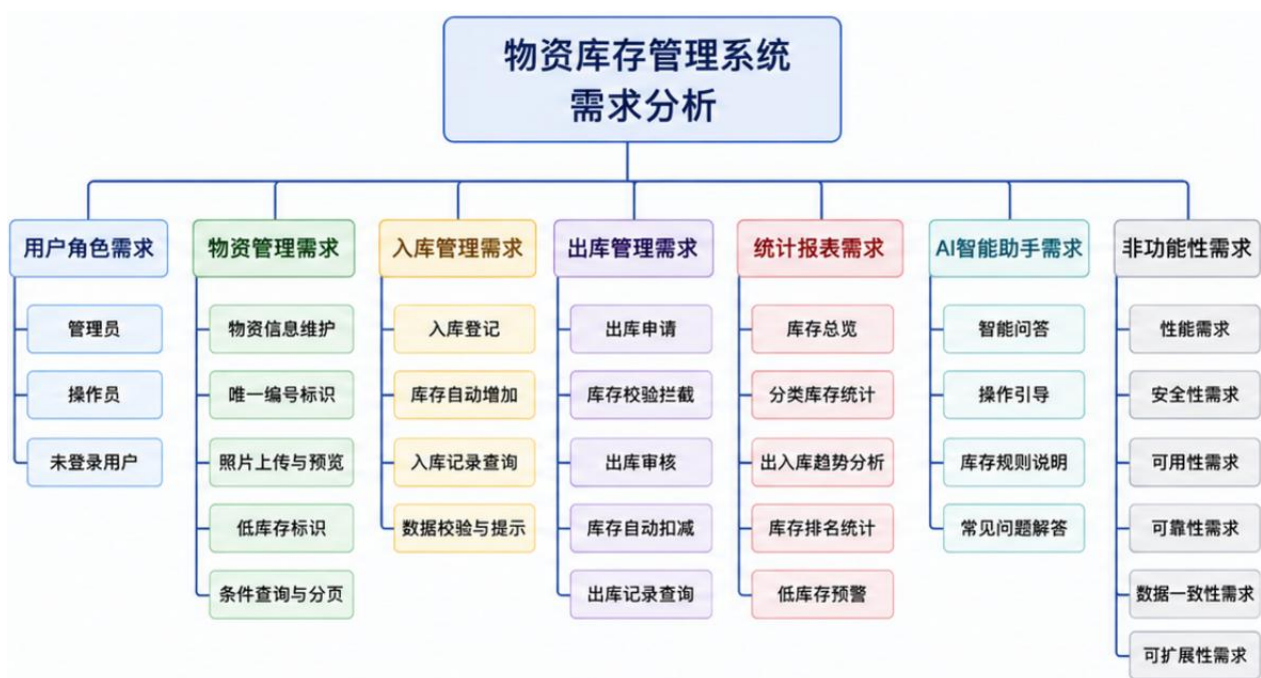


图 2 物资库存管理系统需求分析结构图

图 2 建议绘制为中心发散式结构图。中心节点为“物资库存管理系统需求分析”，向外分为六个部分：用户角色需求、物资管理需求、入库管理需求、出库管理需求、统计报表需求和非功能性需求。其中，用户角色需求包括管理员、操作员和未登录用户；物资管理需求包括唯一编号、基础信息维护、照片上传和低库存标识；入库管理需求包括入库登记、库存增加和入库记录查询；出库管理需求包括出库登记、库存扣减和负库存拦截；统计报表需求包括库存总览、分类统计、趋势分析和低库存预警；非功能性需求包括安全性、易用性、性能、可靠性、可维护性和数据一致性。

3.3 用户角色需求分析

根据系统使用场景和项目实际实现情况，本系统主要面向管理员和操作员两类用户，同时对未登录用户进行访问限制。不同角色具有不同的操作权限，可以避免普通用户误操作系统管理数据，提高系统安全性和管理规范性。

表 2 用户角色需求分析

用户角色	角色说明	主要需求	权限范围
管理员	系统最高权限用户，负责系统维护和基础数据管理	管理系统用户、维护物资信息、查看入库出库记录、查看统计报表、审核出库申请	可访问全部功能，包括用户管理、物资管理、入库管理、出库管理和库存统计
操作员	日常业务操作人员，负责物资入库、提交出库申请和库存查询	完成入库登记、提交出库申请、物资查询和库存查看	可访问物资查询、入库管理、出库管理和统计报表，不能管理系统用户

用户角色	角色说明	主要需求	权限范围
未登录用户	尚未通过身份认证的访问者	只能访问登录页面	不能访问系统业务功能, 请求受保护接口时应被系统拦截

通过用户角色划分，系统可以实现基本的权限控制。管理员负责系统维护和基础数据管理，操作员负责日常库存业务操作，未登录用户无法访问系统业务页面。这样的角色设计既满足课程设计中“基础系统用户管理”的要求，也提高了系统数据的安全性。

3.4 系统用例分析

系统用例分析主要从用户操作角度描述系统应提供的功能。管理员和操作员是系统的主要参与者。管理员除完成物资管理、入库管理、出库管理和统计查看外，还可以进行系统用户管理；操作员主要负责物资查询、入库登记、出库登记和库存统计查看。

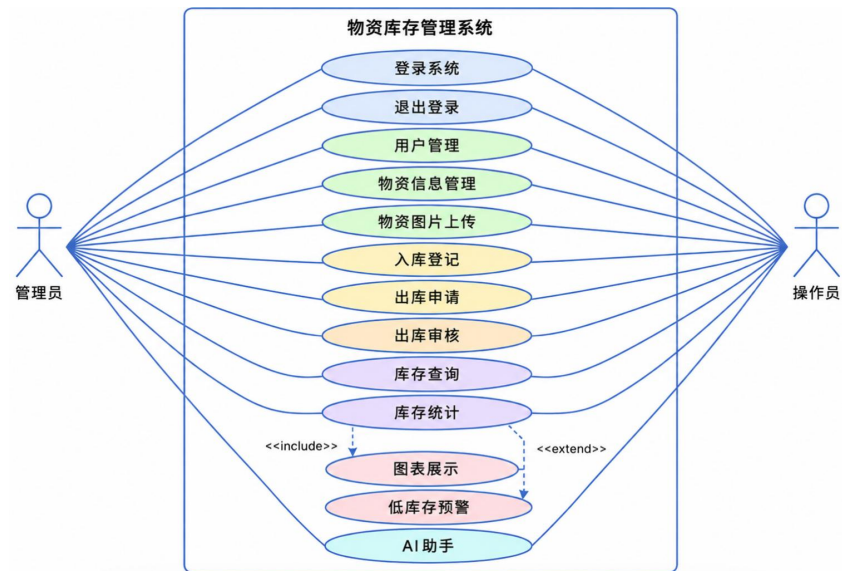


图 3 物资库存管理系统用例图

图 3 建议绘制为 UML 用例图。左侧为“管理员”，右侧为“操作员”，中间为系统边界“物资库存管理系统”。系统内部用例包括：登录系统、退出登录、用户管理、物资信息管理、物资照片上传、入库登记、出库登记、库存查询、库存统计、图表展示和低库存预警。管理员与所有用例建立关联；操作员与登录系统、退出登录、物资查询、入库登记、出库登记、库存统计和图表展示建立关联。

表 3 系统主要用例说明

用例名称	参与者	用例说明
登录系统	管理员、操作员	用户输入用户名和密码，系统完成身份认证，认证成功后进入系统首页
用户管理	管理员	管理员对系统用户进行新增、修改、删除、启用、停用和条件查询

用例名称	参与者	用例说明
物资信息管理	管理员、操作员	对物资编号、名称、分类、规格、单位、供应商、库位、安全库存等信息进行维护或查询
物资照片上传	管理员、操作员	在新增或修改物资时上传物资照片，便于识别物资
入库登记	管理员、操作员	对采购、补货或退还物资进行入库登记，系统自动增加库存
出库登记	管理员、操作员	提交出库申请：操作员填写物资、数量、领用人、用途等信息，系统校验库存后生成待审核出库记录。 出库审核：管理员查看待审核出库申请，可选择审核通过或审核驳回。 审核通过后系统扣减库存，审核驳回后库存不变。
库存查询	管理员、操作员	按物资编号、名称、分类、库存状态等条件查询当前库存情况
库存统计	管理员、操作员	查看库存总量、分类库存、出入库趋势和低库存物资等统计数据
图表展示	管理员、操作员	使用柱状图、折线图、饼图等形式展示库存统计结果
低库存预警	管理员、操作员	当当前库存小于或等于安全库存时，系统显示预警信息
退出登录	管理员、操作员	用户退出系统，清除登录状态和 Token 信息
AI 智能助手	管理员、操作员	用户在系统中输入库存管理或系统操作相关问题，系统调用 AI 接口生成回答，为用户提供操作引导和智能问答服务。

3.5 功能性需求分析

功能性需求是指系统必须完成的业务功能。系统功能性需求主要包括登录认证、用户管理、物资基本信息管理、物资入库管理、物资出库管理、库存统计图表和扩展功能。

3.5.1 登录认证功能需求

登录认证是系统使用的入口，也是系统安全控制的基础。用户进入系统后，需要通过用户名和密码登录。登录成功后，系统保存登录状态；登录失败时，系统应给出明确提示。

登录认证功能需求如下：

- （1）用户输入用户名和密码后，系统应进行必填校验。
- （2）用户名不存在或密码错误时，系统应拒绝登录并给出提示。
- （3）账号被停用时，系统应禁止该用户登录。
- （4）登录成功后，系统应进入首页或工作台页面，并显示当前用户信息。
- （5）用户未登录时访问业务页面，系统应自动跳转到登录页面。
- （6）用户退出登录后，系统应清除本地 Token 和用户信息。

该模块主要解决“谁可以进入系统”的问题，为后续物资管理、入库管理、出库管理

和统计报表查看提供身份认证基础。

3.5.2 用户管理功能需求

用户管理模块是管理员专属功能，主要用于维护系统用户信息。通过用户管理，可以实现多用户协同使用系统，并通过角色区分不同用户的操作权限。

用户管理功能需求如下：

- (1) 管理员可以分页查看系统用户列表。
- (2) 管理员可以根据用户名、姓名或角色进行条件查询。
- (3) 管理员可以新增用户，并设置用户名、姓名、密码、角色和账号状态。
- (4) 系统应保证用户名唯一，避免重复账号。
- (5) 用户密码应加密存储，数据库中不能保存明文密码。
- (6) 管理员可以修改用户姓名、角色和账号状态。
- (7) 管理员可以启用或停用用户账号，停用后的用户不能登录系统。
- (8) 系统应对管理员账号进行保护，避免误删或误停用关键账号。

用户管理模块不仅需要完成基础的增删改查，还需要体现系统的权限控制 and 安全性设计。

3.5.3 物资基本信息管理功能需求

物资基本信息管理是系统的基础模块。入库、出库和统计报表都依赖于准确的物资基础数据。系统需要为每种物资建立唯一标识，并维护完整的物资信息。

物资基本信息管理功能需求如下：

- (1) 支持新增物资信息，录入物资编号、物资名称、分类、规格型号、单位、供应商、库位、安全库存等字段。
- (2) 每种物资必须具有唯一编号，系统应在新增或修改时校验物资编号是否重复。
- (3) 支持物资照片上传，上传后的照片可以在物资列表或详情中展示。
- (4) 支持分页显示物资列表，避免数据较多时页面一次性加载过慢。
- (5) 支持按物资编号、物资名称、分类、库存状态等条件进行查询。
- (6) 支持修改物资基础信息，例如名称、分类、规格、单位、供应商、库位和安全库存等。
- (7) 支持删除物资信息，但对于存在出入库记录的物资，应谨慎处理，避免破坏历史数据完整性。
- (8) 物资当前库存应由入库和出库业务自动维护，避免人工随意修改造成库存不准确。
- (9) 当当前库存小于或等于安全库存时，系统应显示低库存状态或预警标识。

表 4 物资信息字段需求说明

字段名称	字段含义	是否必填	约束说明
物资编号	物资唯一标识	是	不允许重复，是区分不同物资的核心字段

字段名称	字段含义	是否必填	约束说明
物资名称	物资显示名称	是	用于列表展示和模糊查询
物资分类	物资所属类别	是	用于分类管理和分类统计
规格型号	物资规格描述	否	用于区分同类物资的不同型号
单位	物资计量单位	是	如个、件、箱、瓶、包等
供应商	物资来源单位	否	用于记录采购或供应来源
库位	物资存放位置	否	用于辅助仓库查找物资
当前库存	当前可用数量	是	由入库和出库业务自动计算
安全库存	库存预警阈值	是	当前库存低于或等于该值时触发预警
物资照片	物资图片地址	否	用于物资识别和页面展示

3.5.4 物资入库功能需求

物资入库功能用于记录物资增加的过程。当采购、补货、调拨或退还物资进入仓库时，操作人员需要登记入库信息。系统保存入库记录后，应自动增加对应物资的当前库存。

物资入库功能需求如下：

- (1) 入库时必须选择系统中已存在的物资。
- (2) 入库时需要填写入库数量、入库单价、来源和备注等信息。
- (3) 入库数量必须大于 0，不能为负数、0 或空值。
- (4) 入库成功后，系统应自动增加对应物资库存。
- (5) 系统应保存入库操作人和入库时间，便于后期追溯。
- (6) 支持分页查看入库记录。
- (7) 支持按物资名称、物资分类、日期范围等条件查询入库记录。
- (8) 入库记录应与物资信息关联，列表中应能显示物资编号、名称、单位和入库数量等信息。
- (9) 当入库保存失败时，库存数量不能发生异常变化。

入库模块的核心需求是“记录入库流水”和“自动增加库存”，重点体现库存计算的自动化和数据一致性。

3.5.5 物资出库功能需求

物资出库功能用于记录物资领用或消耗过程。为了提高物资领用的规范性，本系统在出库模块中增加管理员审核机制。操作员提交出库申请后，系统先判断出库数量是否合法以及当前库存是否充足，校验通过后生成待审核出库记录，库存暂不扣减。管理员审核通过后，系统正式扣减对应物资库存；若管理员审核驳回，则出库记录状态变为已驳回，物

资库存不发生变化。

物资出库功能需求如下：

- (1) 出库时必须选择系统中已存在的物资。
- (2) 出库时需要填写出库数量、领用人、用途和备注等信息。
- (3) 出库数量必须大于 0，不能为负数、0 或空值。
- (4) 系统应在提交出库申请前查询当前库存，判断库存是否足够。
- (5) 当出库数量大于当前库存时，系统必须拦截出库申请，并提示库存不足。
- (6) 操作员提交出库申请后，系统生成待审核出库记录，此时不立即扣减库存。
- (7) 管理员可以查看待审核出库记录，并进行审核通过或审核驳回操作。
- (8) 管理员审核通过后，系统自动扣减对应物资库存，并将出库记录状态更新为已通过。
- (9) 管理员审核驳回后，系统不扣减库存，并将出库记录状态更新为已驳回。
- (10) 系统应保存出库申请人、申请时间、审核人、审核时间和审核状态，便于后期追溯。
- (11) 支持分页查看出库记录，并支持按物资名称、领用人、审核状态、日期范围等条件查询。
- (12) 当审核通过后的库存小于或等于安全库存时，系统应在物资列表或统计报表中显示低库存预警。

出库模块的核心需求是“提交出库申请”“管理员审核”“审核通过后自动扣减库存”和“负库存拦截”。其中，出库审核机制是本系统在基本出库功能上的扩展，可以避免操作员直接出库造成误操作，提高物资领用流程的规范性和安全性。

3.5.6 库存统计与图表功能需求

库存统计与图表功能用于对系统中的物资数据、入库数据和出库数据进行汇总分析。通过统计报表，管理员可以直观了解库存总量、物资分类分布、近期出入库变化和低库存物资情况，从而辅助库存管理和采购决策。

库存统计与图表功能需求如下：

- (1) 统计物资总数、当前库存总量、今日入库数量、今日出库数量和低库存物资数量。
- (2) 按物资分类统计库存数量，使用柱状图或饼图展示分类库存分布。
- (3) 按日期统计入库和出库趋势，使用折线图展示一段时间内的库存流转变化的。
- (4) 统计库存数量排名靠前的物资，便于查看主要库存物资。
- (5) 统计低库存物资列表，提醒管理员及时补充库存。
- (6) 统计数据应来自真实业务数据，包括物资表、入库记录表和出库记录表。
- (7) 图表应具备标题、图例、坐标轴和数据提示，做到展示清晰、美观、易理解。
- (8) 报表页面应与系统整体风格保持一致，重点数据应突出显示。

表 5 库存统计图表需求说明

统计项目	数据来源	展示形式	功能作用
物资总数	物资信息表	数据卡片	展示系统中物资种类数量
当前库存总量	物资信息表	数据卡片	展示所有物资库存数量总和
今日入库数量	入库记录表	数据卡片	反映当天物资补充情况
今日出库数量	出库记录表	数据卡片	反映当天物资领用情况
分类库存统计	物资信息表	柱状图 / 饼图	展示不同类别物资的库存分布
出入库趋势	入库记录表、出库记录表	折线图	展示库存流转变化趋势
库存排名	物资信息表	横向柱状图	展示库存数量较多的物资
低库存预警	物资信息表	表格 / 预警列表	提醒管理员及时补充库存

3.5.7 功能扩展需求

在完成课程设计基本功能的基础上，为提高系统完整性和实用性，本系统进一步考虑以下扩展需求：

- （1）低库存预警功能：当物资库存低于或等于安全库存时，系统自动标记预警状态。
- （2）物资照片预览功能：物资信息支持图片上传和查看，提高物资识别效率。
- （3）角色权限控制功能：管理员和操作员具有不同权限，避免普通用户误操作系统管理功能。
- （4）多条件查询功能：物资、入库记录和出库记录均支持关键字、分类、日期等条件查询。
- （5）统计图表功能：通过图表展示库存分类、出入库趋势和库存排名。
- （6）异常提示功能：对重复编号、非法数量、库存不足、未登录访问等情况进行提示。
- （7）后续可扩展 Excel 导入导出、操作日志、库存盘点、多仓库管理和消息通知功能，使系统更加接近真实库存管理业务。
- （8）出库审核功能：操作员提交出库申请后，需要管理员审核通过后才正式扣减库存，审核驳回时库存不变。
- （9）AI 智能助手功能：系统增加智能助手入口，用户可以通过聊天窗口咨询系统操作方法、库存管理规则、出库审核流程、低库存预警和统计报表查看方式等内容。后端统一调用智谱 API 获取回答，前端负责展示问答结果，从而提高系统的智能化水平和用户使用体验。

3.6 非功能性需求分析

非功能性需求主要描述系统在运行质量、用户体验、安全性、性能和维护性等方面需

要达到的要求，能够体现系统是否规范、稳定、易用和具有扩展价值。

3.6.1 系统性能需求

物资库存管理系统属于典型后台管理类 Web 应用，主要操作包括用户登录、物资列表查询、物资新增修改、入库登记、出库登记和统计报表查看等。系统应保证在常规数据规模下能够稳定、流畅运行。

(1) 物资列表、入库记录列表、出库记录列表和用户列表应支持分页查询，避免一次性加载大量数据导致页面响应缓慢。

(2) 系统应支持条件查询，例如按物资编号、物资名称、分类、库存状态、领用人、日期范围等条件筛选数据，提高用户检索效率。

(3) 统计报表数据应按需加载，避免页面打开时一次性请求过多接口，影响页面加载速度。

(4) 前端页面切换、表单提交、弹窗打开和图表渲染应保持较好的响应速度，保证用户操作流畅。

(5) 后端接口应合理处理数据库查询逻辑，常规增删改查和分页查询应在较短时间内返回结果，满足课程设计演示和中小型库存管理场景的使用需求。

3.6.2 系统安全性需求

系统涉及用户账号、物资数据、出入库记录和库存统计信息，因此需要具备基本的安全认证和权限控制机制。

(1) 用户必须登录后才能访问系统业务页面，未登录用户访问受保护页面时应自动跳转到登录页面。

(2) 后端接口需要进行身份校验，前端请求业务接口时应携带 JWT Token，后端验证 Token 合法后才允许访问。

(3) 用户密码不能以明文形式存储，应使用 BCrypt 等加密方式进行哈希处理，提高账号安全性。

(4) 系统应区分管理员和操作员权限。管理员可以进行用户管理和系统数据维护，操作员主要负责物资查询、入库登记、出库登记和报表查看，不能访问用户管理等敏感功能。

(5) 对于权限不足、登录过期、Token 无效等情况，系统应给出明确提示，避免用户误以为系统异常。

(6) 系统应对重要业务操作进行后端校验，不能只依赖前端校验。例如出库数量是否超过库存、物资编号是否重复、用户名是否唯一等规则必须在后端进行判断。

(7) 系统应限制出库审核权限，只有管理员可以进行审核通过或审核驳回操作，操作员只能提交申请和查看审核状态。

(8) AI 智能助手调用第三方模型接口时，API Key 应保存在后端配置或环境变量中，不能写入前端代码，防止密钥泄露。前端只调用本系统后端接口，由后端统一代理访问智谱 API。

3.6.3 系统可用性需求

可用性主要体现为系统界面是否友好、操作是否简单、提示是否清晰。物资库存管理系统面向日常管理人员使用，因此界面和交互设计应尽量直观。

(1) 系统界面应清晰、友好、美观、易用，采用统一的后台管理布局，使用户能够快速找到对应功能。

(2) 登录页面应简洁明确，输入框、登录按钮和错误提示应布局合理。

(3) 物资管理、入库管理和出库管理页面应采用表格加表单的方式展示和操作数据，符合后台管理系统使用习惯。

(4) 新增、修改、删除、入库、出库等操作应提供明确反馈，例如“保存成功”“删除成功”“库存不足”“编号已存在”等提示信息。

(5) 删除等可能影响数据的操作应提供二次确认，减少误操作风险。

(6) 统计报表页面应通过数据卡片和图表展示核心数据，使管理员能够直观了解库存总量、分类分布、出入库趋势和低库存情况。

(7) 系统应提供 AI 智能助手入口，用户可以通过自然语言提问获得系统操作提示，减少对说明文档的依赖，提高系统易用性。

3.6.4 系统可靠性需求

可靠性是库存管理系统的重要要求，尤其体现在库存数量计算和业务操作一致性方面。

(1) 系统应保证入库操作成功后，入库记录和物资库存数量同时更新，不能出现只保存记录但库存未变化的情况。

(2) 系统应保证统应保证出库申请、审核状态和库存扣减逻辑一致。待审核和已驳回的出库记录不应扣减库存，只有审核通过的出库记录才允许扣减库存，不能出现库存扣减成功但出库记录未保存的情况。

(3) 当出库数量大于当前库存时，系统必须拒绝出库操作，防止出现负库存。

(4) 当入库或出库过程中出现异常时，系统应返回错误提示，并保证原有库存数据不被错误修改。

(5) 系统应保存入库和出库操作的时间、物资、数量、操作人和备注等信息，保证库存流转过程可追溯。

3.6.5 数据一致性需求

物资库存管理系统的数据一致性直接影响系统可信度，因此需要通过业务规则和数据库约束保证数据正确。

(1) 每种物资应有唯一编号，避免同一物资重复录入。

(2) 系统用户名应具有唯一性，避免多个用户使用同一登录账号。

(3) 入库数量和出库数量必须大于 0，不能出现负数、0 或空值。

(4) 当前库存应由入库和出库业务自动计算，普通用户不应随意手动修改库存数量。

(5) 出入库记录应与具体物资关联，不能存在没有对应物资的业务记录。

(6) 低库存判断应根据当前库存和安全库存自动计算，当当前库存小于或等于安全库存时，系统应显示预警状态。

3.6.6 可维护性与可扩展性需求

系统应具备清晰的代码结构和良好的可维护性，便于后续修改和扩展。

（1）前端代码应按照页面、组件、路由、接口请求等结构进行组织，使登录页、物资管理页、入库管理页、出库管理页和统计报表页相互独立。

（2）后端代码应采用分层结构，按照 Controller、Service、Repository、Entity、DTO、Security 等模块划分职责。

（3）Controller 层主要负责接收请求和返回响应，Service 层负责业务逻辑处理，Repository 层负责数据库访问，避免代码混杂。

（4）关键业务逻辑应添加必要注释，例如库存自动计算、负库存拦截、JWT 认证和文件上传等部分。

（5）接口命名、变量命名和数据库字段命名应尽量规范，便于阅读和后期维护。

（6）系统后续可扩展 Excel 导入导出、操作日志、库存盘点、多仓库管理、消息通知和更加细粒度的 RBAC 权限控制等功能。

表 6 系统非功能性需求汇总

需求类型	具体要求	实现目标
性能需求	支持分页查询、 条件查询和按需加载	保证列表查询、表单提交和图表展示响应流畅
安全性需求	登录认证、Token 鉴权、密码加密、角色权限控制	防止未授权访问，保护用户和库存数据
可用性需求	界面清晰、操作简单、提示明确	提高用户使用体验，减少误操作
可靠性需求	入库、出库和库存计算保持一致	防止库存数据错误和负库存问题
数据一致性需求	唯一编号、数量校验、 关联记录完整	保证物资数据和出入库数据准确可靠
可维护性需求	前后端分层清晰，代码结构规范	便于后续修改、调试和功能维护
可扩展性需求	支持后续增加导入导出、 日志、盘点等功能	提高系统后续扩展能力
兼容性需求	支持常见浏览器访问	保证系统在 Chrome、Edge 等浏览器中正常使用

3.7 核心业务规则分析

为了保证系统业务逻辑正确，库存管理过程中需要遵循明确的业务规则。核心业务规则如表 7 所示。

表 7 核心业务规则说明

业务场景	规则说明	异常处理
用户登录	用户名和密码必须正确，账号状态必须启用	用户不存在、密码错误或 账号停用时禁止登录
新增用户	用户名必须唯一，密码不能为空	用户名重复时提示错误
新增物资	物资编号必须唯一，名称、分类、单位 等关键字段不能为空	编号重复或必填项为空时禁止保存
上传照片	图片应与物资信息关联保存	上传失败时提示用户重新上传
入库登记	入库数量必须大于 0，物资必须存在	数量非法或物资不存在时禁止入库
入库库存计算	入库成功后，当前库存 = 原库存 + 入库数量	入库失败时库存不应变化
出库登记	出库数量必须大于 0，物资必须存在	数量非法或物资不存在时禁止出库
负库存拦截	出库数量不能大于当前库存	库存不足时拒绝出库并提示
出库库存计算	出库成功后，当前库存 = 原库存 - 出库数量	出库失败时库存不应变化
低库存预警	当前库存 $\leq$ 安全库存时标记为低库存	在列表或报表中显示预警状态
权限控制	管理员和操作员权限不同	权限不足时禁止访问对应功能
数据查询	支持分页和条件查询	查询条件为空时返回分页列表
提交出库申请	出库数量必须大于 0，且不能超过当前库存	数量非法或库存不足时禁止提交申请
出库审核通过	只有管理员可以审核，通过后扣减库存	权限不足时禁止审核
出库审核驳回	驳回后库存不变，记录状态为已驳回	审核意见可作为备注保存
重复审核限制	已通过或已驳回记录不能重复审核	重复操作时提示记录已处理

3.8 数据需求分析

根据系统功能需求和项目代码中的核心数据结构，系统至少需要保存用户数据、物资数据、入库记录和出库记录四类核心数据。用户数据用于登录认证和权限控制；物资数据用于维护库存基础信息；入库记录用于记录库存增加过程；出库记录用于记录库存减少过程；统计数据则根据物资信息、入库记录和出库记录计算得出。

表 8 系统核心数据需求

数据对象	主要字段	数据作用
用户数据	用户名、密码密文、姓名、角色、状态、创建时间	用于登录认证、权限控制和记录业务操作人
物资数据	物资编号、名称、分类、规格、单位、供应商、库位、当前库存、安全库存、照片地址	用于维护物资基础信息和当前库存状态
入库记录	物资 ID、入库数量、单价、来源、备注、创建人、创建时间	用于记录物资增加过程，并作为库存增加依据
出库记录	物资 ID、出库数量、领用人、用途、备注、创建人、创建时间、审核状态、审核人、审核时间、驳回原因	用于记录物资出库申请、审核状态和库存扣减依据，只有审核通过的出库记录才作为库存扣减依据
统计数据	分类库存、出入库趋势、低库存数量、库存排名	用于库存报表和图表展示

数据需求中最重要的是物资编号唯一性、库存数量准确性和业务记录可追溯性。物资编号用于唯一识别每一种物资；当前库存由入库和出库操作自动计算；入库和出库记录用于保存库存变化过程，为统计报表和后续追溯提供数据依据。

四、 总体设计

本系统主要围绕系统用户管理、物资基本信息管理、物资入库管理、物资出库管理、库存查询和统计报表展示等功能展开设计。从功能模块划分、数据库设计、接口设计和权限安全设计等方面进行总体说明。

本系统总体设计目标如下：

（1）功能模块完整：系统应包含登录认证、用户管理、物资管理、入库管理、出库管理、库存管理和统计报表等核心模块，能够覆盖物资库存管理的主要业务流程。

（2）业务逻辑清晰：物资入库后自动增加库存，物资出库后自动扣减库存，当出库数量超过当前库存时，系统应进行出库拦截，避免出现负库存。

（3）数据结构合理：系统围绕用户、物资、入库记录和出库记录四类核心数据进行设计，通过主键、唯一约束和外键关联保证数据完整性和可追溯性。

（4）界面操作友好：系统采用后台管理系统常见布局方式，支持分页查询、条件筛选、弹窗表单、图片上传和图表展示，使用户能够快速完成日常操作。

（5）具备扩展能力：系统在完成基本功能的基础上，进一步支持低库存预警、角色权限控制、统计图表、文件上传等扩展内容，为后续增加 Excel 导入导出、操作日志和库存

盘点等功能保留扩展空间。

4.1 系统功能模块设计

根据需求分析结果，物资库存管理系统可以划分为系统管理、物资管理、入库管理、出库管理、库存管理和统计报表六个主要功能模块。系统功能模块设计如图 4 所示。

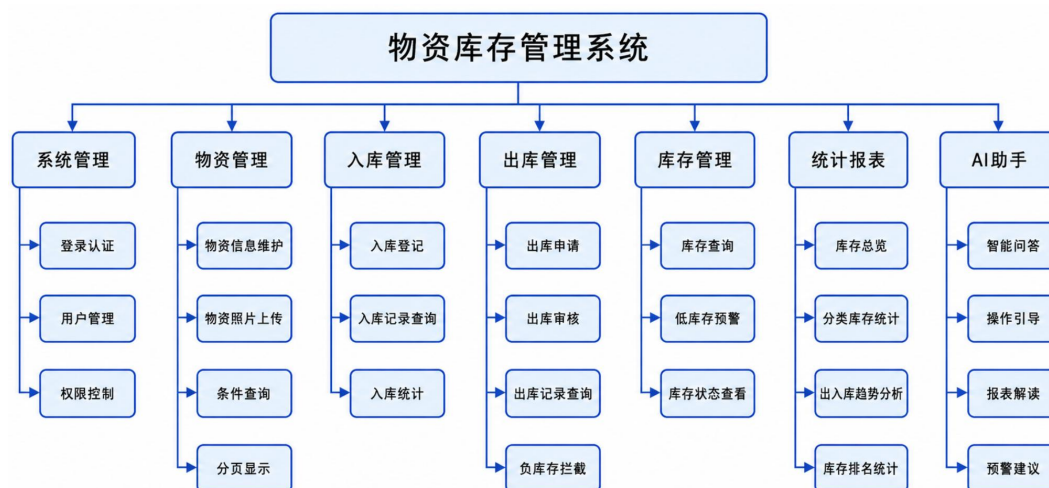


图 4 物资库存管理系统 Web 功能模块图

由图 4 可知，本系统功能模块围绕“物资基础信息维护—入库登记—出库登记—库存状态查看—统计报表分析”这一业务主线展开设计。系统管理模块用于完成用户登录、用户维护和权限控制；物资管理模块用于维护物资基础资料和物资照片；入库管理模块用于记录物资入库并自动增加库存；出库管理模块用于记录物资出库并自动扣减库存；库存管理模块用于查看当前库存状态和低库存预警；统计报表模块用于对库存数据和出入库数据进行统计分析，并通过图表展示结果。各功能模块具体说明如下：

（1）系统管理：系统管理模块主要包括用户登录、退出登录、用户管理和权限控制功能。用户通过用户名和密码登录系统，登录成功后系统保存当前登录信息；当检测到用户未登录或登录状态失效时，自动跳转到登录页面。管理员可以对系统用户进行新增、编辑、删除、启用和停用操作，并支持按用户名、姓名或角色进行查询。用户密码以加密形式存储，页面中不显示明文密码。

（2）物资管理：物资管理模块用于维护物资基本信息，是入库、出库和统计报表的基础。系统支持物资新增、编辑、删除、分页查询、条件查询和照片上传。物资信息包括物资编号、名称、分类、规格、单位、供应商、库位、当前库存、安全库存和照片等内容。其中，物资编号作为唯一标识，系统需要校验其唯一性，避免重复录入。

（3）入库管理：入库管理模块用于记录物资进入仓库的过程。操作员选择已有物资，填写入库数量、入库单价、来源和备注后提交入库信息。系统保存入库记录，并自动将入库数量累加到对应物资的当前库存中。入库记录支持分页显示和条件查询，便于追溯物资补充情况。

（4）出库管理：出库管理模块用于记录物资领用或消耗过程。操作员选择物资，填写

出库数量、领用人、用途和备注后提交出库信息。系统会先判断当前库存是否充足，若库存不足，则拒绝提交出库申请；若库存充足，则生成待审核出库记录；若库存不足，则拒绝出库并提示库存不足，防止出现负库存。管理员审核通过后，系统正式扣减库存；管理员审核驳回后，库存数量不发生变化。

（5）库存管理：库存管理模块用于查看物资当前库存状态。当前库存由入库和出库业务自动维护，用户无需手动计算库存。系统支持按物资编号、名称、分类和库存状态进行查询。当当前库存小于或等于安全库存时，系统显示低库存预警，提醒管理员及时补充采购。

（6）统计报表：统计报表模块用于对物资数据和出入库数据进行汇总分析。系统可以统计物资总数、库存总量、今日入库数量、今日出库数量、低库存物资数量、分类库存分布、出入库趋势和库存排名等内容，并通过图表展示统计结果，使库存状态更加直观。

（7）智能助手：智能助手模块用于为用户提供系统操作指导和库存业务问答服务。用户可以在系统页面中输入问题，例如如何新增物资、如何提交出库申请、管理员如何审核出库申请、库存不足如何处理、如何查看低库存物资等。前端将用户问题发送给后端，后端调用智谱 API 获取回答后返回前端展示。该模块不直接修改库存数据，只作为辅助说明和操作引导功能，能够提高系统的易用性和智能化程度。

4.2 数据库设计

数据库设计是系统总体设计的重要组成部分。本系统数据库采用 MySQL 8.0，数据库名为 `inventory_db`。根据系统业务需求，数据库主要包含用户表、物资表、入库记录表和出库记录表四类核心数据表。用户表用于保存系统账号与角色信息；物资表用于保存物资基础资料和当前库存；入库记录表用于记录库存增加过程；出库记录表用于记录库存减少过程。

4.2.1 数据库概念结构设计

根据物资库存管理业务流程，用户、物资、入库记录和出库记录之间存在明确的关联关系。系统数据库 ER 图如图 5 所示。

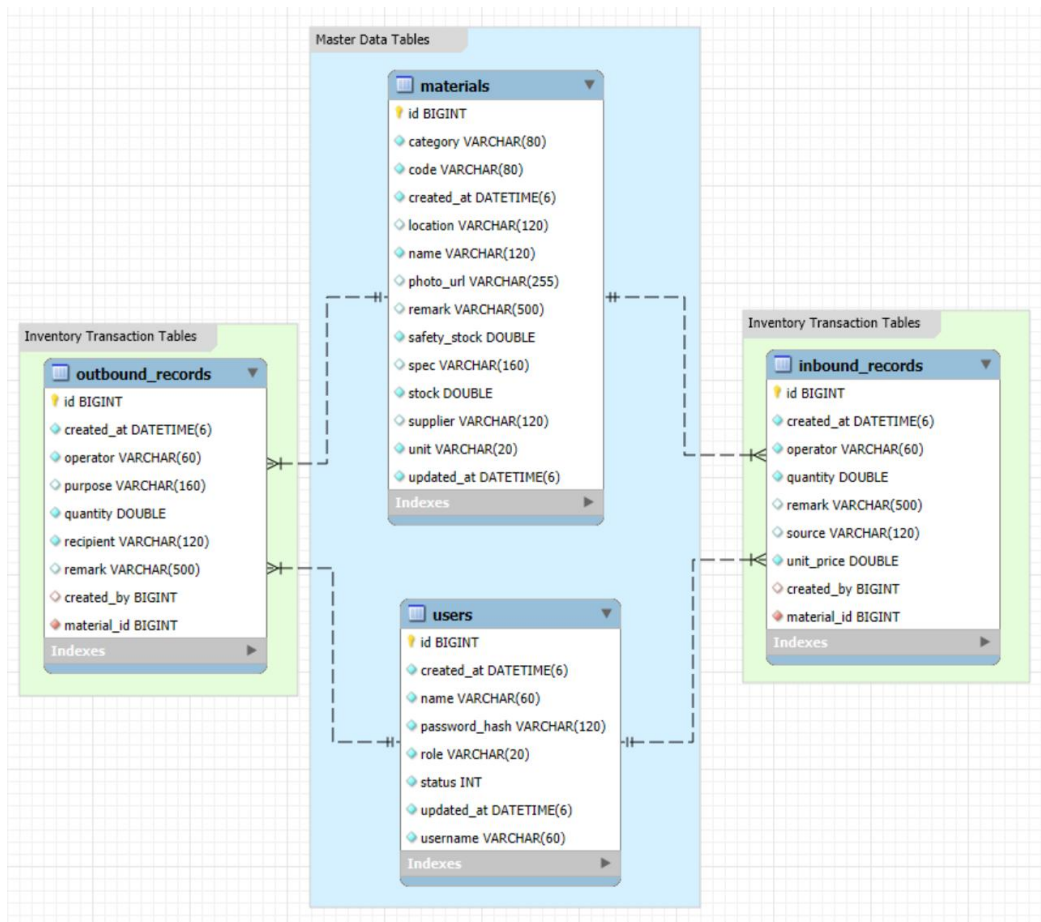


图 5 物资库存管理系统 ER 图

由图 5 可知，系统核心实体关系如下：

- (1) 用户与入库记录是一对多关系。一个用户可以创建多条入库记录，一条入库记录只能对应一个创建用户。
- (2) 用户与出库记录是一对多关系。一个用户可以创建多条出库记录，一条出库记录只能对应一个创建用户。
- (3) 物资与入库记录是一对多关系。一种物资可以对应多条入库记录，一条入库记录只能对应一种物资。
- (4) 物资与出库记录是一对多关系。一种物资可以对应多条出库记录，一条出库记录只能对应一种物资。

通过以上关系设计，系统能够记录每次库存变化对应的物资、数量、时间和操作人，实现库存流转过程的可追溯管理。

4.2.2 数据库逻辑结构设计

根据项目代码和数据库实际结构，系统主要数据表包括 `users`、`materials`、`inbound_records` 和 `outbound_records`。

表 9 系统主要数据表说明

表名	中文名称	主要作用	关联关系
users	用户表	保存系统用户账号、密码密文、姓名、角色、状态等信息，用于登录认证和权限控制	与 inbound_records、outbound_records 通过 created_by 建立关联
materials	物资表	保存物资编号、名称、分类、规格、单位、供应商、库位、库存、安全库存和照片地址等基础信息	与 inbound_records、outbound_records 通过 material_id 建立关联
inbound_records	入库记录表	保存物资入库流水，包括入库数量、单价、来源、操作人和创建时间等信息	关联 users 表和 materials 表
outbound_records	出库记录表	保存物资出库申请和审核流水，包括出库数量、领用人、用途、操作人、审核状态、审核人、审核时间和驳回原因等信息。	关联 users 表和 materials 表

表 10 用户表 users 设计

字段名	数据类型	约束	字段说明
id	BIGINT	主键	用户 ID，系统自动生成，用于唯一标识用户
username	VARCHAR(60)	唯一	用户名，用于系统登录
password_hash	VARCHAR(120)	非空	用户密码密文，采用加密方式存储，不保存明文密码
name	VARCHAR(60)	非空	用户姓名或显示名称
role	VARCHAR(20)	非空	用户角色，用于区分管理员和操作员
status	INT	非空	用户状态，用于表示账号启用或停用
created_at	DATETIME(6)	非空	用户创建时间
updated_at	DATETIME(6)	非空	用户信息更新时间

表 11 物资表 materials 设计

字段名	数据类型	约束	字段说明
id	BIGINT	主键	物资 ID，系统自动生成，用于唯一标识物资记录
code	VARCHAR(80)	唯一	物资编号，每种物资的唯一标识，避免重复录入
name	VARCHAR(120)	非空	物资名称，用于页面展示和条件查询
category	VARCHAR(80)	非空	物资分类，用于分类管理和库存统计
spec	VARCHAR(160)	可空	规格型号，用于描述物资规格信息
unit	VARCHAR(20)	非空	计量单位，如个、件、箱、瓶等
supplier	VARCHAR(120)	可空	供应商信息，用于记录物资来源
location	VARCHAR(120)	可空	库位信息，用于记录物资存放位置
stock	DOUBLE	非空	当前库存，由入库和出库业务自动维护
safety_stock	DOUBLE	非空	安全库存，用于判断低库存预警
photo_url	VARCHAR(255)	可空	物资照片访问地址，用于图片展示
remark	VARCHAR(500)	可空	备注信息，用于补充说明
created_at	DATETIME(6)	非空	物资信息创建时间
updated_at	DATETIME(6)	非空	物资信息更新时间

表 12 入库记录表 inbound_records 设计

字段名	数据类型	约束	字段说明
id	BIGINT	主键	入库记录 ID，系统自动生成
material_id	BIGINT	外键	关联 materials 表，表示入库的具体物资
quantity	DOUBLE	非空	入库数量，必须大于 0
unit_price	DOUBLE	可空	入库单价，用于记录物资入库价格
source	VARCHAR(120)	可空	入库来源，如采购、补货、退还等
operator	VARCHAR(60)	可空	操作人名称，用于页面显示和业务追溯
created_by	BIGINT	外键	关联 users 表，表示创建该入库记录的系统用户

字段名	数据类型	约束	字段说明
remark	VARCHAR(500)	可空	备注信息，用于补充说明入库原因
created_at	DATETIME(6)	非空	入库记录创建时间

表 13 出库记录表 outbound_records 设计

字段名	数据类型	约束	字段说明
id	BIGINT	主键	出库记录 ID，系统自动生成
material_id	BIGINT	外键	关联 materials 表，表示出库的具体物资
quantity	DOUBLE	非空	出库数量，必须大于 0，且不能超过当前库存
recipient	VARCHAR(120)	可空	领用人，用于记录物资领取人员
purpose	VARCHAR(160)	可空	用途，用于说明物资出库原因
operator	VARCHAR(60)	可空	操作人名称，用于页面显示和业务追溯
created_by	BIGINT	外键	关联 users 表，表示创建该出库记录的系统用户
remark	VARCHAR(500)	可空	备注信息，用于补充说明出库情况
created_at	DATETIME(6)	非空	出库记录创建时间
status	VARCHAR(20)	非空	审核状态，如 PENDING、APPROVED、REJECTED
approved_by	BIGINT	可空	审核人 ID，关联 users 表
approved_at	DATETIME(6)	可空	审核时间
reject_reason	VARCHAR(500)	可空	驳回原因或审核备注

数据库设计中，materials.code 应保持唯一，保证每种物资具有唯一标识；users.username 应保持唯一，保证用户登录账号不重复；inbound_records.material_id 和 outbound_records.material_id 关联物资表，保证每条出入库记录都能追溯到具体物资；created_by 关联用户表，保证每条业务记录都能追溯到具体操作用户。

4.3 前后端接口总体设计

本系统采用前后端分离模式，前端通过 Axios 调用后端 RESTful API，后端返回 JSON 格式数据。接口按照业务资源进行划分，主要包括认证接口、用户接口、物资接口、入库接口、出库接口、统计接口和文件上传接口。

认证接口主要用于用户登录和获取当前用户信息；用户接口用于管理员维护系统用户；

物资接口用于物资信息的新增、修改、删除、分页查询和条件查询；入库接口用于保存入库记录并自动增加库存；出库接口用于保存出库记录并自动扣减库存，同时进行负库存拦截；统计接口用于返回库存总览、分类库存、出入库趋势和库存排名等统计数据；文件上传接口用于上传物资照片，并返回图片访问路径。

表 14 系统主要接口设计

模块	接口路径示例	请求方式	功能说明
登录认证	/api/auth/login	POST	用户登录，返回 Token 和用户信息
用户管理	/api/users	GET	分页查询用户列表
用户管理	/api/users	POST	新增系统用户
用户管理	/api/users/{id}	PUT	修改用户信息
用户管理	/api/users/{id}	DELETE	删除用户
物资管理	/api/materials	GET	分页查询物资列表
物资管理	/api/materials	POST	新增物资
物资管理	/api/materials/{id}	PUT	修改物资信息
物资管理	/api/materials/{id}	DELETE	删除物资
入库管理	/api/inbound	GET	查询入库记录
入库管理	/api/inbound	POST	新增入库记录并增加库存
出库管理	/api/outbound	GET	查询出库记录
出库管理	/api/outbound	POST	新增出库记录并扣减库存
统计报表	/api/reports/summary	GET	查询库存总览数据
统计报表	/api/reports/trend	GET	查询出入库趋势数据
文件上传	/api/files/upload	POST	上传物资照片
AI 智能助手	/api/assistant/chat	POST	接收用户输入的问题， 调用智谱 API 获取智能回答

接口设计遵循 RESTful 风格，使用 GET 表示查询，POST 表示新增，PUT 表示修改，DELETE 表示删除。例如，GET /api/materials 用于查询物资列表，POST /api/materials 用于新增物资，PUT /api/materials/{id} 用于修改物资信息，DELETE /api/materials/{id} 用于删除物资。统一的接口风格有利于前后端联调，也便于后续通过 Swagger UI 查看和测试接口。

4.4 权限与安全设计

为了保证系统数据安全，本系统设计了登录认证和角色权限控制机制。用户登录成功后，后端生成 JWT Token，前端在后续请求中通过请求头携带 Token，后端对 Token 进行校验，校验通过后才允许访问业务接口。未登录用户或 Token 失效用户不能访问系统业务页面和后端受保护接口。

系统权限主要分为管理员和操作员两类。管理员拥有系统管理权限，可以进行用户管理、物资管理、入库管理、出库管理、审核出库申请、驳回出库申请、库存查询和统计报表查看；操作员主要负责日常库存业务，可以进行物资查询、入库登记、提交出库申请、查看出库记录、审核状态和统计查看，但不能访问用户管理等敏感功能。通过角色划分，可以避免普通用户误操作系统管理数据，提高系统安全性。

表 15 系统角色权限设计

功能模块	管理员	操作员	未登录用户
登录系统	可访问	可访问	可访问
用户管理	可访问	不可访问	不可访问
物资管理	可访问	可访问部分功能	不可访问
入库管理	可访问	可访问	不可访问
出库管理	可访问	可访问（申请界面）	不可访问
库存查询	可访问	可访问	不可访问
统计报表	可访问	可访问	不可访问
文件上传	可访问	可访问	不可访问
AI 智能助手	可访问	可访问	不可访问

在密码安全方面，系统不保存明文密码，而是使用加密后的密码密文进行存储。用户登录时，系统将输入密码与数据库中的密码密文进行校验，校验通过后才允许登录。这样即使数据库数据泄露，也能降低明文密码暴露的风险。

4.5 数据一致性设计

物资库存管理系统的核心是保证库存数据准确，因此系统在总体设计中重点考虑数据一致性。入库业务和出库业务不能只保存流水记录，还必须同步更新物资当前库存。

入库时，系统先校验物资是否存在和入库数量是否合法，校验通过后保存入库记录，并将该物资当前库存增加相应数量。出库时，系统先校验物资是否存在、出库数量是否合法以及当前库存是否充足。校验通过后，系统生成待审核出库记录，此时不立即扣减库存。管理员审核通过后，系统再次确认该记录状态是否为待审核，并在事务中完成库存扣减和

记录状态更新；若管理员审核驳回，则只更新出库记录状态，不改变库存数量。这样可以避免待审核记录提前扣减库存，也可以防止已驳回记录影响库存数据。

此外，系统通过物资编号唯一性避免物资重复录入，通过用户名唯一性避免账号冲突，通过出入库记录关联物资和用户实现业务追溯。每条入库记录和出库记录都保存物资、数量、时间、操作人和备注信息，为后续统计分析和问题排查提供依据。

五、详细设计

本系统主要包括登录注册模块、用户管理模块、物资信息管理模块、入库管理模块、出库管理模块、统计报表模块以及 UI 与交互设计模块。系统前端采用 Vue3+Element Plus 实现页面展示和交互操作，后端采用 SpringBoot 提供 RESTful API 服务，数据库使用 MySQL 保存用户、物资、入库记录和出库记录等核心数据。

其中，登录注册模块负责用户身份认证和账号创建；用户管理模块负责系统用户维护和权限控制；物资信息管理模块负责物资基础资料维护和照片上传；入库管理模块负责入库登记和库存增加；出库管理模块负责出库登记、库存扣减和负库存拦截；统计报表模块负责库存数据统计和图表展示；UI 与交互设计模块负责系统整体界面布局、操作反馈和用户体验优化。

5.1 登录注册模块

登录注册模块是系统的入口模块，主要完成用户登录认证、账号注册、登录状态保存和未登录拦截等功能。系统前端提供登录页面和注册页面，用户可以在登录页面输入用户名和密码进入系统，也可以通过注册入口创建新账号。登录成功后，系统保存当前用户信息和 Token；当用户未登录或登录状态失效时，系统自动跳转到登录页面。

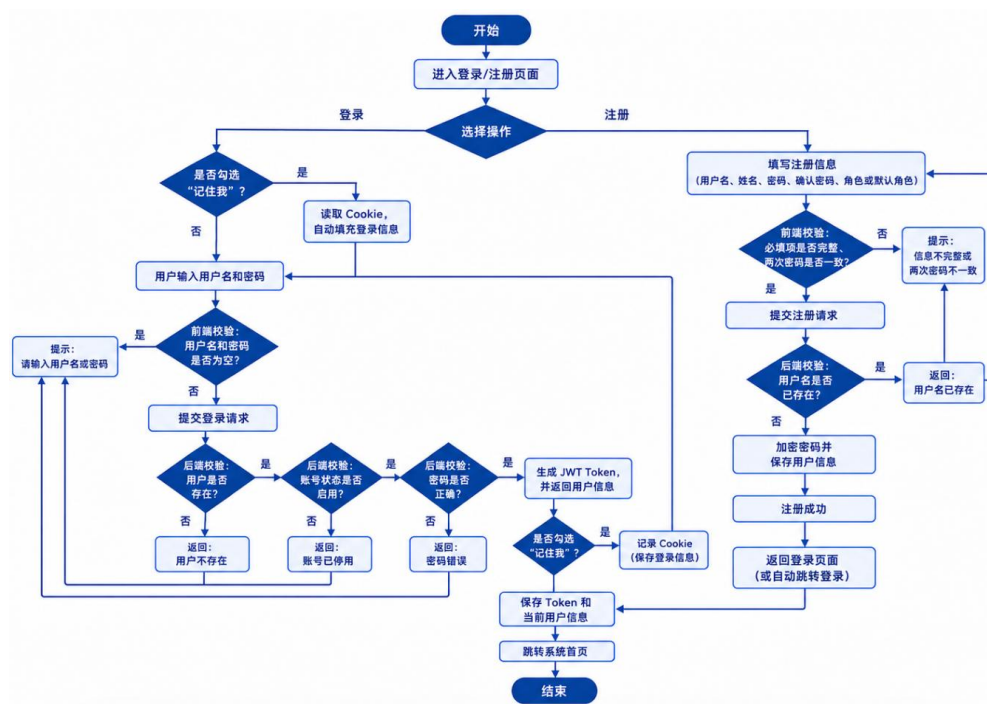


图 6 用户登录注册业务流程图

5.1.1 登录页面设计

登录页面主要由用户名输入框、密码输入框、登录按钮、注册入口和提示信息区域组成。用户输入用户名和密码后，前端首先进行基础表单校验，判断用户名和密码是否为空。若输入为空，则页面直接提示“请输入用户名”或“请输入密码”，不向后端发送请求；若输入完整，则前端通过 Axios 向后端登录接口发送请求。

登录成功后，后端返回 JWT Token、用户 ID、用户名、姓名和角色等信息，前端将这些信息保存到本地存储中，并跳转到系统首页。后续访问物资管理、入库管理、出库管理和统计报表等页面时，前端会在请求头中携带 Token，后端根据 Token 判断用户身份。登录失败时，系统根据后端返回结果提示“用户名或密码错误”“账号已停用”或“登录状态异常”等信息。

用户登录业务处理流程如下：

- (1) 用户进入登录页面，输入用户名和密码。
- (2) 前端对用户名和密码进行必填校验。
- (3) 校验通过后，前端调用登录接口提交账号信息。
- (4) 后端根据用户名查询用户表，判断用户是否存在。
- (5) 若用户不存在，则返回登录失败提示。
- (6) 若用户存在，则判断账号状态是否启用。
- (7) 若账号被停用，则拒绝登录并返回提示。
- (8) 若账号状态正常，则使用 BCrypt 校验输入密码与数据库密码密文是否匹配。
- (9) 密码校验成功后，后端生成 JWT Token，并返回用户信息。
- (10) 前端保存 Token 和用户信息，跳转到系统首页。

5.1.2 注册页面设计

注册功能用于创建系统账号。注册页面主要包括用户名、姓名、密码、确认密码、角色选择和注册按钮等内容。用户填写注册信息后，前端先对输入内容进行校验，包括用户名是否为空、姓名是否为空、密码是否为空、两次输入密码是否一致等。校验通过后，前端向后端注册接口提交数据。

后端接收到注册请求后，首先判断用户名是否已经存在。若用户名重复，则返回“用户名已存在”的提示；若用户名未重复，则对密码进行加密处理，并将用户信息保存到用户表中。注册成功后，系统可以提示用户返回登录页面，也可以根据设计需要直接跳转到登录界面。

注册模块的设计重点是保证用户名唯一和密码安全。用户名作为用户登录账号，不能重复；密码不能以明文形式保存到数据库中，必须经过加密后存储。注册完成后的用户角色可以根据系统设置默认为操作员，也可以由管理员在用户管理模块中调整。

用户注册处理流程如下：

- (1) 用户进入注册页面，填写用户名、姓名、密码、确认密码和角色等信息。
- (2) 前端校验必填项是否完整，并判断两次输入密码是否一致。

- (3) 前端提交注册请求。
- (4) 后端查询用户表，判断用户名是否重复。
- (5) 若用户名重复，则返回注册失败提示。
- (6) 若用户名未重复，则对密码进行加密处理。
- (7) 后端保存用户信息，并设置账号初始状态。
- (8) 注册成功后，前端提示注册成功，并返回登录页面。

5.1.3 登录状态与权限拦截设计

系统采用前后端结合的方式维护登录状态。前端通过本地存储保存 Token 和用户信息，后端通过 JWT 对用户身份进行校验。用户访问系统页面时，前端路由守卫会判断是否存在 Token；用户访问后端接口时，后端安全过滤器会校验 Token 是否有效。

当用户未登录时，系统自动跳转到登录页面；当 Token 过期或无效时，系统清除本地用户信息，并提示用户重新登录。管理员和操作员访问系统时，系统还会根据角色判断其功能权限。例如，管理员可以进入用户管理模块，操作员不能进入用户管理模块。

该模块的核心设计目标是保证系统访问安全，避免未登录用户或权限不足用户访问系统业务数据。

5.2 用户管理模块

用户管理模块主要面向管理员，用于维护系统用户信息和账号状态。管理员登录系统后，可以进入用户管理页面查看系统内所有用户，并对用户进行新增、编辑、删除、启用、停用和条件查询等操作。普通操作员不能访问用户管理模块。

用户管理页面主要由查询区域、用户表格区域、分页区域和操作按钮区域组成。查询区域支持按照用户名、姓名或角色进行筛选；表格区域展示用户 ID、用户名、姓名、角色、状态、创建时间等信息；操作区域提供新增、编辑、删除、启用和停用按钮。

用户管理模块处理流程如下：

- (1) 管理员进入用户管理页面，前端调用用户分页查询接口。
- (2) 后端根据页码、每页条数和查询条件返回用户数据。
- (3) 管理员点击新增按钮，填写用户名、姓名、密码、角色和状态。
- (4) 前端校验必填项，后端校验用户名是否重复。
- (5) 校验通过后，后端对密码加密并保存用户数据。
- (6) 管理员点击编辑按钮，可以修改用户姓名、角色和状态等信息。
- (7) 管理员点击删除按钮时，前端弹出确认提示，确认后调用删除接口。
- (8) 管理员点击启用或停用按钮，后端更新用户状态。

该模块的设计重点是账号唯一性和密码安全性。用户名不允许重复，密码以密文形式保存，页面中不显示明文密码。对于系统管理员账号，应避免误删或误停用，保证系统后续能够正常维护。

5.3 物资信息管理模块

物资信息管理模块是系统的基础业务模块，负责维护物资基础资料。入库管理、出库管理和统计报表都依赖物资基础数据，因此物资信息的完整性和准确性直接影响系统整体业务。

物资信息管理页面主要包括查询区域、物资列表区域、分页区域、物资新增按钮、编辑按钮、删除按钮和照片上传功能。物资列表展示物资编号、名称、分类、规格型号、单位、供应商、库位、当前库存、安全库存、照片和备注等信息。

物资管理模块处理流程如下：

- (1) 用户进入物资管理页面，前端调用物资列表接口，分页加载物资数据。
- (2) 用户可以按照物资编号、物资名称、分类等条件进行查询。
- (3) 新增物资时，用户填写物资编号、名称、分类、规格、单位、供应商、库位、安全库存等信息。
- (4) 用户可以上传物资照片，系统将图片路径保存到物资信息中。
- (5) 前端对必填字段进行校验，后端校验物资编号是否唯一。
- (6) 若物资编号重复，则系统提示“物资编号已存在”。
- (7) 若校验通过，则后端保存物资信息。
- (8) 编辑物资时，用户可以修改物资基础信息和照片。
- (9) 删除物资时，系统弹出二次确认提示，避免误删数据。

物资信息管理模块的核心设计是“唯一编号”和“照片上传”。物资编号作为每种物资的唯一标识，能够避免同一物资重复录入；物资照片用于辅助用户识别物资，提高系统使用体验。

5.4 入库管理模块

入库管理模块用于记录物资进入仓库的业务过程。当采购、补货或退还物资时，操作员需要在系统中登记入库信息。系统保存入库记录后，会自动增加对应物资的当前库存。

入库管理页面主要包括入库记录列表、查询条件、新增入库按钮和入库表单。入库表单中需要选择物资，并填写入库数量、入库单价、来源和备注等信息。

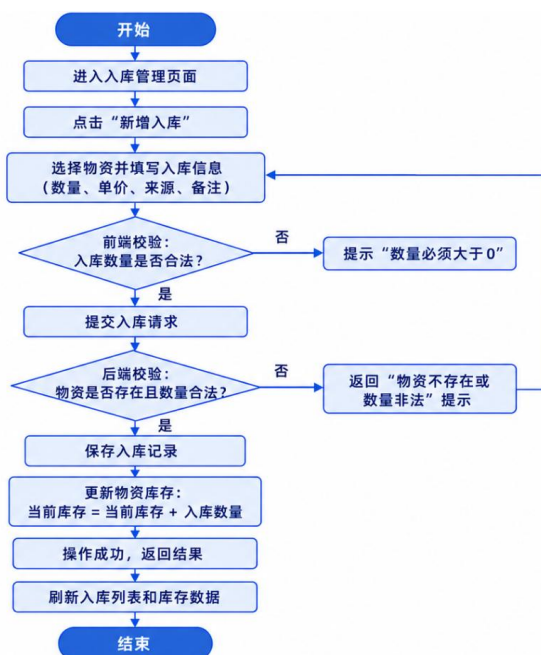


图 7 物资入库业务流程图

入库模块处理流程如下：

- (1) 操作员进入入库管理页面，系统分页显示历史入库记录。
- (2) 操作员点击新增入库按钮，打开入库表单。
- (3) 操作员选择需要入库的物资，并填写入库数量、入库单价、来源和备注。
- (4) 前端校验入库数量是否为空、是否大于 0。
- (5) 后端判断物资是否存在，并再次校验入库数量是否合法。
- (6) 校验通过后，后端保存入库记录。
- (7) 系统将该物资当前库存增加对应入库数量。
- (8) 操作成功后，前端刷新入库记录列表和库存数据。

入库模块的关键设计是保证入库记录和库存增加保持一致。系统不能只保存入库记录而不更新库存，也不能只增加库存而不保存入库流水。入库数量必须大于 0，非法数量需要直接拦截。

5.5 出库管理模块

出库管理模块用于记录物资离开仓库的业务过程。当部门、人员或项目领用物资时，操作员需要在系统中提交出库申请。与入库模块相比，出库模块不仅需要对出库数量进行校验，还需要判断当前库存是否充足，防止出现负库存。为了进一步提高物资领用流程的规范性，本系统在出库管理模块中增加了管理员审核机制。操作员提交出库申请后，系统不会立即扣减库存，而是先生成待审核出库记录；只有管理员审核通过后，系统才正式扣减对应物资库存；如果管理员审核驳回，则库存数量不发生变化。

出库管理页面主要包括出库记录列表、查询条件、新增出库申请按钮、出库申请表单和审核操作按钮。操作员可以填写出库申请信息，查看自己提交的出库记录及审核状态；

管理员除可以查看出库记录外，还可以对待审核的出库申请进行审核通过或审核驳回操作。出库表单中需要选择物资，并填写出库数量、领用人、用途和备注等信息。出库记录列表中需要显示物资名称、出库数量、领用人、用途、申请人、申请时间、审核状态、审核人和审核时间等信息，便于后期追溯物资领用过程。

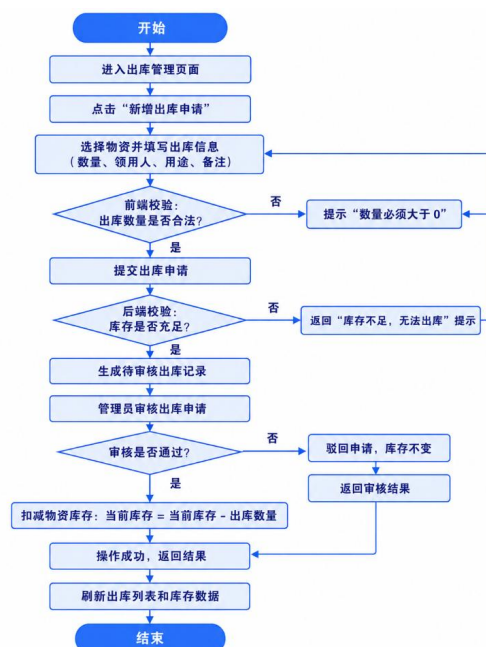


图 8 物资出库业务流程图

出库模块处理流程如下：

- (1) 操作员进入出库管理页面，系统分页显示历史出库记录，并显示每条记录的审核状态。
- (2) 操作员点击新增出库申请按钮，打开出库申请表单。
- (3) 操作员选择需要出库的物资，并填写出库数量、领用人、用途和备注等信息。
- (4) 前端对表单数据进行基础校验，判断出库数量是否为空、是否大于 0，领用人等必要信息是否填写完整。
- (5) 后端接收到出库申请后，查询该物资是否存在，并获取该物资当前库存数量。
- (6) 后端判断出库数量是否大于当前库存。
- (7) 若库存不足，则系统拒绝提交出库申请，并返回“库存不足”提示，出库记录不保存，库存数量也不发生变化。
- (8) 若库存充足，则系统保存出库申请记录，并将该记录的审核状态设置为“待审核”，此时不立即扣减物资库存。
- (9) 管理员进入出库管理页面，查看待审核的出库申请记录。
- (10) 管理员根据实际情况对出库申请进行审核。若审核驳回，则系统将出库记录状态更新为“已驳回”，库存数量不发生变化。
- (11) 若管理员审核通过，则后端再次校验该出库记录是否处于待审核状态，并在事务中扣减对应物资库存，同时将出库记录状态更新为“已通过”。

（12）审核完成后，前端刷新出库记录列表、库存数据和统计报表数据，使页面展示结果与数据库保持一致。

出库模块的关键设计包括负库存拦截和管理员审核机制。系统必须在后端进行库存判断，不能只依赖前端校验。当库存不足时，系统不允许生成出库申请，也不允许扣减库存。当库存充足时，系统只生成待审核记录，库存暂时不变。只有管理员审核通过后，系统才正式扣减库存；若审核驳回，则库存保持不变。通过这种方式，系统既能够防止负库存问题，又能够避免操作员直接出库造成误操作，提高了物资领用流程的安全性、规范性和可追溯性。

5.6 统计报表模块

统计报表模块用于对系统中的物资数据、入库数据和出库数据进行汇总分析，并通过图表形式展示。该模块主要用于查看库存总览、分类库存、出入库趋势、库存排名和低库存预警等信息。

统计报表页面主要包括数据卡片区和图表展示区。数据卡片用于展示物资总数、库存总量、今日入库数量、今日出库数量和低库存物资数量；图表展示区用于展示分类库存统计、出入库趋势分析和库存排名等内容。

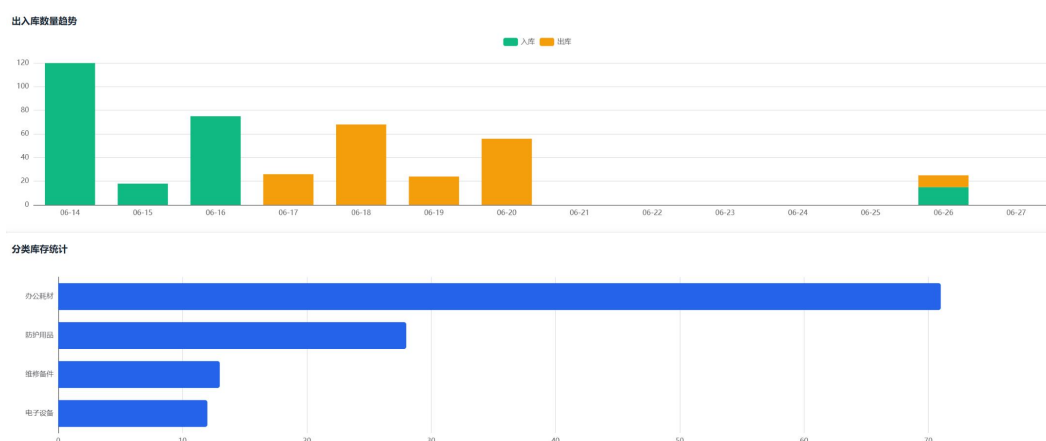


图 9 库存报表示例图

统计模块处理流程如下：

- （1）用户进入统计报表页面后，前端调用统计接口。
- （2）后端根据物资表统计物资总数、库存总量和低库存数量。
- （3）后端根据入库记录表统计今日入库数量和入库趋势。
- （4）后端根据出库记录表统计今日出库数量和出库趋势。
- （5）后端按照物资分类统计库存分布。
- （6）后端按照库存数量生成库存排名数据。
- （7）前端接收统计结果后，将数据渲染到数据卡片和 ECharts 图表中。

统计报表模块的设计重点是保证统计数据真实可靠。统计结果应来自物资表、入库记录表和出库记录表，而不是静态模拟数据。通过图表展示，用户能够更加直观地了解库存

状态和物资流转情况。

5.7 UI 与交互设计

UI 与交互设计主要用于提升系统的易用性和美观性。本系统整体采用后台管理系统布局，页面结构主要包括左侧菜单栏、顶部用户信息区和右侧内容展示区。左侧菜单用于切换系统功能模块，顶部区域显示系统名称和当前用户信息，内容区展示各模块的表格、表单、弹窗和图表。

系统 UI 设计遵循以下原则：

（1）界面统一：系统页面整体采用统一的色彩风格、按钮样式、表格样式和弹窗样式，保证视觉一致性。

（2）布局清晰：登录注册页面突出输入区域和操作按钮，业务管理页面采用查询区、操作区、表格区和分页区的结构，便于用户理解。

（3）操作直观：新增、编辑、删除、入库和出库等常用操作均放置在明显位置，用户可以快速完成业务处理。

（4）反馈明确：系统对登录失败、保存成功、删除成功、库存不足、编号重复、权限不足等情况提供明确提示。

（5）防止误操作：删除用户、删除物资等危险操作需要二次确认，避免用户误删重要数据。

（6）图表美观：统计报表页面通过数据卡片和 ECharts 图表展示库存数据，使管理人员能够直观掌握库存变化情况。

（7）异常友好：当接口请求失败、登录状态失效或数据加载异常时，系统应提示用户重新操作或重新登录，而不是直接显示空白页面。

通过以上 UI 与交互设计，系统能够在保证功能完整的基础上，提高用户操作效率和使用体验，使物资库存管理过程更加清晰、友好和直观。

5.8 AI 智能助手模块详细设计

AI 智能助手模块是本系统在基础库存管理功能上的扩展模块，主要用于为用户提供系统操作指导和库存业务问答服务。管理员和操作员在使用系统过程中，如果对物资管理、入库管理、出库申请、出库审核、低库存预警或统计报表查看方式存在疑问，可以通过智能助手输入问题，系统会返回对应的文字说明。

AI 智能助手页面主要以悬浮按钮或聊天弹窗的形式展示。用户点击“AI 助手”按钮后，系统弹出聊天窗口。用户在输入框中输入问题后，前端将问题内容提交到后端智能助手接口。后端接收到问题后，将用户问题和系统业务背景组织成提示词，并调用智谱 API 获取回答。智谱 API 返回结果后，后端将回答内容返回前端，前端在聊天窗口中展示。

AI 智能助手处理流程如下：

（1）用户点击系统页面中的“AI 助手”按钮，打开聊天窗口。

（2）用户在输入框中输入问题，例如“如何提交出库申请”“管理员如何审核出库申

请”“库存不足怎么办”等。

(3) 前端对输入内容进行校验，若输入为空，则提示用户输入问题。

(4) 前端调用后端/api/assistant/chat 接口，并将用户问题发送给后端。

(5) 后端接收到问题后，组织系统提示词，限定 AI 助手主要围绕物资库存管理系统进行回答。

(6) 后端调用智谱 API 获取智能回答。

(7) 后端将 AI 返回的回答结果封装后返回给前端。

(8) 前端在聊天窗口中显示回答内容。

(9) 若接口调用失败，系统显示“智能助手暂时不可用，请稍后重试”等友好提示。

AI 智能助手模块的关键设计是后端代理调用。由于智谱 API Key 属于敏感信息，不能直接写在前端代码中，因此本系统由后端统一读取 API Key 并调用智谱 API，前端只与本系统后端接口通信。这样既能够实现智能问答功能，也能够降低密钥泄露风险。

六、 系统实现

系统采用前后端分离方式实现，前端负责页面展示、表单交互、路由跳转、接口调用和图表渲染；后端负责接口服务、身份认证、权限校验、业务处理、数据库访问和库存计算。重点从工程结构、前端接口封装、登录注册实现、用户管理实现、物资管理实现、入库出库实现、统计报表实现、安全认证实现和文件上传实现等方面展开说明。

6.1 前端工程创建与依赖引入

前端项目基于 Vue3 和 Vite 创建，使用 Element Plus 作为 UI 组件库，使用 Axios 完成接口请求，使用 Vue Router 完成页面路由管理，使用 ECharts 完成库存统计图表展示。前端工程主要包括页面视图、接口请求封装、路由配置、公共组件和全局样式等内容。

前端工程创建及依赖安装命令如下：

```
npm create vite@latest frontend
cd frontend
npm install
npm install element-plus axios vue-router echarts
npm run dev
```

前端主要目录结构如下：

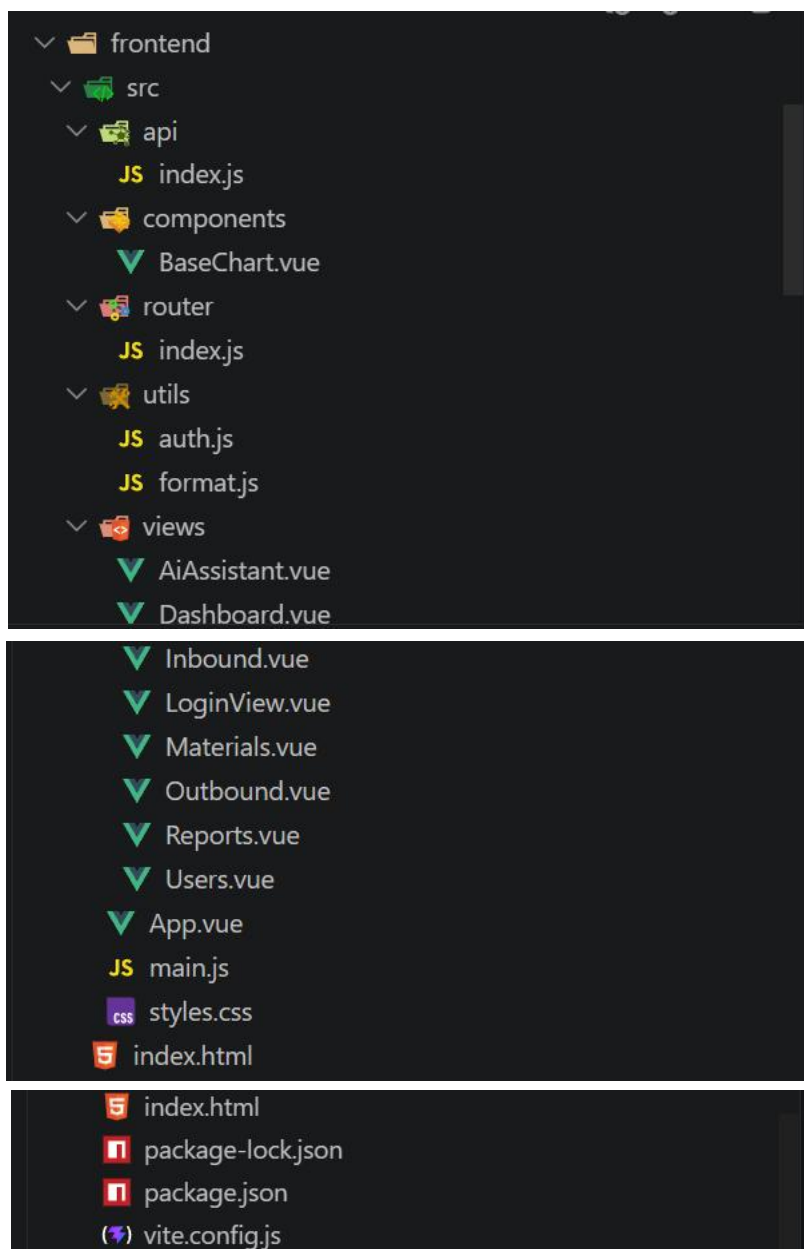


图 10 前端主要目录结构

在 main.js 中引入 Element Plus 和路由配置，使系统能够使用 Element Plus 提供的表格、表单、按钮、弹窗、上传、分页等组件。

1. `import { createApp } from 'vue';`
2. `import ElementPlus from 'element-plus';`
3. `import zhCn from 'element-plus/es/locale/lang/zh-cn';`
4. `import * as ElementPlusIconsVue from '@element-plus/icons-vue';`
5. `import 'element-plus/dist/index.css';`
6. `import './styles.css';`
7. `import App from './App.vue';`
8. `import router from './router';`
- 9.

```

10.  const app = createApp(App);
11.
12.  for (const [key, component] of Object.entries(ElementPlusIconsVue)) {
13.    app.component(key, component);
14.  }
15.
16.  app.use(ElementPlus, { locale: zhCn });
17.  app.use(router);
18.  app.mount('#app');

```

6.2 Axios 接口请求封装实现

为了统一处理前端请求，本系统对 Axios 进行封装。前端请求后端接口时，如果本地存在 JWT Token，则自动将 Token 添加到请求头中；如果后端返回未认证状态，则清除本地登录信息并跳转到登录页面。这样可以避免每个页面重复编写 Token 携带和异常处理代码。

Axios 拦截器核心代码如下：

```

1.  api.interceptors.request.use((config) => {
2.    if (authState.token) {
3.      config.headers.Authorization = `Bearer ${authState.token}`;
4.    }
5.    return config;
6.  });
7.
8.  api.interceptors.response.use(
9.    (response) => response.data,
10.   (error) => {
11.     const message = error.response?.data?.message || error.message || '请求失败';
12.     if (error.response?.status === 401) {
13.       clearAuth();
14.       if (location.pathname !== '/login') location.href = '/login';
15.     }
16.     ElMessage.error(message);
17.     return Promise.reject(new Error(message));
18.   }
19. );

```

通过请求拦截器，系统实现了 Token 的自动携带；通过响应拦截器，系统实现了统一

错误提示和登录失效处理。这样前端页面只需要关注具体业务逻辑，不需要重复处理通用请求逻辑。

6.3 前端路由守卫实现

系统前端使用 Vue Router 进行页面跳转管理。登录页和注册页为公共页面，未登录用户可以访问；物资管理、入库管理、出库管理、统计报表和用户管理页面属于业务页面，需要登录后才能访问。其中，用户管理页面属于管理员功能，需要进一步判断当前用户是否具有管理员权限。

路由守卫核心代码如下：

```
1. router.beforeEach((to) => {
2.   if (!to.meta.public && !authState.token) return '/login';
3.   if (to.meta.admin && !isAdmin()) return '/';
4.   if (to.path === '/login' && authState.token) return '/';
5.   return true;
6. });
```

该代码主要完成三类控制：第一，未登录用户访问业务页面时自动跳转到登录页面；第二，非管理员访问管理员页面时跳转到系统首页；第三，已登录用户访问登录页面时自动跳转到首页。通过该实现，前端完成了基本页面级权限控制。

6.4 登录注册功能实现

登录注册模块是系统入口模块，主要完成用户登录认证、用户注册、登录状态保存和 Token 维护。登录时，前端提交用户名和密码，后端校验用户是否存在、账号是否启用以及密码是否正确；校验通过后，后端生成 JWT Token 并返回用户信息。注册时，系统校验用户名是否重复，校验通过后对密码进行加密存储。

前端登录请求封装如下：

```
1. export const authApi = {
2.   login: (payload) => api.post('/auth/login', payload),
3.   register: (payload) => api.post('/auth/register', payload),
4.   me: () => api.get('/auth/me')
5. };
```

登录提交逻辑写在 frontend/src/views/LoginView.vue 文件中的 submitLogin() 方法中。该方法首先校验账号、密码和验证码是否填写正确，校验通过后调用 authApi.login() 请求后端登录接口。后端返回 Token 和用户信息后，前端调用 setAuth(token, user) 保存登录状态，并跳转到系统首页。

```
1. async function submitLogin() {
2.   if (!loginForm.username) {
3.     ElMessage.warning('账号不能为空');
```

```

4.     return;
5.   }
6.   if (!loginForm.password) {
7.     ElMessage.warning('密码不能为空');
8.     return;
9.   }
10.  if (!loginForm.captcha) {
11.    ElMessage.warning('验证码不能为空');
12.    return;
13.  }
14.  if (loginForm.captcha !== captcha.value) {
15.    ElMessage.error('验证码错误');
16.    refreshCaptcha();
17.    return;
18.  }
19.
20.  loading.value = true;
21.  try {
22.    const { token, user } = await authApi.login({
23.      username: loginForm.username,
24.      password: loginForm.password
25.    });
26.    setAuth(token, user);
27.    router.push('/');
28.  } finally {
29.    loading.value = false;
30.  }
31. }

```

注册提交逻辑写在 `frontend/src/views/LoginView.vue` 文件中的 `submitSignup()` 方法中。注册表单字段包括姓名、邮箱、密码和确认密码。前端先校验字段是否完整，并判断两次密码是否一致，校验通过后调用 `authApi.register()` 请求后端注册接口。注册成功后，系统返回 `Token` 和用户信息，前端保存登录状态并自动进入系统首页。

```

1.  async function submitSignup() {
2.    if (!signupForm.name) {
3.      ElMessage.warning('Name 不能为空');
4.      return;

```

```

5.    }
6.    if (!signupForm.email) {
7.        ElMessage.warning('Email 不能为空');
8.        return;
9.    }
10.   if (!signupForm.password) {
11.       ElMessage.warning('Password 不能为空');
12.       return;
13.   }
14.   if (!signupForm.confirmPassword) {
15.       ElMessage.warning('Confirm Password 不能为空');
16.       return;
17.   }
18.   if (signupForm.password !== signupForm.confirmPassword) {
19.       ElMessage.error('两次密码输入不一致');
20.       return;
21.   }
22.
23.   signupLoading.value = true;
24.   try {
25.       const { token, user } = await authApi.register({
26.           name: signupForm.name,
27.           email: signupForm.email,
28.           password: signupForm.password
29.       });
30.       setAuth(token, user);
31.       ElMessage.success('注册成功, 已自动登录');
32.       router.push('/');
33.   } finally {
34.       signupLoading.value = false;
35.   }
36. }

```

后端登录逻辑主要包括用户查询、账号状态判断、密码校验和 Token 生成。系统使用 BCrypt 校验密码，数据库中不保存明文密码。

```

1.   @PostMapping("/login")
2.   public LoginResponse login(@RequestBody LoginRequest request) {

```

```

3.     User user = userRepository.findByUsername(request.username())
4.         .orElseThrow(() -> ApiException.badRequest("用户名或密码错误"));
5.
6.     if (user.getStatus() == 0) {
7.         throw ApiException.forbidden("账号已停用");
8.     }
9.
10.    if (!passwordEncoder.matches(request.password(), user.getPasswordHash())) {
11.        throw ApiException.badRequest("用户名或密码错误");
12.    }
13.
14.    String token = jwtUtil.generateToken(user.getUsername(), user.getRole());
15.    return new LoginResponse(token, UserView.from(user));
16. }

```

后端注册逻辑如下：

```

1.    @PostMapping("/register")
2.    public UserView register(@RequestBody RegisterRequest request) {
3.        if (userRepository.existsByUsername(request.username())) {
4.            throw ApiException.conflict("用户名已存在");
5.        }
6.
7.        User user = new User();
8.        user.setUsername(request.username());
9.        user.setName(request.name());
10.       user.setPasswordHash(passwordEncoder.encode(request.password()));
11.       user.setRole("OPERATOR");
12.       user.setStatus(1);
13.
14.       return UserView.from(userRepository.save(user));
15.   }

```

6.5 主页面与菜单实现

系统主页面采用后台管理系统常见布局，包括左侧菜单栏、顶部用户信息区域和右侧内容展示区域。左侧菜单用于切换统计报表、物资管理、入库管理、出库管理和用户管理等页面；顶部区域显示系统名称和当前登录用户信息；内容区域根据路由加载不同页面。

菜单核心代码如下：


```

1.   const menus = computed(() => {
2.     const base = [
3.       { path: '/', label: '工作台', icon: House },
4.       { path: '/materials', label: '物资信息管理', icon: Goods },
5.       { path: '/inbound', label: '入库管理', icon: Box },
6.       { path: '/outbound', label: '出库管理', icon: TopRight },
7.       { path: '/reports', label: '分析报表', icon: DataAnalysis }
8.     ];
9.     if (isAdmin()) base.push({ path: '/users', label: '用户管理', icon: User });
10.    return base;
11.  });

```

菜单栏通过 `computed` 动态生成菜单数据，系统默认展示工作台、物资信息管理、入库管理、出库管理和分析报表等基础模块。当 `isAdmin()` 判断当前用户为管理员时，才会向菜单数组中追加“用户管理”入口。管理员可以看到用户管理模块，操作员登录后只能看到日常业务相关模块。这样既保证了页面简洁，也实现了前端层面的权限区分。

6.6 用户管理功能实现

用户管理模块主要用于管理员维护系统用户信息。前端页面以表格形式展示用户列表，支持分页查询、新增用户、编辑用户、删除用户、启用和停用等操作。用户查询条件主要包括用户名、姓名和角色等内容。

用户查询接口封装如下：

```

1.   export const usersApi = {
2.     list: (params) => api.get('/users', { params }),
3.     create: (payload) => api.post('/users', payload),
4.     update: (id, payload) => api.put(`/users/${id}`, payload),
5.     remove: (id) => api.delete(`/users/${id}`)
6.   };

```

用户列表页面核心结构如下：

```

1.   <el-table :data="userList" border>
2.     <el-table-column prop="username" label="用户名" />
3.     <el-table-column prop="name" label="姓名" />
4.     <el-table-column prop="role" label="角色" />
5.     <el-table-column prop="status" label="状态" />
6.     <el-table-column prop="createdAt" label="创建时间" />
7.     <el-table-column label="操作">
8.       <template #default="{ row }">

```

```

9.      <el-button size="small" @click="openEdit(row)">编辑</el-button>
10.     <el-button size="small" type="danger" @click="removeUser(row)">
11.       删除
12.     </el-button>
13.   </template>
14. </el-table-column>
15. </el-table>

```

后端用户管理接口通过分页方式返回用户列表，避免一次性加载过多数据。

```

1.   @GetMapping
2.   public Page<UserView> listUsers(
3.     @RequestParam(defaultValue = "") String keyword,
4.     @RequestParam(defaultValue = "0") int page,
5.     @RequestParam(defaultValue = "10") int size) {
6.     return userService.listUsers(keyword, page, size);
7.   }

```

用户管理实现中需要注意密码安全。新增用户时，后端需要对密码进行加密处理；查询用户列表时，不返回密码明文，避免敏感信息泄露。

6.7 物资信息管理功能实现

物资信息管理模块用于维护物资基础数据。前端页面提供物资列表、查询条件、新增按钮、编辑按钮、删除按钮和照片上传功能。后端负责物资编号唯一性校验、物资数据保存、物资信息更新和分页查询。

物资接口封装如下：

```

1.   export const materialsApi = {
2.     list: (params) => api.get('/materials', { params }),
3.     categories: () => api.get('/materials/categories/options'),
4.     create: (payload) => api.post('/materials', payload),
5.     update: (id, payload) => api.put(`/materials/${id}`, payload),
6.     remove: (id) => api.delete(`/materials/${id}`),
7.     uploadPhoto: (id, file) => {
8.       const formData = new FormData();
9.       formData.append('photo', file);
10.      return api.post(`/materials/${id}/photo`, formData, {
11.        headers: { 'Content-Type': 'multipart/form-data' }
12.      });
13.    }

```

```
14.    };
```

物资实体类核心字段如下：

```
1.    @Entity
2.    @Table(name = "materials")
3.    public class Material {
4.        @Id
5.        @GeneratedValue(strategy = GenerationType.IDENTITY)
6.        private Long id;
7.        @Column(nullable = false, unique = true, length = 80)
8.        private String code;
9.        @Column(nullable = false, length = 120)
10.       private String name;
11.       private String category;
12.       private String spec;
13.       private String unit;
14.       private String supplier;
15.       private String location;
16.       private Double stock;
17.       private Double safetyStock;
18.       private String photoUrl;
19.       private String remark;
20.       private LocalDateTime createdAt;
21.       private LocalDateTime updatedAt;
22.    }
```

新增物资时，后端需要先校验物资编号是否重复。若编号已经存在，则返回错误提示；若编号未重复，则保存物资信息。当前库存为空时，系统将其初始化为 0。

```
1.    public Material createMaterial(Material material) {
2.        if (materialRepository.existsByCode(material.getCode())) {
3.            throw ApiException.conflict("物资编号已存在");
4.        }
5.
6.        if (material.getStock() == null) {
7.            material.setStock(0.0);
8.        }
9.
10.       material.setCreatedAt(LocalDateTime.now());
```

```

11.     material.setUpdatedAt(LocalDateTime.now());
12.
13.     return materialRepository.save(material);
14. }

```

物资管理是入库、出库和统计报表的基础，系统通过物资编号唯一性保证物资数据不重复，通过库存字段记录当前库存数量，通过安全库存字段支持低库存预警判断。

6.8 入库管理功能实现

入库管理模块用于记录物资进入仓库的业务过程。前端提交入库请求后，后端校验入库数量是否合法，并判断物资是否存在；校验通过后，系统保存入库记录，并将对应物资库存增加入库数量。

前端入库接口封装如下：

```

1.     export const inboundApi = {
2.         list: (params) => api.get('/inbound', { params }),
3.         create: (payload) => api.post('/inbound', payload),
4.         remove: (id) => api.delete(`/inbound/${id}`)
5.     };

```

入库业务核心实现如下：

```

1.     @Transactional
2.     public Map<String, Object> createInbound(InboundRequest request, User currentUser) {
3.         double quantity = normalizePositive(request.quantity(), "入库数量必须大于 0");
4.
5.         Material material = materialRepository.findByIdForUpdate(request.materialId())
6.             .orElseThrow(() -> ApiException.notFound("物资不存在"));
7.
8.         material.setStock(material.getStock() + quantity);
9.         materialRepository.save(material);
10.
11.         InboundRecord record = new InboundRecord();
12.         record.setMaterial(material);
13.         record.setQuantity(quantity);
14.         record.setOperator(currentUser.getName());
15.         record.setCreatedBy(currentUser);
16.
17.         return inboundView(inboundRecordRepository.save(record));
18.     }

```

该方法使用@Transactional 注解保证入库记录保存和库存增加处于同一事务中。如果入库记录保存失败或库存更新失败，事务会回滚，避免出现库存数量和入库记录不一致的问题。方法中通过 normalizePositive 校验入库数量必须大于 0，通过 findByIdForUpdate 查询并锁定对应物资，减少并发修改库存带来的数据异常。

6.9 出库管理功能实现

出库管理模块用于记录物资离开仓库的业务过程。与入库功能相比，出库功能不仅需要进行出库数量校验和库存充足性校验，还需要结合管理员审核机制，避免操作员提交出库后直接扣减库存造成误操作。本系统中，操作员提交出库信息后，系统先判断当前库存是否充足，若库存不足则直接返回提示；若库存充足，则生成一条“待审核”的出库申请记录，此时不立即扣减库存。只有管理员审核通过后，系统才正式扣减对应物资库存；若管理员审核驳回，则库存数量不发生变化。

前端出库接口封装如下：

```
1. export const outboundApi = {
2.   list: (params) => api.get('/outbound', { params }),
3.   create: (payload) => api.post('/outbound', payload),
4.   approve: (id) => api.post(`/outbound/${id}/approve`),
5.   reject: (id, payload) => api.post(`/outbound/${id}/reject`, payload),
6.   remove: (id) => api.delete(`/outbound/${id}`)
7. };
```

其中，list 方法用于分页查询出库记录，create 方法用于操作员提交出库申请，approve 方法用于管理员审核通过出库申请，reject 方法用于管理员驳回出库申请，remove 方法用于删除出库记录。前端页面根据当前登录用户角色控制审核按钮的显示，普通操作员只能提交出库申请和查看审核状态，管理员可以对待审核记录进行通过或驳回操作。

操作员提交出库申请的核心实现如下：

```
1. @Transactional
2. public Map<String, Object> createOutbound(OutboundRequest request, User currentUser) {
3.   double quantity = normalizePositive(request.quantity(), "出库数量必须大于 0");
4.   Material material = materialRepository.findByIdForUpdate(request.materialId())
5.     .orElseThrow(() -> ApiException.notFound("物资不存在"));
6.   if (material.getStock() < quantity) {
7.
8.     throw ApiException.conflict("当前库存不足，无法提交出库申请");
9.
10.  }
```

```

11.     OutboundRecord record = new OutboundRecord();
12.     record.setMaterial(material);
13.     record.setQuantity(quantity);
14.     record.setRecipient(request.recipient());
15.     record.setPurpose(request.purpose());
16.     record.setRemark(request.remark());
17.     record.setOperator(currentUser.getName());
18.     record.setCreatedBy(currentUser);
19.     record.setStatus("PENDING");
20.     return outboundView(outboundRecordRepository.save(record));
21.
22. }

```

该方法用于提交出库申请。系统首先校验出库数量是否大于 0，然后通过 `findByIdForUpdate` 查询并锁定对应物资记录，防止并发出库导致库存计算错误。接着，系统判断当前库存是否足够。如果库存不足，则直接抛出异常并返回“库存不足”提示；如果库存充足，则保存出库申请记录，并将审核状态设置为 `PENDING`，表示待审核。此时系统不会扣减物资库存，库存数量保持不变。

管理员审核通过出库申请的核心实现如下：

```

1.     @Transactional
2.     public Map<String, Object> approveOutbound(Long id, User currentUser) {
3.         OutboundRecord record = outboundRecordRepository.findById(id)
4.             .orElseThrow(() -> ApiException.notFound("出库记录不存在"));
5.         if (!"PENDING".equals(record.getStatus())) {
6.             throw ApiException.conflict("该出库申请已处理，不能重复审核");
7.         }
8.         Material material = materialRepository.findByIdForUpdate(record.getMaterial().getId())
9.             .orElseThrow(() -> ApiException.notFound("物资不存在"));
10.        if (material.getStock() < record.getQuantity()) {
11.            throw ApiException.conflict("当前库存不足，无法审核通过");
12.        }
13.        material.setStock(material.getStock() - record.getQuantity());
14.        materialRepository.save(material);
15.        record.setStatus("APPROVED");
16.        record.setApprovedBy(currentUser);
17.        record.setApprovedAt(LocalDateDateTime.now());
18.        return outboundView(outboundRecordRepository.save(record));

```

18. }

该方法用于管理员审核通过出库申请。系统首先根据出库记录 ID 查询对应记录，并判断该记录是否仍处于待审核状态。如果记录已经通过或驳回，则不允许重复审核。随后，系统再次查询并锁定对应物资记录，重新判断库存是否充足。库存充足时，系统在同一个事务中完成库存扣减和审核状态更新，将出库记录状态改为 APPROVED，并保存审核人和审核时间。这样可以保证库存扣减和审核记录更新同时成功，避免数据不一致。

管理员驳回出库申请的核心实现如下：

```
1. @Transactional
2. public Map<String, Object> rejectOutbound(Long id, RejectRequest request, User currentUser) {
3.     OutboundRecord record = outboundRecordRepository.findById(id).orElseThrow(() ->
        ApiException.notFound("出库记录不存在"));
4.     if (!"PENDING".equals(record.getStatus())) {
5.         throw ApiException.conflict("该出库申请已处理，不能重复审核");
6.     }
7.     record.setStatus("REJECTED");
8.     record.setApprovedBy(currentUser);
9.     record.setApprovedAt(LocalDate.now());
10.    record.setRejectReason(request.rejectReason());
11.    return outboundView(outboundRecordRepository.save(record));
12. }
```

该方法用于管理员驳回出库申请。系统首先查询出库记录，并判断该记录是否处于待审核状态。若记录已经被处理，则不允许重复审核。驳回时，系统只更新出库记录的审核状态、审核人、审核时间和驳回原因，不修改物资库存。这样可以保证被驳回的出库申请不会影响库存数量。

出库管理功能的关键实现点是“提交申请、审核通过后扣减库存、审核驳回后库存不变”。系统在提交出库申请时进行库存校验，防止库存不足的申请进入审核流程；在管理员审核通过时再次校验库存，并使用事务控制库存扣减和状态更新，避免库存数据异常。通过这种方式，系统既能够实现负库存拦截，又增加了管理员审核机制，使物资出库流程更加规范、安全和可追溯。

6.10 物资照片上传实现

物资照片上传用于保存物资图片，提高物资识别效率。系统上传图片时，后端首先判断文件是否为空，然后校验文件类型是否属于允许上传的图片格式。校验通过后，后端将图片保存到上传目录，并把图片访问路径保存到物资记录中。

物资照片上传核心代码如下：


```

1.  @Transactional
2.  public String uploadPhoto(Long id, MultipartFile file) {
3.      Material material = get(id);
4.      if (file == null || file.isEmpty()) {
5.          throw ApiException.badRequest("请选择要上传的图片");
6.      }
7.      String contentType = file.getContentType() == null ? "" :
8.          file.getContentType().toLowerCase(Locale.ROOT);
9.      if (!IMAGE_TYPES.contains(contentType)) {
10.         throw ApiException.badRequest("只能上传 JPG、PNG、WEBP 或 GIF 图片");
11.     }
12.     Files.createDirectories(uploadPath);
13.
14.     String filename = "material-" + System.currentTimeMillis() + "-" +
15.         UUID.randomUUID() + extension;
16.     file.transferTo(uploadPath.resolve(filename).normalize());
17.     String url = "/uploads/" + filename;
18.     material.setPhotoUrl(url);
19.     materialRepository.save(material);
20.     return url;
21. }

```

该实现方式没有将图片二进制内容直接存入数据库，而是将图片保存到服务器目录中，数据库只保存图片访问路径。这样可以减少数据库存储压力，同时方便前端通过 `photoUrl` 字段加载物资图片。

6.11 统计报表功能实现

统计报表模块用于对物资数据、入库数据和出库数据进行汇总分析。后端通过 SQL 聚合查询统计物资总数、库存总量、低库存数量、分类库存分布和出入库趋势等数据；前端通过 ECharts 将统计结果渲染为数据卡片、柱状图、折线图等图表。

前端统计接口封装如下：

```

1.  export const reportsApi = {
2.      dashboard: () => api.get('/reports/dashboard'),
3.      movement: (params) => api.get('/reports/movement', { params }),
4.      stockByCategory: () => api.get('/reports/stock-by-category'),
5.      lowStock: () => api.get('/reports/low-stock')
6.  };

```


库存总览统计 SQL 如下：

```
1.  SELECT
2.      COUNT(*) AS material_count,
3.      COALESCE(SUM(stock), 0) AS total_stock,
4.      COALESCE(SUM(CASE WHEN stock <= safety_stock THEN 1 ELSE 0 END), 0) AS low_stock_count
5.  FROM materials;
```

出入库趋势统计 SQL 如下：

```
1.  SELECT DATE(created_at) AS day, COALESCE(SUM(quantity), 0) AS quantity
2.  FROM inbound_records
3.  WHERE DATE(created_at) >= ?
4.  GROUP BY DATE(created_at);
```

前端接收到后端返回的统计数据后，使用 ECharts 初始化图表并设置图表参数。示例代码如下：

```
1.  const chart = echarts.init(document.getElementById('categoryChart'));
2.
3.  chart.setOption({
4.      title: { text: '分类库存统计' },
5.      tooltip: {},
6.      xAxis: {
7.          type: 'category',
8.          data: categoryNames
9.      },
10.     yAxis: {
11.         type: 'value'
12.     },
13.     series: [{
14.         name: '库存数量',
15.         type: 'bar',
16.         data: categoryValues
17.     }]
18. });
```

统计报表实现中，数据来源于真实业务表，包括 materials、inbound_records 和 outbound_records。这样能够保证报表数据与实际库存业务保持一致。

6.12 AI 智能助手功能实现

AI 智能助手功能用于为用户提供系统操作问答和库存管理辅助说明。前端通过聊天弹窗收集用户问题，并调用后端智能助手接口；后端接收到问题后，调用智谱 API 获取回答，再将结果返回前端展示。

前端 AI 助手接口封装如下：

```
1. export const assistantApi = {  
2.   chat: (payload) => api.post('/assistant/chat', payload)  
3. };
```

前端页面中，用户输入问题后，系统调用 `assistantApi.chat()` 方法发送请求。请求参数主要包含用户输入的 `message` 内容。后端返回回答后，前端将回答追加到聊天列表中。

后端接口设计如下：

```
1. @PostMapping("/chat")  
2. public Map<String, Object> chat(@RequestBody AssistantRequest request) {  
3.   return assistantService.chat(request.message());  
4. }
```

后端 `Service` 层主要完成三项工作：第一，对用户输入内容进行非空校验；第二，将用户问题与系统业务背景组合成提示词；第三，调用智谱 API 并解析返回结果。为了保证系统安全，智谱 API Key 不写入前端代码，而是通过后端配置文件或环境变量读取。

AI 助手提示词中会限定回答范围，例如系统只回答与物资库存管理、物资信息维护、入库管理、出库申请、出库审核、低库存预警、统计报表和系统使用方法相关的问题。对于与系统无关的问题，可以提示用户重新输入与系统业务相关的问题。通过这种方式，可以让 AI 助手更贴合本系统业务场景。

该模块的实现并不直接修改数据库中的库存数据，也不会替代管理员审核操作，而是作为系统的辅助功能，为用户提供操作说明和业务规则解释，从而提高系统易用性和智能化程度。

6.13 Spring Security + JWT 身份鉴权实现

系统后端使用 Spring Security + JWT 实现接口级身份鉴权。登录和注册接口允许匿名访问，物资管理、入库管理、出库管理、统计报表和用户管理等接口需要登录后访问。前端请求接口时在请求头中携带 `oken`，后端通过过滤器解析 Token 并判断用户身份。

Spring Security 核心配置如下：

```
1. @Bean  
2. public SecurityFilterChain securityFilterChain(HttpSecurity http, JwtAuthenticationFilter jwtFilter) throws Exception {  
3.   http.csrf(AbstractHttpConfigurer::disable)  
4.   .cors(cors -> cors.configurationSource(corsConfigurationSource()))
```

```

5.         .sessionManagement(session -> session.sessionCreationPolicy(SessionCreation
Policy.STATELESS))
6.         .exceptionHandling(errors -> errors
7.         .authenticationEntryPoint((request, response, error) -> {
8.             response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
9.             response.setContentType("application/json;charset=UTF-8");
10.            response.getWriter().write("{\"message\":\"请先登录\"}");
11.        })
12.        .accessDeniedHandler((request, response, error) -> {
13.            response.setStatus(HttpServletResponse.SC_FORBIDDEN);
14.            response.setContentType("application/json;charset=UTF-8");
15.            response.getWriter().write("{\"message\":\"没有权限执行该操作\"}");
16.        })
17.    )
18.    .authorizeHttpRequests(auth -> auth
19.        .requestMatchers(HttpMethod.POST, "/api/auth/login").permitAll()
20.        .requestMatchers(HttpMethod.POST, "/api/auth/register").permitAll()
21.        .requestMatchers(
22.            "/api/health",
23.            "/api/docs",
24.            "/api/docs/**",
25.            "/api/swagger-ui/**",
26.            "/v3/api-docs/**",
27.            "/uploads/**"
28.        ).permitAll()
29.        .anyRequest().authenticated()
30.    )
31.    .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
32.    return http.build();
33. }

```

该配置中，登录、注册、健康检查、接口文档和图片访问路径允许匿名访问，其余接口需要认证。系统采用无状态会话模式，不依赖服务端 Session，而是通过 JWT Token 识别用户身份。

6.14 Swagger 接口文档实现

为了方便前后端联调，系统集成 Swagger UI 自动生成接口文档。后端启动后，可以通

过 Swagger UI 查看接口路径、请求方式、请求参数和响应结果。系统主要接口包括认证接口、用户接口、物资接口、入库接口、出库接口、统计接口和文件上传接口。

Swagger 依赖配置如下：

```
1. <dependency>
2.   <groupId>org.springdoc</groupId>
3.   <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
4.   <version>2.6.0</version>
5. </dependency>
```

接口注解示例如下：

```
1. @PostMapping
2. public Material create(@Valid @RequestBody MaterialRequest request) {
3.     return materialService.create(request);
4. }
```

通过 Swagger UI，开发过程中可以直接查看接口说明并进行接口测试，有利于前后端联调和后期文档整理。

七、 系统测试和问题分析

本章主要对物资库存管理系统的主要功能进行运行测试，并展示系统运行效果。测试内容包括用户登录与注册、用户权限控制、用户管理、物资信息管理、物资照片上传、入库管理、出库管理、库存不足拦截、统计报表展示、接口文档访问等功能。同时，本章结合开发过程中遇到的主要问题，分析问题产生原因，并说明对应的解决方法。

7.1 测试环境与系统启动

本系统采用前后端分离方式开发，前端使用 Vue3+Vite+Element Plus，后端使用 SpringBoot 3.3.5+Spring Security+JWT+Spring Data JPA，数据库使用 MySQL 8.0。系统测试前，需要保证 MySQL 数据库服务已经启动，并配置好数据库连接密码。

表 16 系统测试环境表

测试项	配置说明
操作系统	Windows 10 / Windows 11
前端框架	Vue3 + Vite + Element Plus
后端框架	SpringBoot 3.3.5
数据库	MySQL 8.0
持久化框架	Spring Data JPA

测试项	配置说明
安全认证	Spring Security + JWT
图表组件	ECharts
前端访问地址	http://localhost:5173/login
后端访问地址	http://localhost:3000
接口文档地址	http://localhost:3000/api/docs

本项目根目录中已经配置了统一启动命令，前端和后端可以通过 `npm run dev` 同时启动。首次运行前，需要先安装依赖：

`npm install`

`npm run install:all`

在 Windows PowerShell 中设置 MySQL 数据库密码环境变量：

`$env:DB_PASSWORD="MySQL root 密码"`

然后在项目根目录执行：

`npm run dev`

执行后，系统会同时启动后端 SpringBoot 服务和前端 Vite 服务。前端默认运行在 5173 端口，后端默认运行在 3000 端口。系统启动完成后，在浏览器中输入：

`http://localhost:5173/login`

即可进入系统登录页面。

```
> material-inventory-system@1.0.0 dev
> concurrently "npm --prefix backend run dev" "npm --prefix frontend run dev"

[0]
[0] > inventory-springboot-backend@1.0.0 dev
[0] > set DEBUG=&& mvn -DskipTests package && set DEBUG=&& java -jar target/inventory-backend-1.0.0.jar
[0]
[1]
[1] > inventory-frontend@1.0.0 dev
[1] > vite --host 0.0.0.0
[1]
[1]
[1] VITE v7.3.5 ready in 453 ms
[1]
[1] → Local:   http://localhost:5173/
[1] → Network: http://192.168.11.1:5173/
[1] → Network: http://192.168.226.1:5173/
[1] → Network: http://192.168.43.160:5173/
[0] [INFO] Scanning for projects...
[0] [INFO]
[0] [INFO] -----< com.inventory:inventory-backend >-----
[0] [INFO] Building inventory-backend 1.0.0
[0] [INFO]    from pom.xml
[0] [INFO] -----[ jar ]-----
[0] [INFO]
[0] [INFO] --- resources:3.3.1:resources (default-resources) @ inventory-backend ---
[0] [INFO] Copying 1 resource from src/main/resources to target/classes
[0] [INFO] Copying 1 resource from src/main/resources to target/classes
[0] [INFO]
[0] [INFO] --- compiler:3.13.0:compile (default-compile) @ inventory-backend ---
[0] [INFO] Nothing to compile - all classes are up to date.
[0] [INFO]
[0] [INFO] --- resources:3.3.1:testResources (default-testResources) @ inventory-backend ---
[0] [INFO] skip non existing resourceDirectory D:\AAA\workspace\code\backend\src\test\resources
[0] [INFO]
[0] [INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ inventory-backend ---
[0] [INFO] No sources to compile
[0] [INFO]
[0] [INFO] --- surefire:3.2.5:test (default-test) @ inventory-backend ---
[0] [INFO] Tests are skipped.
[0] [INFO]
[0] [INFO] --- jar:3.4.2:jar (default-jar) @ inventory-backend ---
```

图 11 系统启动成功截图

7.2 用户登录与注册测试

用户登录是系统入口功能。系统提供管理员账号和操作员账号，其中管理员默认账号密码为 admin/admin123，操作员默认账号密码为 operator/operator123。登录页面包括账号、密码、验证码、记住我、忘记密码、联系管理员等功能。用户必须输入账号、密码和验证码，校验通过后才能登录系统。



欢迎回来!

高效管理库存、入库出库、数据报表
让物资管理更简单

SIGN IN

用户登录

账号

请输入账号

密码

请输入密码

验证码

请输入验证码

5199 换一张

☐ 记住我

忘记密码?

登录

联系管理员

没有账号? [创建账号](#)

图 12 用户登录页面

登录测试时，输入管理员账号 admin、密码 admin123，并填写页面中显示的验证码，点击“登录”按钮。系统会先在前端校验账号、密码和验证码是否为空，若验证码错误，则提示“验证码错误”并刷新验证码；若输入正确，则调用后端登录接口。后端校验用户状态和密码成功后返回 JWT Token 和用户信息，前端保存登录状态并跳转到系统首页。

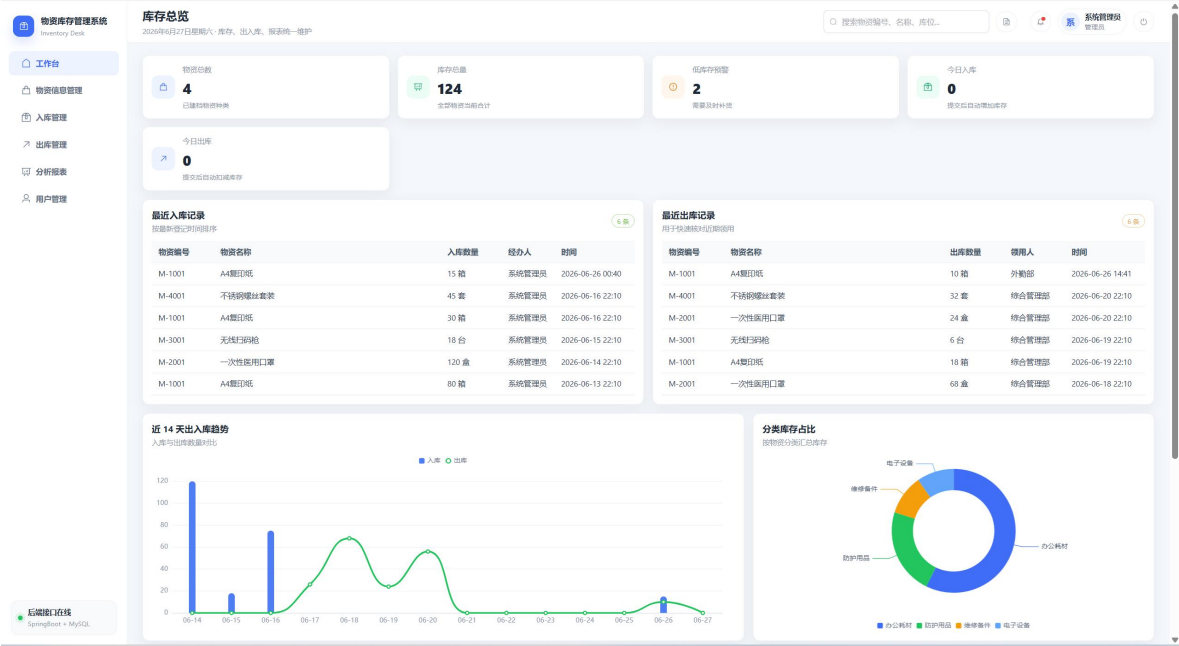


图 13 管理员登录成功页面

以下是几种登录失败界面：

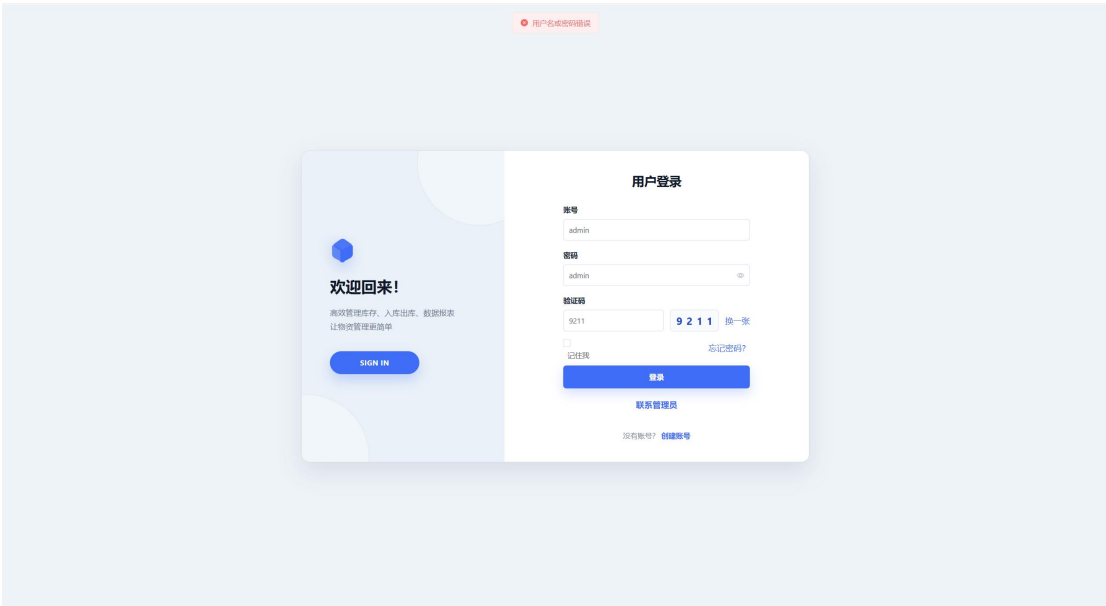


图 14 登录用户名/密码错误

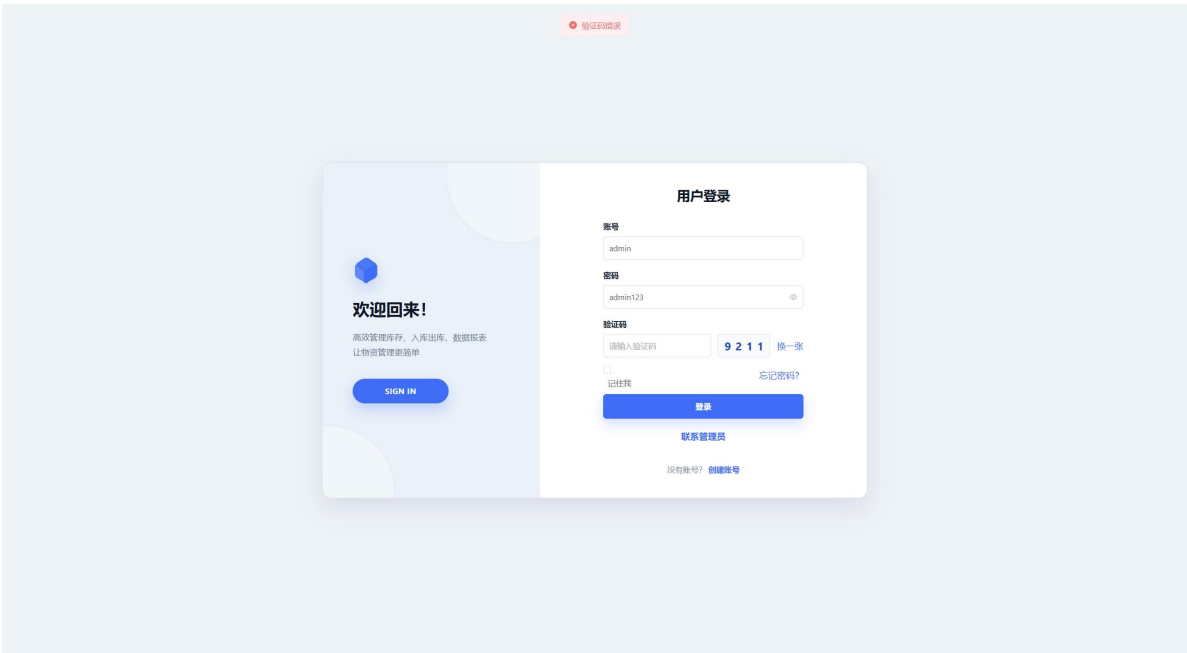


图 15 验证码错误

注册功能用于普通用户自助创建账号。用户点击“创建账号”后进入注册表单，填写姓名、邮箱、密码和确认密码。系统首先校验字段是否完整，并判断两次密码是否一致；校验通过后，前端调用注册接口。注册成功后，系统会自动保存 Token 和用户信息，并进入系统首页。自助注册用户默认角色为操作员。

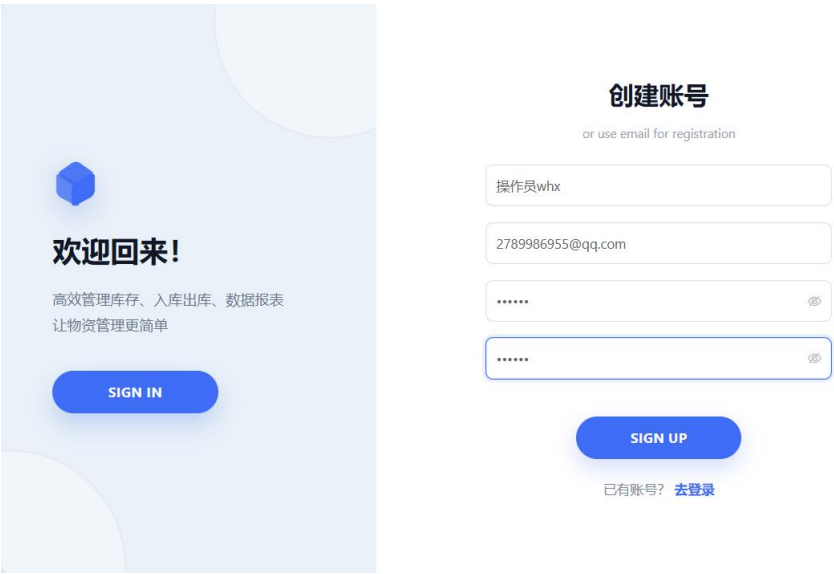


图 16 用户注册页面



图 17 用户注册成功

以下是几种注册失败界面：

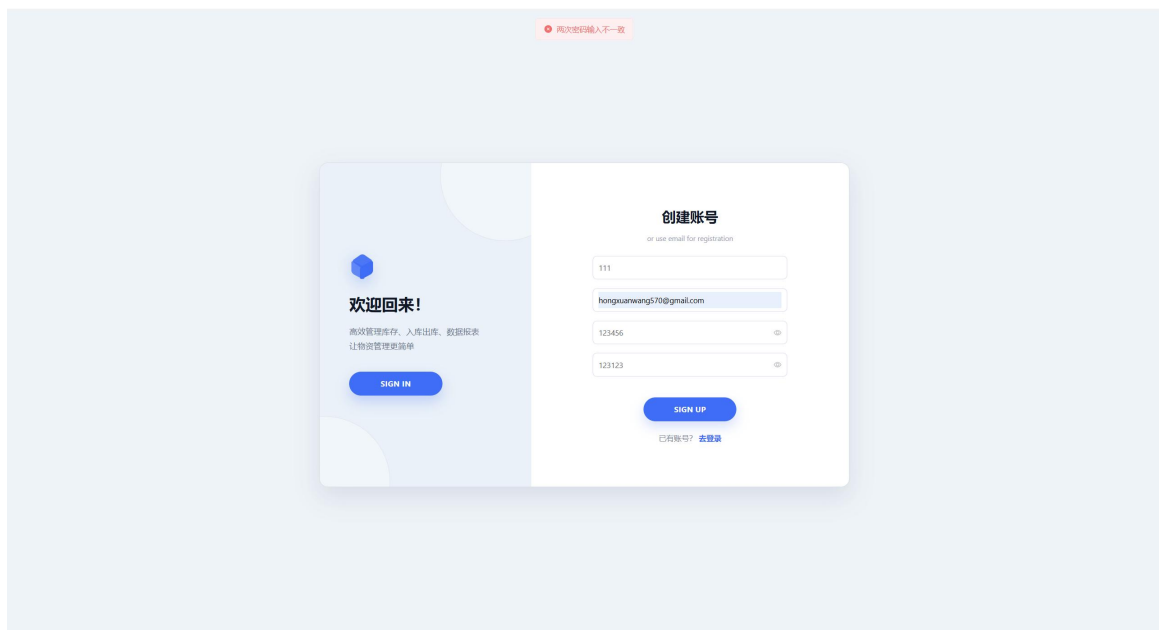


图 18 两次密码输入不一样

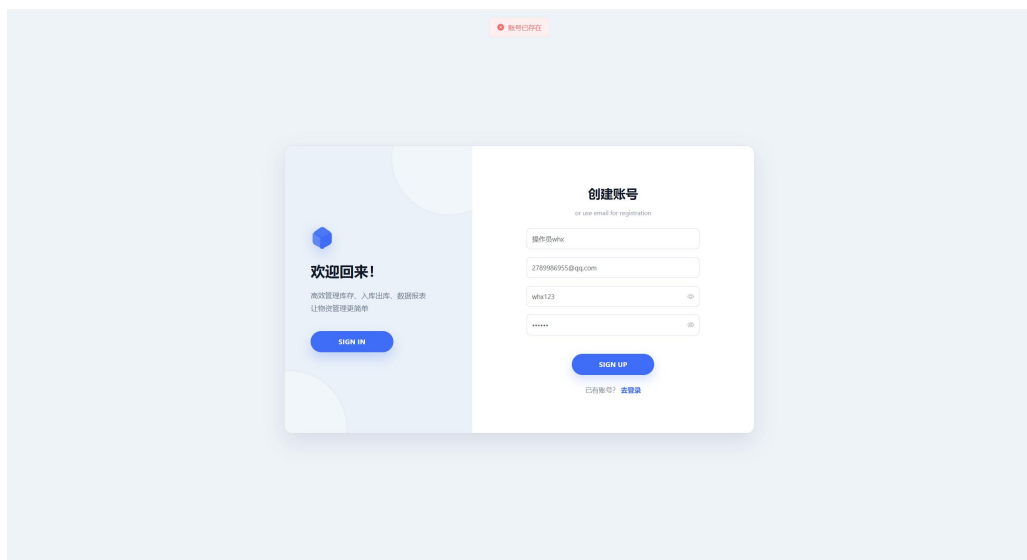


图 19 账号存在时会注册失败

7.3 权限控制测试

本系统设置了管理员和操作员两类角色。管理员拥有完整系统权限，可以访问工作台、物资信息管理、入库管理、出库管理、分析报表和用户管理等模块；操作员只能访问日常业务相关功能，不能进入用户管理模块。系统前端通过 `isAdmin()` 判断当前用户角色，只有角色为 `admin` 时才显示“用户管理”菜单。

管理员登录后，左侧菜单中可以看到“用户管理”入口。



图 20 管理员菜单页面

操作员登录后，左侧菜单中只显示工作台、物资信息管理、入库管理、出库管理和分析报表，不显示用户管理模块。



图 21 操作员菜单页面

7.4 用户管理功能测试

用户管理功能仅管理员可使用，主要用于维护系统用户信息。管理员进入“用户管理”页面后，可以查看用户列表，并根据用户名或姓名进行关键字查询。页面支持新增用户、编辑用户、删除用户、设置角色和启停账号等操作。

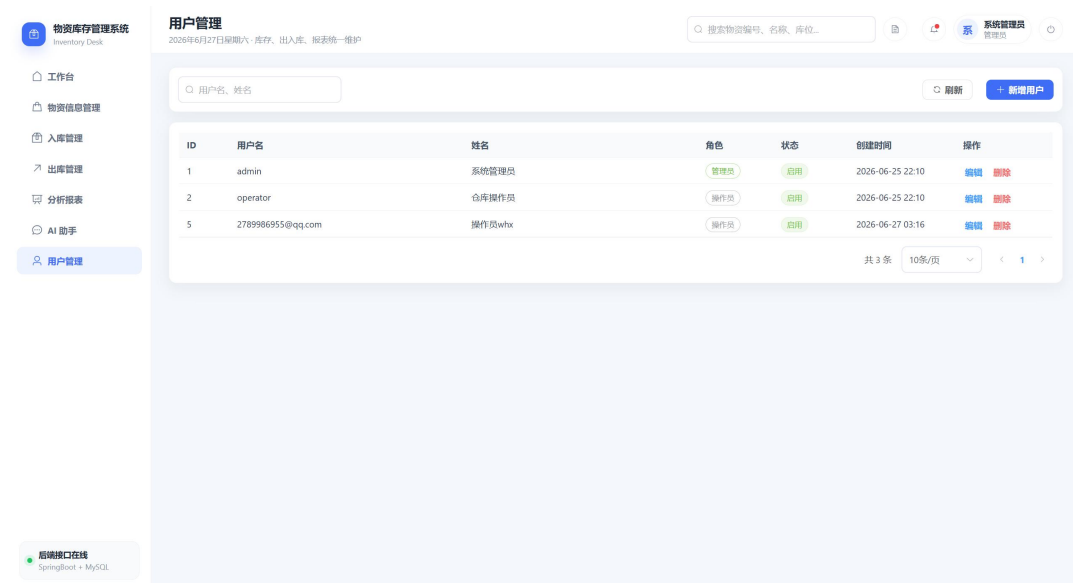


图 22 用户管理页面

新增用户时，管理员需要填写用户名、姓名、密码、角色和状态。其中新增用户密码不能为空，且密码长度至少为 6 位。编辑用户时，用户名不能修改，密码可以留空，表示不修改原密码。系统保存用户时会在后端进行校验，并使用 BCrypt 对密码进行加密存储。

A modal window titled "新增用户" (Add User) with a close button (X) in the top right corner. It contains five input fields: "用户名" (Username) with the value "operator2", "姓名" (Name) with the value "操作员2", "密码" (Password) with the value "123456" and a toggle icon, "角色" (Role) with a dropdown menu showing "操作员", and "状态" (Status) with a toggle switch set to "启用" (Enabled). At the bottom right, there are two buttons: "取消" (Cancel) and "保存" (Save).

图 23 新增用户页面

A modal window titled "编辑用户" (Edit User) with a close button (X) in the top right corner. It contains five input fields: "用户名" (Username) with the value "operator2", "姓名" (Name) with the value "操作员2", "密码" (Password) with the value "不修改可留空", "角色" (Role) with a dropdown menu showing "操作员", and "状态" (Status) with a toggle switch set to "停用" (Disabled). At the bottom right, there are two buttons: "取消" (Cancel) and "保存" (Save).

图 24 编辑用户页面

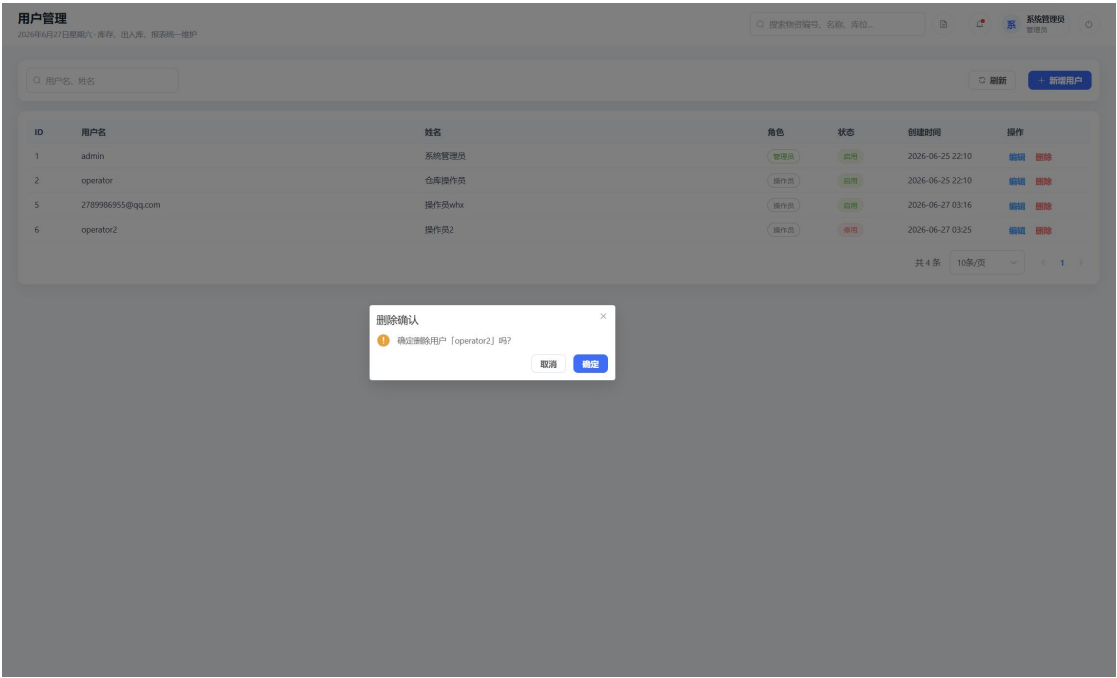


图 25 删除用户

7.5 物资信息管理功能测试

物资信息管理模块用于维护物资基础档案，包括物资编号、物资名称、分类、规格型号、单位、供应商、库位、安全库存、当前库存、照片和备注等信息。物资页面支持按物资编号、物资名称、分类和库存状态进行筛选，库存状态包括正常、低库存和库存不足等。

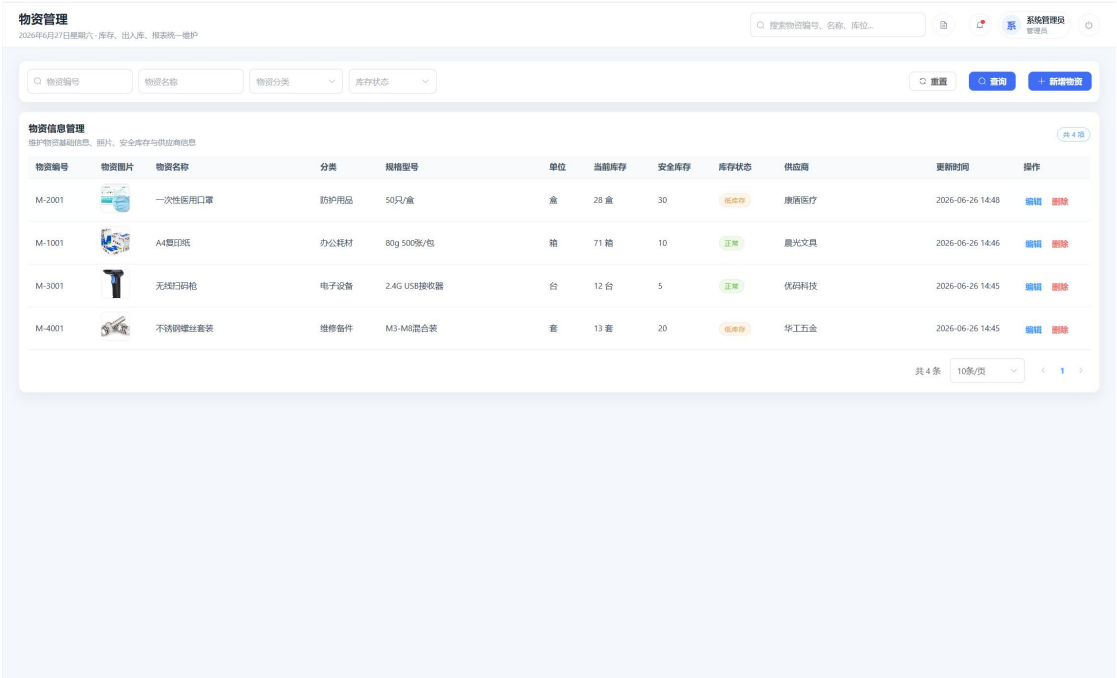


图 26 物资信息管理页面

新增物资时，系统要求填写物资编号、物资名称、分类和单位等必要信息。物资编号是系统中每种物资的唯一标识，不能重复。新增物资成功后，当前库存默认为 0，后续库

注等基础信息。如果修改物资编号，系统会再次校验编号是否重复。删除物资时，如果该物资已经存在入库或出库记录，系统会拦截删除，避免历史出入库流水失去关联。

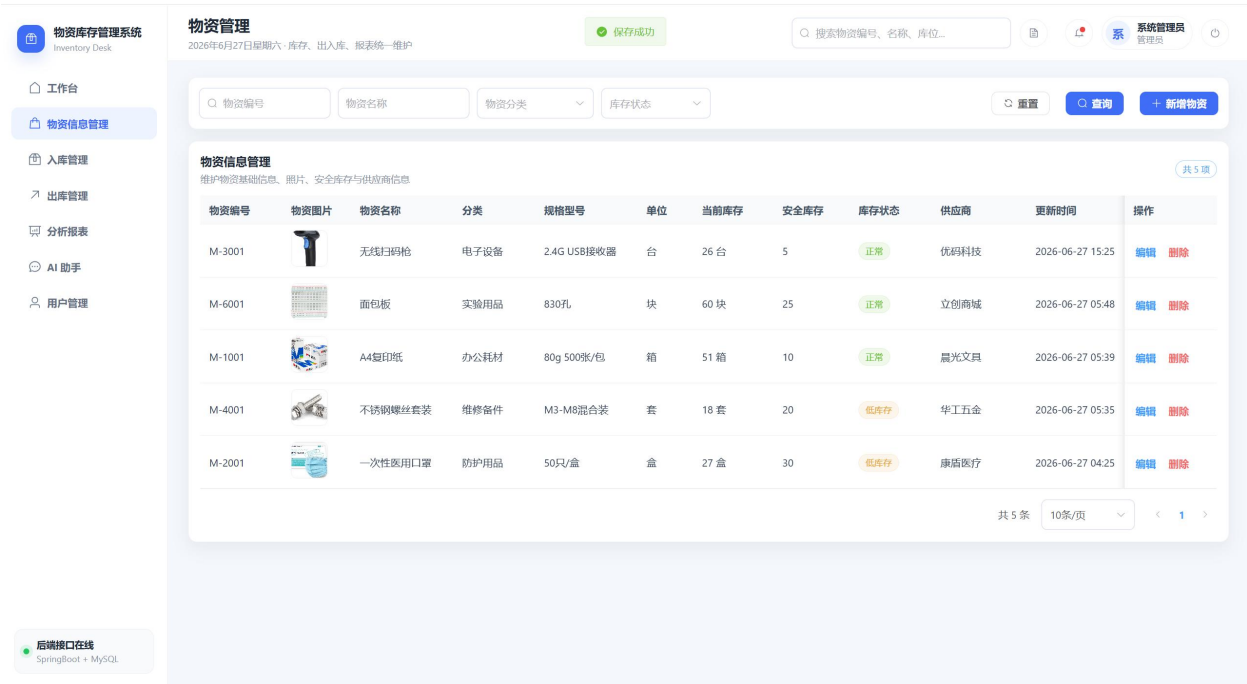


图 29 编辑物资页面（修改物资编号）

7.6 物资照片上传功能测试

物资照片上传功能用于为物资绑定图片，提高物资识别效率。在新增或编辑物资时，用户可以选择本地图片文件上传。系统上传成功后，后端将图片保存到 uploads 目录，并把图片访问路径保存到物资信息中。前端物资列表中会显示物资图片，并支持图片预览。

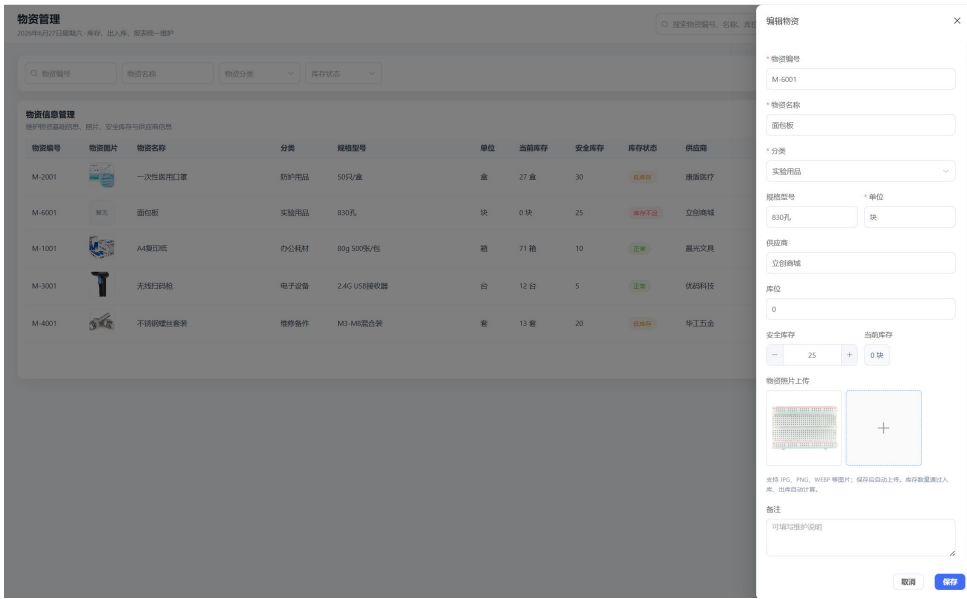


图 30 物资照片上传页面

系统对上传文件格式和大小进行了限制。根据后端实现，系统允许上传 JPG、PNG、WEBP 或 GIF 图片；根据配置文件，单个上传文件最大为 3MB，请求最大为 4MB。若上

传非图片文件或文件为空，系统会给出错误提示。

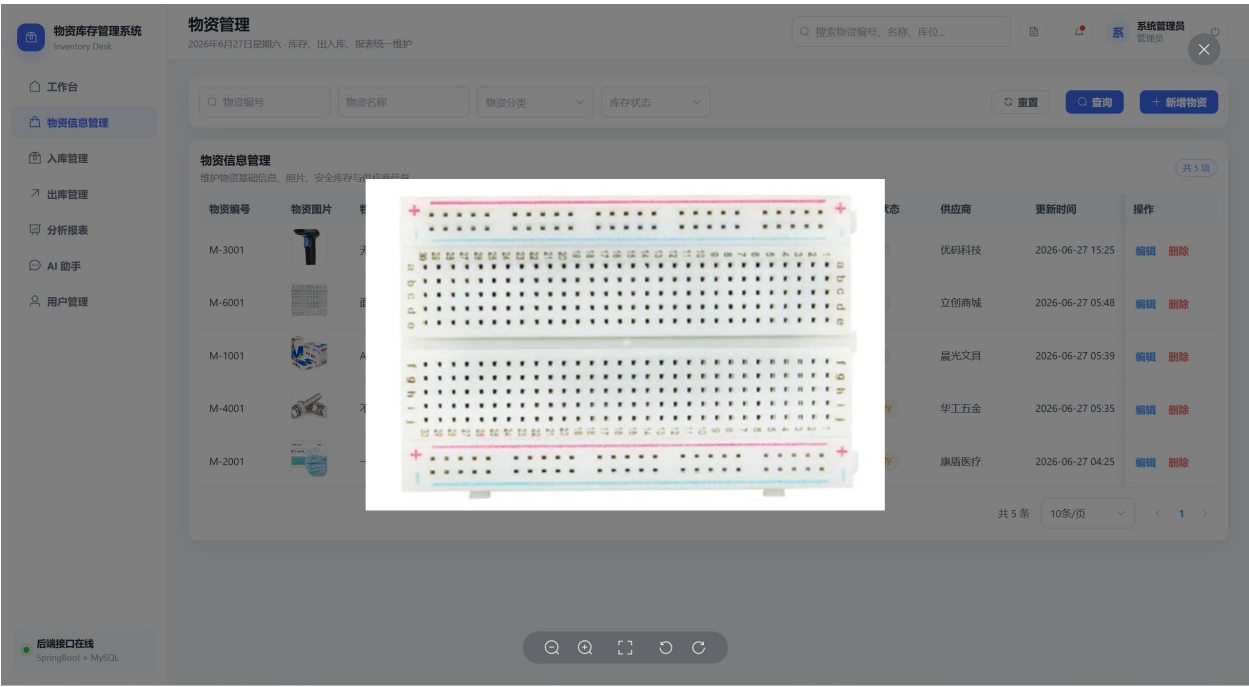


图 31 物资图片显示效果

7.7 入库管理功能测试

入库管理模块用于记录物资进入仓库的业务过程。用户进入“入库管理”页面后，可以查看入库流水，并根据物资、关键字和日期范围进行筛选。新增入库时，需要选择入库物资，填写入库数量、单价、来源和备注等信息。

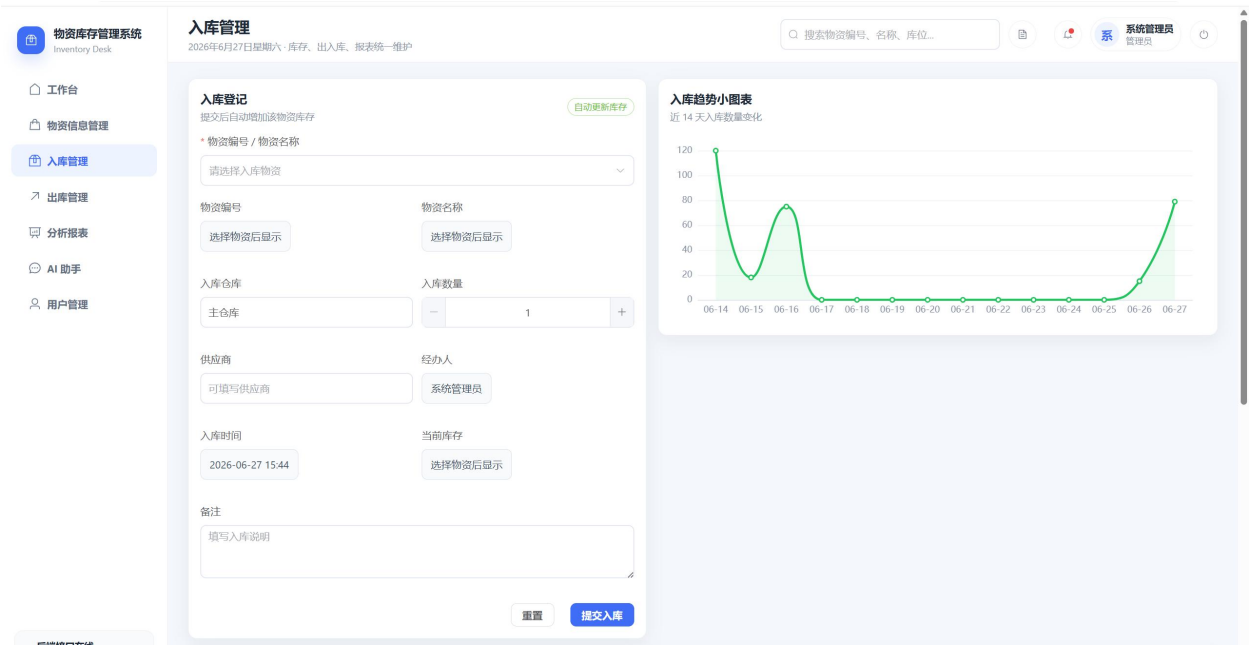




图 32 入库管理页面

用户提交入库信息后，系统先校验入库物资和入库数量。入库数量必须大于 0，入库单价不能小于 0。校验通过后，后端在同一事务中保存入库记录，并同步增加对应物资的当前库存。例如某物资原库存为 20，入库数量为 10，入库成功后库存自动更新为 30。

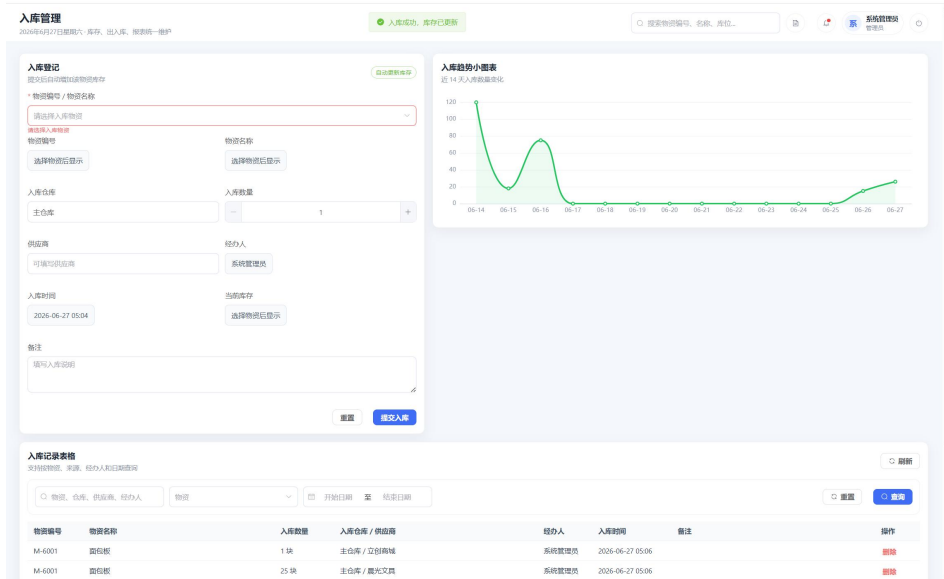
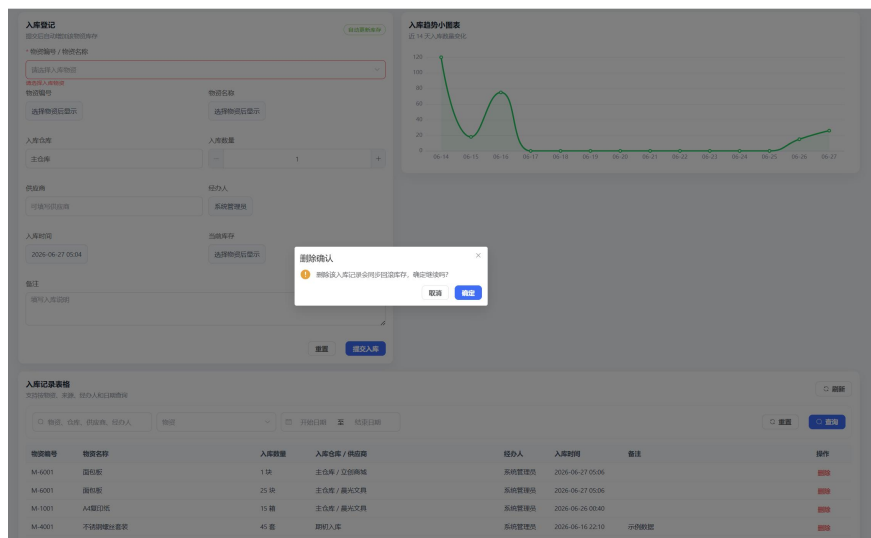


图 33 新增入库页面

管理员还可以删除入库记录。删除入库记录时，系统会同步回滚库存，即从当前库存中减去原入库数量。如果删除后库存会变成负数，系统会拦截删除，避免库存数据异常。



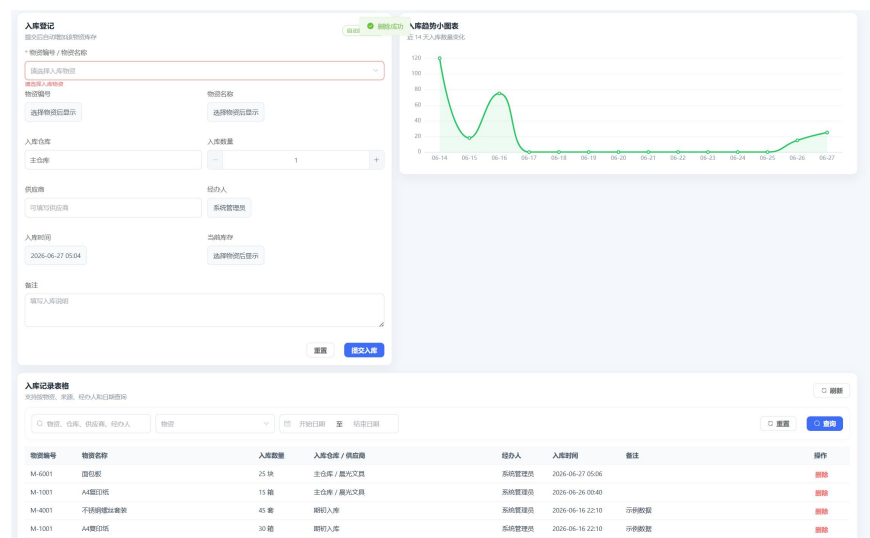


图 34 删除入库记录提示

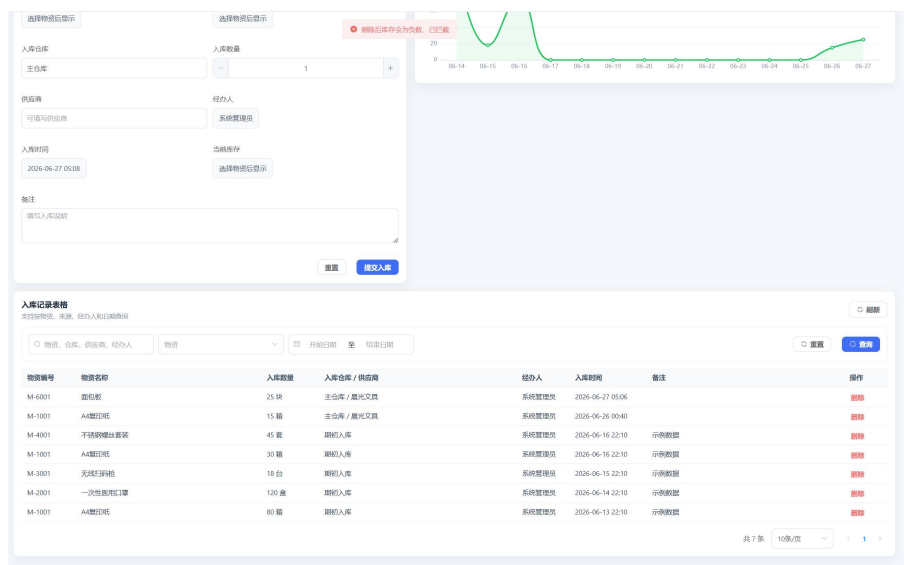


图 35 删除入库记录失败并拦截（删除后库存会变成负数）

7.8 出库管理功能测试

出库管理模块用于记录物资领用或发出的业务过程。用户进入“出库管理”页面后，可以查看出库记录列表，并根据物资、关键字、审核状态和日期范围进行查询。新增出库时，操作员需要选择出库物资，填写出库数量、领用人或领用部门、用途和备注等信息。系统会记录申请人、申请时间、审核状态等信息，便于后续追溯。

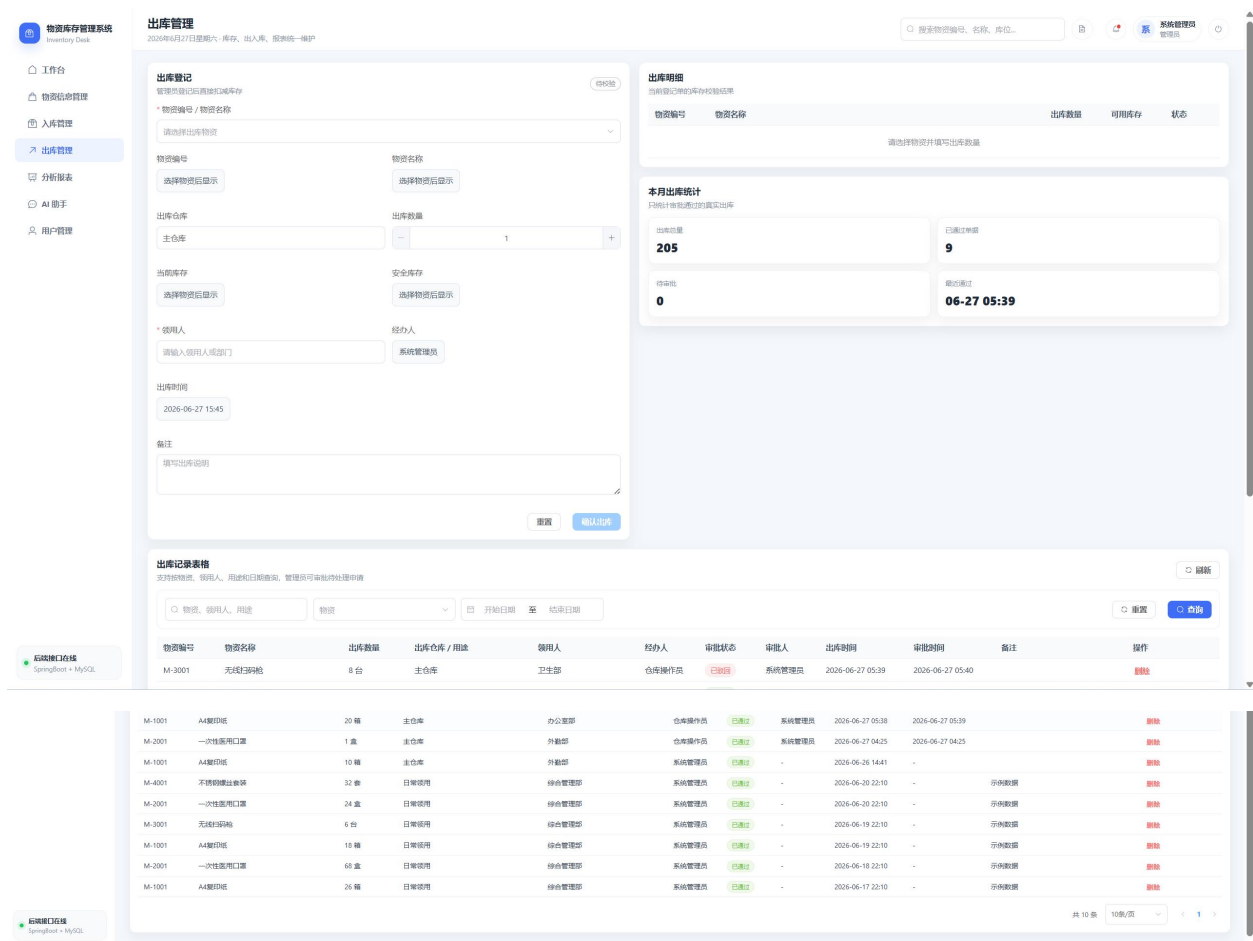
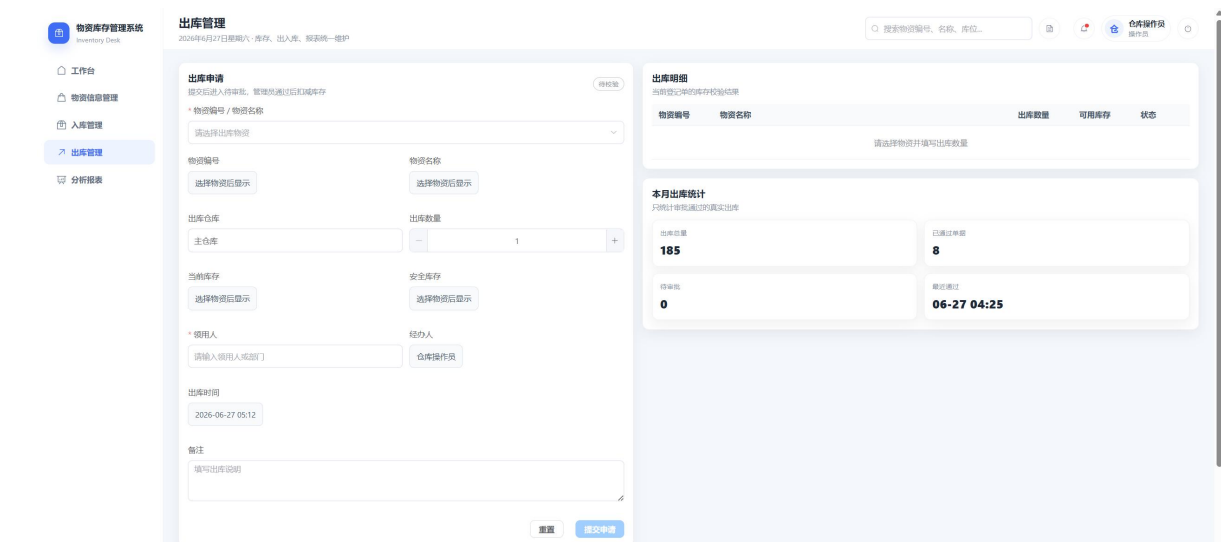


图 36 出库管理页面（管理员）



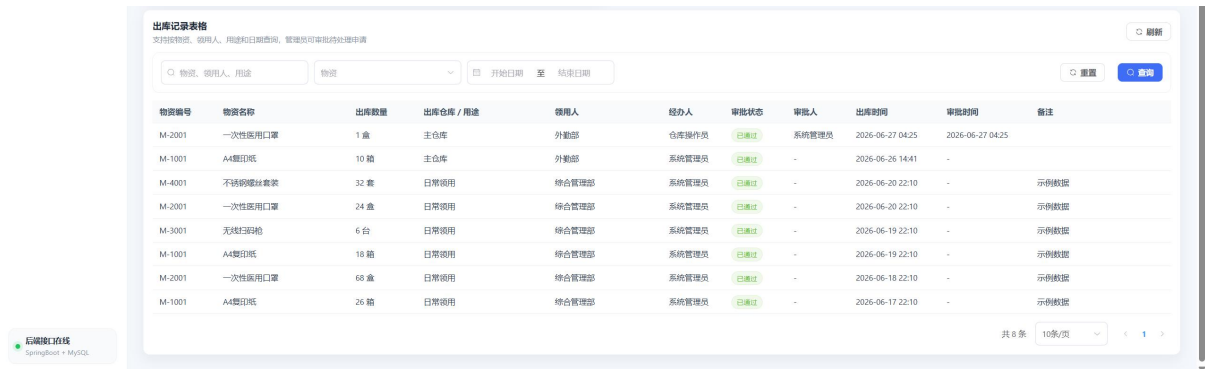


图 37 出库管理页面（操作员）

出库功能的核心是出库申请、库存校验、管理员审核和库存扣减。操作员提交出库申请时，系统首先校验出库物资、出库数量和领用人信息。出库数量必须大于 0，领用人或领用部门不能为空。前端基础校验通过后，后端会查询当前物资库存，判断库存是否充足。如果库存不足，系统直接拒绝提交出库申请，并提示当前库存不足；如果库存充足，系统会生成一条“待审核”的出库记录，此时不会立即扣减库存。

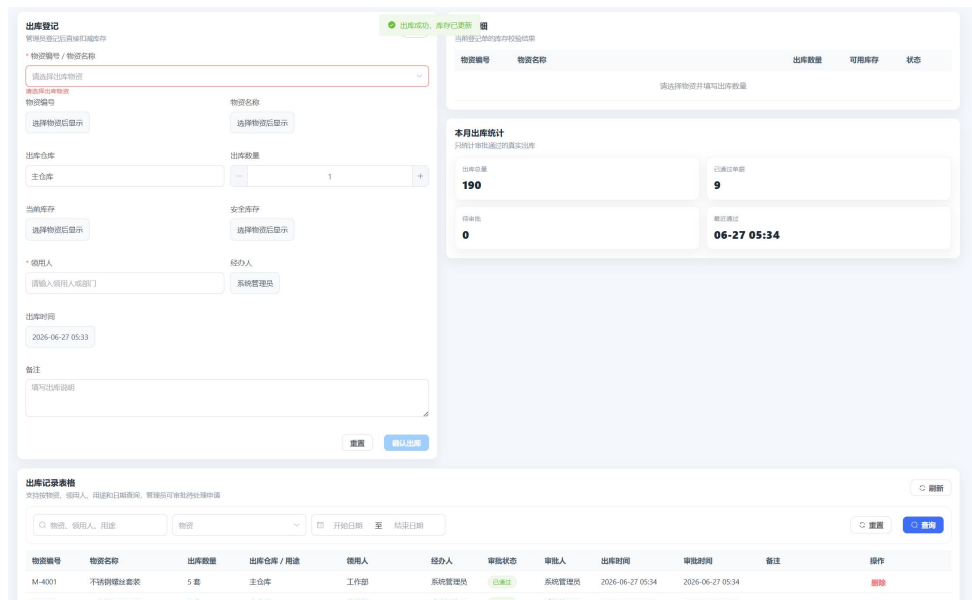


图 38 出库成功（管理员）

2020年06月27日星期六 / 库存, 出入库, 单据统一维护

搜索物资编号、名称、用途...

仓库操作员
操作

出库申请

提交后进入待审核, 管理岗通过后可直接出库

可提交申请

* 物资编号 / 物资名称

M-1001 - A4复印纸

物资编号

M-1001

物资名称

A4复印纸

出库仓库

主仓库

出库数量

20

当前库存

71 箱

安全库存

10 箱

* 领用人

办公室部

经办人

仓库操作员

出库时间

2026-06-27 05:36

备注

填写出库说明

重置

提交申请

出库单据

当前通过审核的库存校验结果

物资编号	物资名称	出库数量	可用库存	状态
M-1001	A4复印纸	20 箱	71 箱	可审核

本月出库统计

当前统计周期内的真实出库

出库总量

185

已通过单据

8

待审核

0

通过时间

06-27 04:25

出库记录表格

支持按物资、领用人、用途和日期查询, 管理岗可审核待处理申请

物资、领用人、用途

物资

开始日期

至

结束日期

重置

查询

物资编号	物资名称	出库数量	出库仓库 / 用途	领用人	经办人	审核状态	审核人	出库时间	审核时间	备注
M-1001	A4复印纸	20 箱	主仓库	办公室部	仓库操作员	待审核	-	2026-06-27 05:36	-	
M-2001	一次性医用口罩	1 盒	主仓库	外勤部	仓库操作员	已通过	系统管理员	2026-06-27 04:25	2026-06-27 04:25	
M-1001	A4复印纸	10 箱	主仓库	外勤部	系统管理员	已通过	-	2026-06-26 14:41	-	
M-4001	不锈钢螺丝套装	32 套	日常使用	综合管理部	系统管理员	已通过	-	2026-06-20 22:10	-	示例数据
M-2001	一次性医用口罩	24 盒	日常使用	综合管理部	系统管理员	已通过	-	2026-06-20 22:10	-	示例数据
M-3001	无线对讲机	6 台	日常使用	综合管理部	系统管理员	已通过	-	2026-06-19 22:10	-	示例数据
M-1001	A4复印纸	18 箱	日常使用	综合管理部	系统管理员	已通过	-	2026-06-19 22:10	-	示例数据
M-2001	一次性医用口罩	68 盒	日常使用	综合管理部	系统管理员	已通过	-	2026-06-18 22:10	-	示例数据
M-1001	A4复印纸	26 箱	日常使用	综合管理部	系统管理员	已通过	-	2026-06-17 22:10	-	示例数据

共 9 条 100条/页 < 1 >

图 39 新增出库申请页面（操作员）

当用户输入的出库数量大于当前库存时，系统不会保存出库申请，也不会扣减库存，而是返回库存不足提示。该测试主要用于验证系统的负库存拦截功能，保证库存不足时不能进入后续审核流程。

2020年06月27日星期六 / 库存, 出入库, 单据统一维护

搜索物资编号、名称、用途...

仓库操作员
操作

出库申请

提交后进入待审核, 管理岗通过后可直接出库

可提交申请

* 物资编号 / 物资名称

M-1001 - A4复印纸

物资编号

M-1001

物资名称

A4复印纸

出库仓库

主仓库

出库数量

100

当前库存

51 箱

安全库存

10 箱

* 领用人

请输入领用人所属部门

经办人

系统管理员

出库时间

2026-06-27 15:45

备注

填写出库说明

重置

提交申请

出库单据

当前通过审核的库存校验结果

物资编号	物资名称
M-1001	A4复印纸

本月出库统计

当前统计周期内的真实出库

出库总量

205

已通过单据

9

待审核

0

通过时间

06-27 05:39

出库记录表格

支持按物资、领用人、用途和日期查询, 管理岗可审核待处理申请

物资、领用人、用途

物资

开始日期

至

结束日期

重置

查询

物资编号	物资名称	出库数量	出库仓库 / 用途	领用人	经办人	审核状态	审核人	出库时间	审核时间	备注	操作
M-3001	无线对讲机	8 台	主仓库	IT管理部	仓库操作员	待审核	系统管理员	2026-06-27 05:39	2026-06-27 05:40		删除

图 40 库存不足拦截提示

操作员成功提交出库申请后，出库记录状态显示为“待审核”。管理员登录系统后，可以在出库管理页面查看待审核记录，并根据实际情况选择“审核通过”或“审核驳回”。如果管理员审核通过，系统会正式扣减对应物资库存，并将出库记录状态更新为“已通过”；如果管理员审核驳回，系统只更新出库记录状态，库存数量不发生变化。

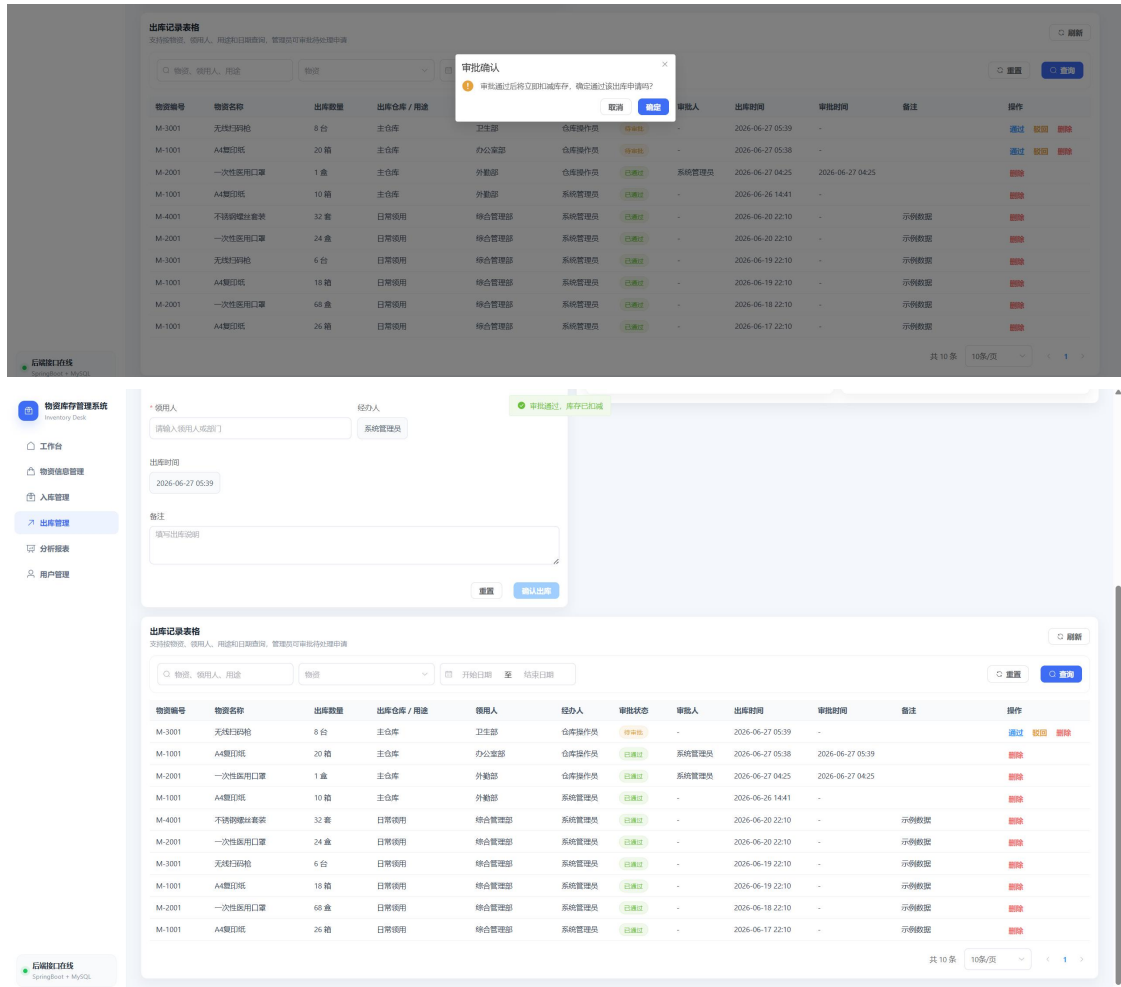
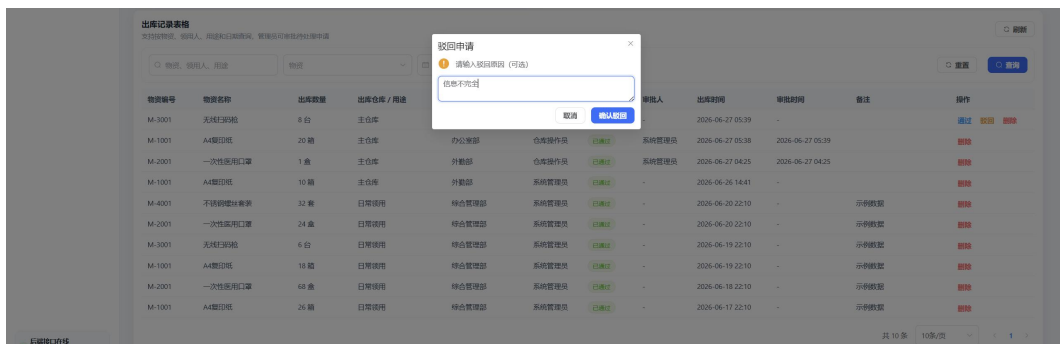


图 41 管理员审核出库申请页面（通过）



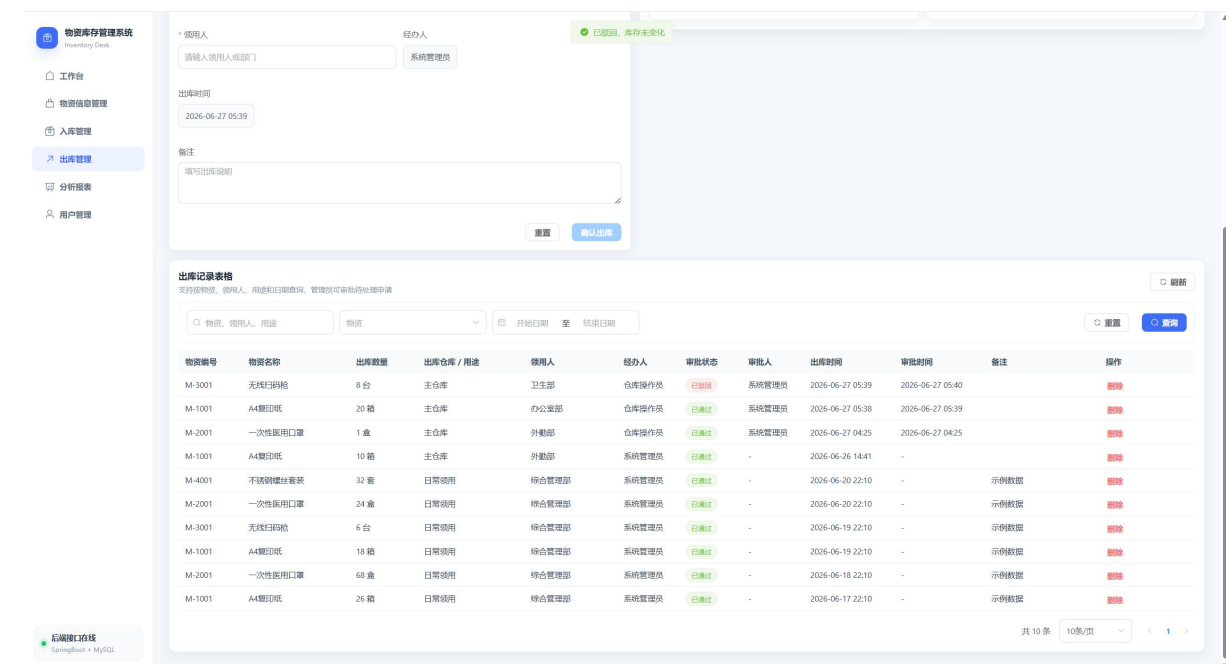


图 42 管理员审核出库申请页面（驳回）

为了验证出库审核功能的正确性，测试时先记录某物资的当前库存，然后由操作员提交一条出库申请。提交成功后检查库存数量，库存应保持不变；管理员审核通过后再次查看库存，库存应减少对应的出库数量。若管理员选择审核驳回，则该出库记录状态变为“已驳回”，物资库存保持不变。

物资信息管理											
维护物资基础信息、图片、安全库存与供应商信息											
物资编号	物资图片	物资名称	分类	规格型号	单位	当前库存	安全库存	库存状态	供应商	更新时间	操作
M-1001		A4复印纸	办公耗材	80g 500张/包	箱	51 箱	10	正常	晨光文具	2026-06-27 05:39	编辑 删除
M-4001		不锈钢螺丝套装	维修备件	M3-M8混合装	套	18 套	20	低库存	华工五金	2026-06-27 05:35	编辑 删除
M-6001		面包板	实验用品	830孔	块	25 块	25	低库存	立创商城	2026-06-27 05:22	编辑 删除
M-2001		一次性医用口罩	防护用品	50只/盒	盒	27 盒	30	低库存	康盾医疗	2026-06-27 04:25	编辑 删除
M-3001		无线扫码枪	电子设备	2.4G USB接收器	台	12 台	5	正常	优码科技	2026-06-26 14:45	编辑 删除

图 43 审核通过后库存变化页面

管理员可以删除出库记录。删除出库记录时，系统需要根据出库记录的审核状态进行处理：如果删除的是已通过的出库记录，则系统需要将原出库数量加回物资库存，实现库存回滚；如果删除的是待审核或已驳回的出库记录，由于这些记录尚未扣减库存，因此删除时只移除记录，不需要修改库存数量。

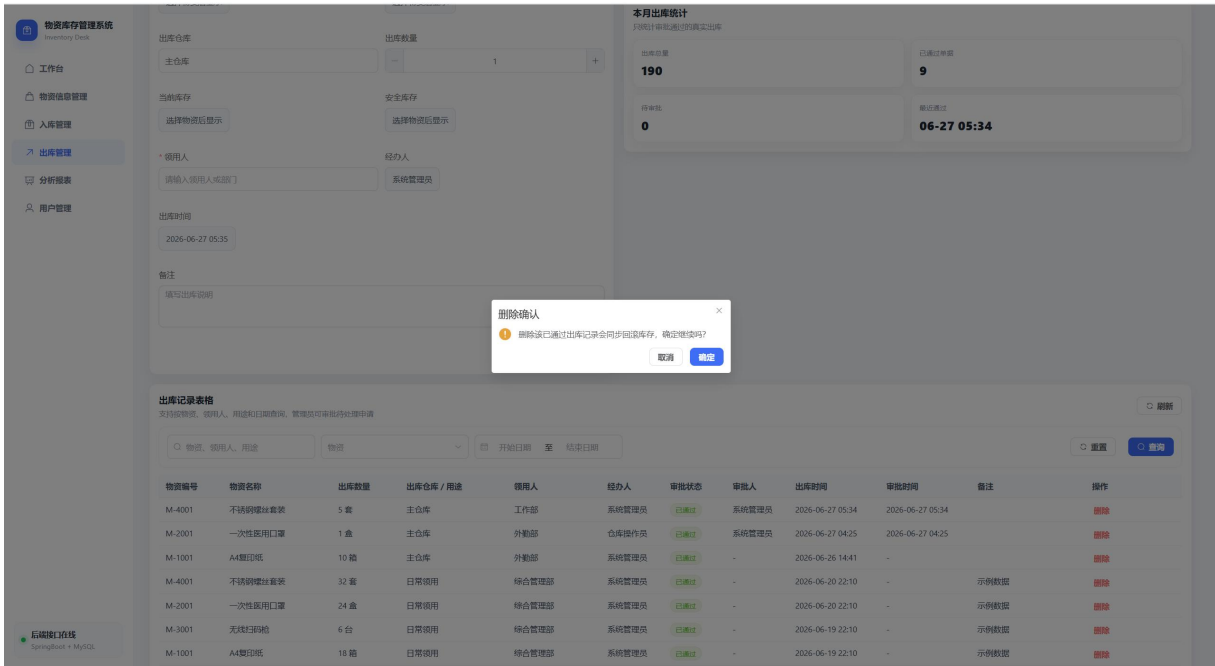


图 44 删除出库记录提示

通过以上测试可以看出，出库管理模块能够完成出库申请提交、库存不足拦截、管理员审核、审核通过后扣减库存、审核驳回后库存不变以及删除记录时库存回滚等功能。该模块不仅保证了出库业务的正常执行，也通过管理员审核机制提高了物资领用流程的规范性和安全性。

7.9 工作台与统计报表功能测试

系统工作台用于展示库存总览数据，包括物资总数、库存总量、低库存数量、今日入库数量和今日出库数量等。工作台还展示分类库存、出入库趋势、库存排行和低库存预警等内容，便于用户快速了解当前库存状态。

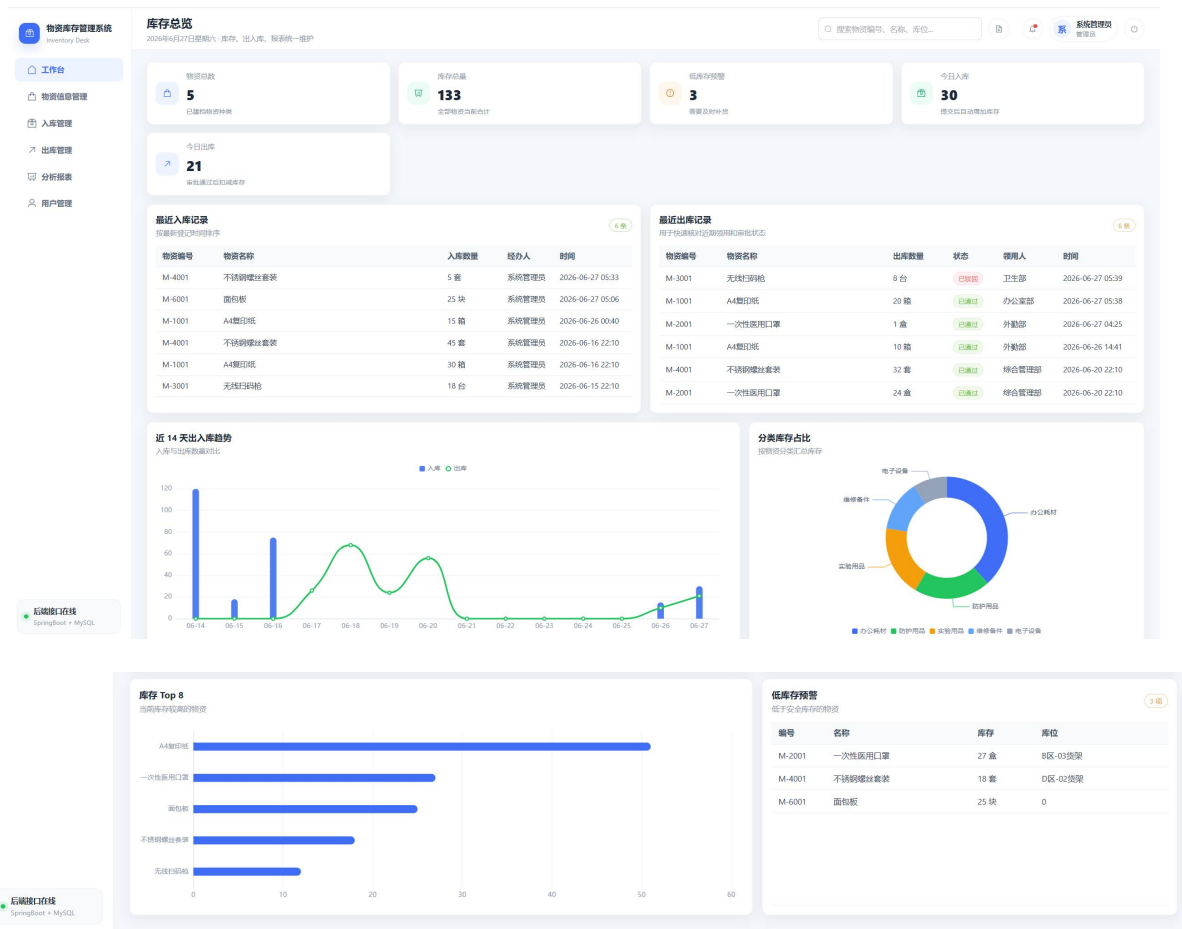
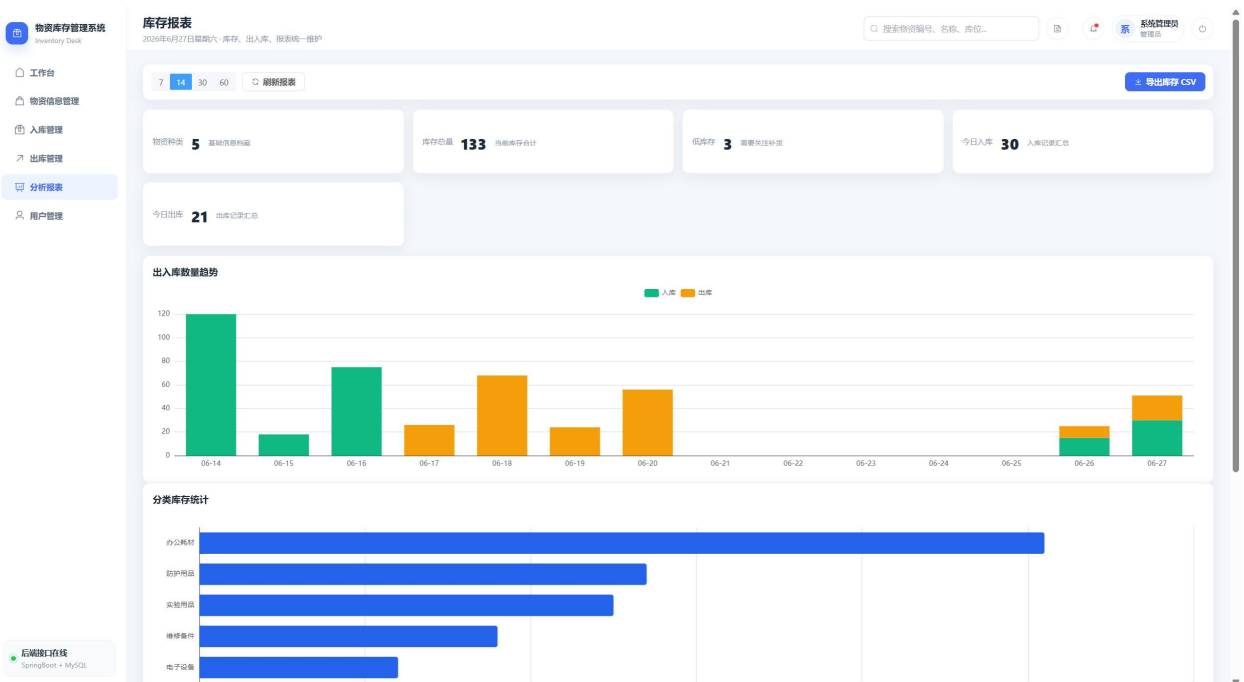


图 45 工作台库存总览页面

分析报表页面用于进一步展示库存统计结果。系统通过后端 SQL 聚合查询物资表、入库记录表和出库记录表中的数据，并使用 ECharts 在前端展示图表。报表页面支持设置趋势统计天数，并支持导出库存报表 CSV 文件。



库存报表明细

编号	名称	分类	规格	库存	安全库存	状态	库位	供应商
M-1001	A4复印纸	办公耗材	80g 500张/包	51 箱	10	正常	A区-01货架	晨光文具
M-4001	不锈钢螺丝套装	维修备件	M3-M8混合装	18 套	20	低库存	D区-02货架	华工五金
M-6001	面包板	实验用品	830孔	25 块	25	低库存	0	立创商城
M-2001	一次性医用口罩	防护用品	50只/盒	27 盒	30	低库存	B区-03货架	康盾医疗
M-3001	无线扫码枪	电子设备	2.4G USB接收器	12 台	5	正常	C区-设备柜	优码科技

后端接口在线
SpringBoot + MySQL

图 46 分析报表页面

	A	B	C	D	E	F	G	H	I	J	K
1	编号	名称	分类	规格	单位	当前库存	安全库存	状态	库位	供应商	
2	M-1001	A4复印纸	办公耗材	80g 500张/箱		51	10	正常	A区-01货架	晨光文具	
3	M-4001	不锈钢螺丝	维修备件	M3-M8混合装		18	20	低库存	D区-02货架	华工五金	
4	M-6001	面包板	实验用品	830孔	块	25	25	低库存		立创商城	
5	M-2001	一次性医用	防护用品	50只/盒	盒	27	30	低库存	B区-03货架	康盾医疗	
6	M-3001	无线扫码枪	电子设备	2.4G USB接	台	12	5	正常	C区-设备柜	优码科技	
7											

图 47 库存报表导出结果

7.10 AI 智能助手功能测试

AI 智能助手模块是本系统在基础库存管理功能上的扩展功能，主要用于为用户提供系统操作指导、库存业务问答以及实时库存数据查询服务。用户进入系统后，可以点击页面中的“AI 助手”按钮打开智能问答窗口，在输入框中输入与物资库存管理相关的问题。系统接收到问题后，会先判断用户问题类型，如果是普通操作类问题，则调用智谱 API 生成操作说明；如果问题涉及库存数量、低库存物资、待审核出库申请、库存排行等实时数据，则后端会先查询数据库中的最新业务数据，再将查询结果与用户问题一起发送给 AI 助手进行整理和回答。

物资库存管理系统
Inventory Desk

工作台

物资信息管理

入库管理

出库管理

分析报表

AI 助手

用户管理

后端接口在线
SpringBoot + MySQL

AI 助手

2026年6月27日星期六 · 库存、出入库、报表统一维护

可查询实时库存、低库存预警、出入库记录，也可咨询系统操作和答疑问题

实时数据

当前库存总览是什么？

现在有哪些低库存物资？

待审核出库申请有多少？

最近入库记录有哪些？

最近出库记录有哪些？

按分类统计库存情况

如何新增物资？

如何提交出库申请？

管理员如何审核出库申请？

库存不足怎么办？

库存智能问答

每次提问都会由后端读取当前数据库数据，再交给 AI 生成回答

清空

AI 助手

你好，我是物资库存管理系统的 AI 助手。你可以问我当前库存总览、某个物资库存、低库存预警、最近入库/出库记录、待审核出库申请，也可以咨询系统操作和答疑问题。

输入问题，例如：当前低库存物资有哪些？

Enter 发送, Shift + Enter 换行

发送

图 48 AI 智能助手页面

测试时，输入“如何提交出库申请？”，系统能够返回出库申请的操作说明，包括进入出库管理页面、点击新增出库申请、选择物资、填写数量和领用人、提交后等待管理员审核等步骤，说明 AI 助手能够根据系统业务场景给出有效回答。



图 49 AI 助手回答出库申请问题

继续输入“管理员如何审核出库申请？”，系统能够返回管理员审核流程，包括管理员进入出库管理页面、查看待审核记录、选择审核通过或审核驳回、审核通过后系统扣减库存、审核驳回后库存不变等内容，说明 AI 助手能够正确解释本系统的出库审核机制。



图 50 AI 助手查询低库存物资页面

为了测试 AI 助手的实时数据查询能力，继续输入“当前有哪些低库存物资？”。后端接收到该问题后，会查询物资表中当前库存小于或等于安全库存的物资数据，例如物资名称、当前库存、安全库存和存放位置等信息，并将查询结果交给 AI 助手生成自然语言回答。系统能够返回当前数据库中最新的低库存物资列表，说明 AI 助手可以结合数据库实时数据进行回答。

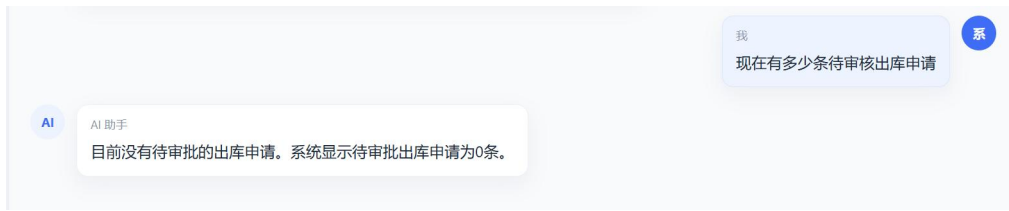


图 51 AI 助手查询待审核出库申请页面

接着输入“现在有多少条待审核出库申请？”。系统会查询出库记录表中审核状态为“待审核”的记录，并统计待审核数量，同时返回对应的物资名称、申请数量、领用人、申请人和申请时间等信息。测试结果显示，AI 助手能够根据数据库中的出库申请数据，返回当前待审核出库记录的数量和简要说明，便于管理员快速了解需要处理的出库申请。

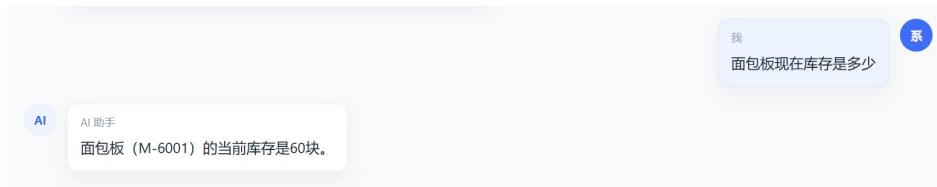


图 52 AI 助手查询单个物资库存页面

继续输入“面包板现在库存是多少？”。系统会根据用户输入的物资名称查询物资表，获取该物资的当前库存、安全库存、单位和库位等信息，并由 AI 助手整理为自然语言回答。例如系统可以返回“面包板当前库存为 30 块，安全库存为 8 块，当前库存状态正常，存放位置为 F 区-01”。该测试用于验证 AI 助手能够根据物资名称查询单个物资的实时库存信息。

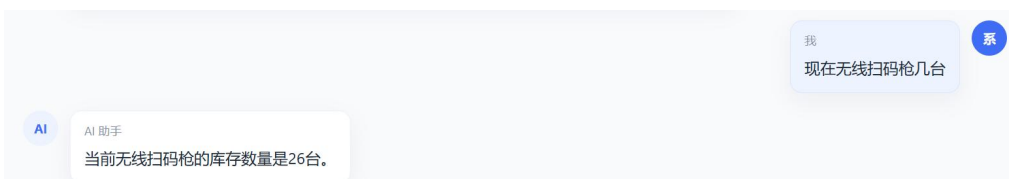
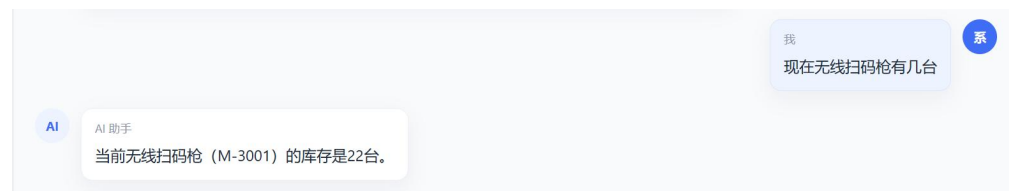


图 53 入库后 AI 助手查询库存变化页面（添加无线扫码枪 4 台）

为了进一步验证数据实时性，测试过程中先在入库管理页面对某物资新增一条入库记录，使该物资库存发生变化，然后再次在 AI 助手中询问该物资库存。测试结果显示，AI 助手返回的库存数量与物资管理页面中的当前库存一致，说明 AI 助手查询的是数据库中的最新数据，而不是固定写死的回答内容。

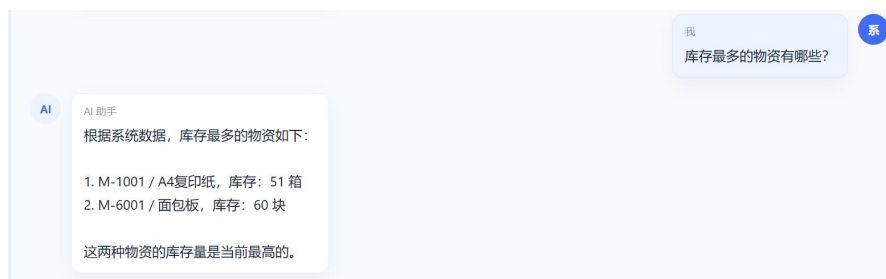


图 54 AI 助手查询库存排行页面

当用户输入“库存最多的物资有哪些？”时，系统会查询物资表中库存数量排名靠前的物资，并返回物资名称、分类、当前库存和单位等信息。AI 助手会将查询结果整理成清晰的文字说明，帮助用户快速了解库存较多的物资情况。该测试说明 AI 助手不仅可以回答操作问题，也可以结合库存数据进行简单分析。

7.11 接口文档测试

本系统集成了 Springdoc OpenAPI，并通过自定义页面提供接口文档访问地址。后端启动成功后，在浏览器中输入：

`http://localhost:3000/api/docs`

即可查看系统接口文档。接口文档中可以查看认证接口、用户接口、物资接口、入库接口、出库接口和报表接口等内容，便于开发阶段进行前后端联调和接口测试。

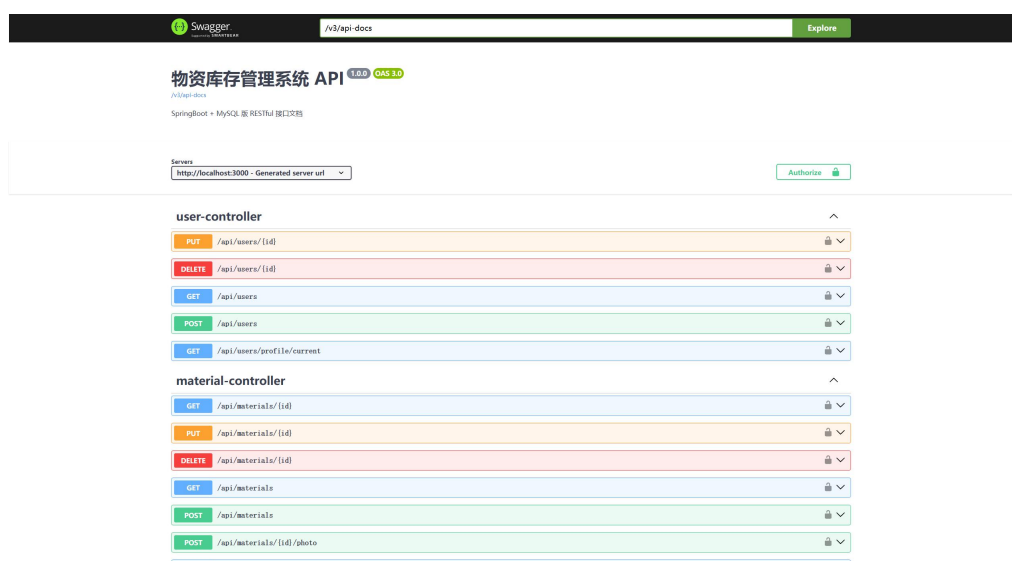


图 51 Swagger 接口文档页面

7.12 开发过程中遇到的问题及解决方法

7.12.1 前端启动命令使用错误问题

在开发初期，曾参考 Vue CLI 项目使用 `npm run serve` 启动前端工程，但本系统前端基于 Vite 创建，前端 `package.json` 中配置的启动命令是 `npm run dev`，根目录中也配置了统一启动命令 `npm run dev`。如果使用 `npm run serve`，会出现脚本不存在或启动失败的问题。

解决方法是查看 `package.json` 文件中的 `scripts` 配置，确认真实启动命令。本系统最终

采用根目录 `npm run dev` 同时启动前端和后端，或进入前端目录单独执行 `npm run dev` 启动前端工程。

7.12.2 登录后访问接口返回 401 问题

开发过程中，登录成功后访问物资管理、入库管理等业务页面时，曾出现接口返回 401 的情况。原因是部分请求没有携带 JWT Token，后端无法识别当前用户身份。

解决方法是在前端 Axios 请求拦截器中统一读取本地保存的 Token，并添加到请求头 `Authorization: Bearer token` 中。后端通过 JWT 过滤器解析 Token，并设置当前登录用户。这样业务接口请求都会自动携带 Token，避免了重复编写请求头的问题。

7.12.3 物资编号重复问题

在物资新增测试中，如果不对物资编号进行唯一性校验，就可能导致同一种物资被重复录入，影响后续入库、出库和统计结果。

解决方法是在后端新增物资时调用 `Repository` 判断物资编号是否已经存在，同时在数据库层面设置物资编号唯一约束。如果编号重复，系统返回“物资编号已存在”提示；如果编号未重复，才允许保存物资信息。

7.12.4 已有出入库记录的物资删除问题

物资删除功能测试时发现，如果直接删除已经发生过出入库业务的物资，会导致历史入库记录和出库记录失去关联，影响数据追溯。

解决方法是在后端删除物资前检查该物资是否存在入库记录或出库记录。若存在关联记录，则拒绝删除并提示“该物资已有出入库记录，不能删除”。这样既保证了历史流水完整性，也避免了数据关系错误。

7.12.5 出库负库存问题

出库业务开发过程中，如果只保存出库记录并扣减库存，而不判断当前库存是否充足，就可能出现库存为负数的情况。

解决方法是在后端出库方法中先通过带锁查询读取当前物资库存，再判断当前库存是否大于或等于出库数量。若库存不足，系统直接拒绝出库；若库存充足，才保存出库记录并扣减库存。同时，前端在提交出库前也会进行库存不足提示，从而实现前后端双重拦截。

7.12.6 入库、出库记录与库存不同步问题

入库和出库业务同时涉及流水记录和库存数量更新，如果其中一步成功而另一步失败，就可能导致库存和流水不一致。

解决方法是在后端入库、出库以及删除回滚方法上使用 `@Transactional` 事务控制。入库时，保存入库记录和增加库存必须同时成功；出库时，保存出库记录和扣减库存必须同时成功；删除出入库记录时，流水删除和库存回滚也必须同时成功。任何一步出现异常时，事务都会回滚，从而保证数据一致性。

7.12.7 并发出入库导致库存计算错误问题

如果多个用户同时对同一种物资进行入库或出库操作，普通查询可能会出现库存读取不一致，从而导致最终库存计算错误。

解决方法是在物资库存变更前使用悲观写锁查询物资记录。系统通过 `findByIdForUpdate()` 方法在库存变更前锁定对应物资，保证同一时刻对同一物资的库存修改按顺序执行，减少并发出入库造成的数据错误。

7.12.8 图表不显示或报表数据不更新问题

统计报表页面开发时，曾出现接口有数据但图表没有显示，或入库出库后报表数据没有及时更新的问题。原因可能是图表容器没有高度、数据请求完成前就初始化图表，或业务操作后没有重新加载报表数据。

解决方法是在前端为图表容器设置固定高度，在数据加载完成后再更新 ECharts 配置；同时在完成入库、出库操作后重新请求报表接口，使统计卡片、出入库趋势和低库存预警数据能够及时刷新。

八、 课程设计总结和展望

首先，本次课程设计完成了一个基于前后端分离架构的物资库存管理系统。系统前端采用 Vue3、Vite 和 Element Plus 实现页面展示与交互，后端采用 SpringBoot、Spring Security、JWT 和 Spring Data JPA 实现接口服务、身份认证、权限控制和数据库持久化，数据库使用 MySQL 8.0 保存系统数据。系统主要实现了用户登录注册、用户管理、物资信息管理、物资照片上传、入库管理、出库管理、出库审核、库存统计报表、低库存预警、权限控制和 AI 智能助手等功能，基本满足了物资库存管理的日常使用需求。

本设计的创新之处主要体现在以下几个方面。第一，本系统在基本出库功能基础上进一步增加了出库审核机制。操作员提交出库申请后，系统不会立即扣减库存，而是生成待审核记录；管理员审核通过后系统才正式扣减库存，审核驳回则库存保持不变。该功能提高了物资领用流程的规范性，也减少了普通操作员误操作导致库存异常的风险。第二，系统不是简单地完成物资信息的增删改查，而是将物资基础信息、入库记录、出库记录和库存统计结合起来，实现了库存数据的自动维护。第三，系统在出库环节加入了库存不足拦截机制，当出库数量大于当前库存时，系统会拒绝出库，避免出现负库存问题。第四，系统增加了低库存预警功能，可以根据安全库存判断物资是否处于低库存状态，从而提醒用户及时补充物资。第五，系统支持物资照片上传，使物资信息展示更加直观，方便用户识别和管理物资。第六，系统使用 ECharts 实现库存数据可视化展示，相比单纯的表格数据，图表能够更加直观地反映库存变化情况。第七，系统区分管理员和操作员两类角色，管理员可以进行用户管理，操作员只能进行日常业务操作，提高了系统权限管理的规范性。在功能扩展方面，本系统在基础库存管理功能之外，增加了物资照片上传、低库存预警、报表统计、CSV 导出、接口文档和权限控制等扩展功能，使系统不仅能够完成基本的数据管理，也具有一定的数据分析和辅助管理能力。例如，低库存预警可以帮助用户及时发现库存不足的物资；统计报表可以帮助用户掌握库存总量、出入库变化和库存排行；CSV 导出可以方便后续进行数据留存和分析；接口文档可以提高前后端联调和系统维护效率。第八，此外，系统增加了 AI 智能助手功能。用户可以通过智能助手咨询物资管理、入库管理、

出库申请、出库审核、低库存预警和统计报表等相关问题，系统通过后端调用智谱 API 生成回答，并在前端聊天窗口中展示。该功能能够为用户提供操作引导和业务规则说明，减少用户对说明文档的依赖，提高系统的智能化水平和使用体验。

但是，本设计仍然存在一些不足之处。第一，系统目前主要适用于单仓库或简单库存场景，对于多仓库、多库区、多货架之间的库存调拨支持还不够完善。如果应用到真实仓库环境中，还需要增加仓库管理、库区管理、货架管理和调拨管理等功能。第二，目前系统只实现了出库审核，入库审核、采购审批、多级审批和审批消息提醒等功能还可以进一步完善，在真实业务中还需要进一步完善。第三，系统目前对操作日志和数据备份支持不足，如果后续用于真实环境，还需要增加操作日志记录、数据备份和异常恢复等功能，以提高系统的安全性和可靠性。

其次，在课程设计过程中，我遇到了一些困难，并通过不断调试和修改逐步解决。第一个问题是前后端联调问题，由于前端和后端端口不同，开发过程中出现过跨域请求、接口路径错误和 Token 未携带等问题。解决方法是统一封装 Axios 请求，在请求拦截器中自动携带 Token，并在后端配置跨域和安全认证规则。第二个问题是库存数据一致性问题，入库和出库都涉及流水记录和库存数量变化，为避免数据不同步，后端使用事务控制，使库存更新和记录保存能够同时成功或同时回滚。第三个问题是图表不显示或数据不刷新的问题，通过设置图表容器高度，并在数据加载完成后再渲染 ECharts 图表解决。

最后，通过本次课程设计，我对 Web 应用开发的完整流程有了更加清楚的认识。从需求分析、数据库设计、系统设计，到前后端编码、接口联调、功能测试和问题分析，每一步都需要认真考虑。通过完成该系统，我进一步熟悉了 Vue3、Element Plus、Axios、ECharts、SpringBoot、Spring Data JPA、Spring Security 和 MySQL 等技术，也更加理解了前后端分离开发模式。

本次课程设计让我认识到，一个系统不能只关注页面是否能够显示，还要考虑业务逻辑是否合理、数据是否安全、异常情况是否能够处理、用户操作是否方便。特别是在库存管理业务中，库存数量的准确性非常重要，因此必须考虑事务控制、库存校验和异常拦截。总体来说，本次课程设计提高了我分析问题、解决问题和综合开发项目的能力，也为以后继续开发更复杂的 Web 系统积累了经验。

参考文献

- [1] 王松. Spring Boot+Vue 全栈开发实战[M]. 北京: 清华大学出版社, 2019.
- [2] 柳伟卫. Vue.js 3 企业级应用开发实战[M]. 北京: 电子工业出版社, 2022.
- [3] Craig Walls. Spring 实战: 第 4 版[M]. 北京: 人民邮电出版社, 2016.
- [4] Laurentiu Spilca. Spring Security 实战[M]. 北京: 人民邮电出版社, 2022.
- [5] Christian Bauer, Gavin King, Gary Gregory. Java Persistence with Hibernate: Second Edition[M]. Shelter Island: Manning Publications, 2015.

- [6] 李小威. MySQL 8.x 从入门到精通: 视频教学版[M]. 北京: 清华大学出版社, 2022.
- [7] Li D, Mei H, Shen Y, et al. ECharts: A Declarative Framework for Rapid Construction of Web-based Visualization[J]. Visual Informatics, 2018, 2(2): 136-146.
- [8] Mark Masse. REST API Design Rulebook[M]. Sebastopol: O'Reilly Media, 2011.
- [9] 王松. Spring Boot+Vue 全栈开发实战[M]. 清华大学出版社, 2019.35-40
- [10] Vue 官方文档. <https://cn.vuejs.org/>
- [11] Element Plus 官方文档. <https://element-plus.org/>
- [12] SpringBoot 官方文档. <https://springdoc.cn/spring-boot/>
- [13] MyBatis 官方文档. <https://mybatis.net.cn/>

附录

一、API 文档

本系统采用前后端分离架构, 前端通过 Axios 调用后端 RESTful API, 后端使用 SpringBoot 提供接口服务。系统接口统一以 /api 作为基础路径, 接口数据主要采用 JSON 格式进行传输。系统同时集成 Swagger 接口文档, 访问 <http://localhost:3000/api/docs> 可以查看系统后端接口, 包括用户认证、用户管理、物资管理、入库管理、出库管理、统计报表和系统健康检查等接口。

物资库存管理系统 API 1.0.0 OAS 3.0
/v3/api-docs
SpringBoot + MySQL 版 RESTful 接口文档

Servers
http://localhost:3000 - Generated server url

Authorize

user-controller

- PUT /api/users/{id}
- DELETE /api/users/{id}
- GET /api/users
- POST /api/users
- GET /api/users/profile/current

material-controller

- GET /api/materials/{id}
- PUT /api/materials/{id}
- DELETE /api/materials/{id}
- GET /api/materials
- POST /api/materials
- POST /api/materials/{id}/photo
- GET /api/materials/categories/options

outbound-controller

- GET /api/outbound
- POST /api/outbound
- POST /api/outbound/{id}/reject
- POST /api/outbound/{id}/approve
- DELETE /api/outbound/{id}

inbound-controller ^	
GET	/api/inbound
POST	/api/inbound
DELETE	/api/inbound/{id}
auth-controller ^	
POST	/api/auth/register
POST	/api/auth/login
GET	/api/auth/me
report-controller ^	
GET	/api/reports/stock-by-category
GET	/api/reports/movement
GET	/api/reports/low-stock
GET	/api/reports/dashboard
health-controller ^	
GET	/api/health
swagger-page-controller ^	
GET	/api/docs
GET	/api/docs/
ai-controller ^	
POST	/api/ai/chat

1. 接口认证方式

系统采用 Spring Security + JWT 实现接口认证。用户通过登录接口提交用户名和密码，登录成功后后端返回 JWT Token 和当前用户信息。前端保存 Token 后，在后续请求中通过请求头携带 Token 访问受保护接口。

请求头格式如下：

```
Authorization: Bearer <token>
```

在 Swagger 页面中测试受保护接口时，需要先调用登录接口获取 Token，然后点击页面右上角的 Authorize 按钮，将 Token 填入授权框中。授权成功后，即可测试用户管理、物资管理、入库管理、出库管理和统计报表等接口。

系统主要用户角色包括管理员和操作员。其中，管理员拥有用户管理、物资管理、入库管理、出库审核、出库管理和统计报表等权限；操作员主要负责物资查询、入库登记、出库申请和统计查看，不能访问用户管理功能，也不能审核出库申请。

2. 通用返回格式

普通接口通常返回对象数据或操作结果。例如登录接口返回 Token 和用户信息，删除接口返回成功标识。分页查询接口统一返回分页结果，格式如下：

```
{
  "items": [],
  "total": 0,
  "page": 1,
  "pageSize": 10
}
```

其中, `items` 表示当前页数据列表, `total` 表示符合条件的总记录数, `page` 表示当前页码, `pageSize` 表示每页数量。

当接口发生异常时, 后端统一返回错误信息, 例如:

```
{
  "message": "库存不足"
}
```

状态码	含义	说明
200	请求成功	接口调用成功并返回结果
400	参数错误	请求参数为空、数量非法或格式错误
401	未认证	用户未登录或 <code>Token</code> 无效
403	权限不足	当前用户无权执行该操作
404	数据不存在	请求的数据记录不存在
409	数据冲突	编号重复、库存不足或数据被关联引用
500	服务器错误	后端出现未知异常

3. 用户认证接口

用户认证接口由 `auth-controller` 提供, 主要用于用户登录、用户注册和获取当前登录用户信息。

请求方式	接口路径	功能说明	是否需要登录
POST	<code>/api/auth/login</code>	用户登录, 返回 <code>JWT Token</code> 和用户信息	否
POST	<code>/api/auth/register</code>	用户自助注册, 注册成功后返回 <code>Token</code>	否
GET	<code>/api/auth/me</code>	获取当前登录用户信息	是

登录接口请求示例:

```
{
  "username": "admin",
  "password": "admin123"
}
```

登录接口返回示例:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "user": {
    "id": 1,
    "username": "admin",
    "name": "系统管理员",
    "role": "admin",
    "status": 1
  }
}
```

```
}  
}
```

4. 用户管理接口

用户管理接口由 `user-controller` 提供，主要用于管理员维护系统用户。该模块属于管理员专属功能，普通操作员不能访问。

请求方式	接口路径	功能说明	权限要求
GET	/api/users	分页查询用户列表，支持关键字查询	管理员
POST	/api/users	新增系统用户	管理员
PUT	/api/users/{id}	修改指定用户信息	管理员
DELETE	/api/users/{id}	删除指定用户	管理员
GET	/api/users/profile/current	获取当前登录用户资料	管理员

参数名	类型	是否必填	说明
keyword	String	否	按用户名或姓名模糊查询
page	int	否	当前页码，默认 1
pageSize	int	否	每页数量，默认 10

新增用户请求示例：

```
{  
  "username": "operator01",  
  "name": "操作员一号",  
  "password": "123456",  
  "role": "operator",  
  "status": 1  
}
```

用户角色字段中，`admin` 表示管理员，`operator` 表示操作员。用户状态字段中，`1` 表示启用，`0` 表示停用。用户密码在后端使用 `BCrypt` 加密存储，接口返回数据中不会包含密码信息。

5. 物资管理接口

物资管理接口由 `material-controller` 提供，主要用于维护物资基础信息、上传物资照片和查询物资分类。物资编号具有唯一性，当前库存不通过物资编辑直接修改，而是由入库和出库业务自动维护。

请求方式	接口路径	功能说明	是否需要登录
GET	/api/materials	分页查询物资列表	是
GET	/api/materials/{id}	根据 ID 查询物资详情	是
POST	/api/materials	新增物资信息	是
PUT	/api/materials/{id}	修改物资基础信息	是
DELETE	/api/materials/{id}	删除物资信息	是

POST	/api/materials/{id}/photo	上传物资照片	是
GET	/api/materials/categories/options	获取物资分类选项	是

参数名	类型	是否必填	说明
keyword	String	否	按物资编号或名称查询
category	String	否	按物资分类筛选
lowStock	boolean	否	是否只查询低库存物资
page	int	否	当前页码，默认 1
pageSize	int	否	每页数量，默认 10

新增物资请求示例：

```
{
  "code": "SY-001",
  "name": "面包板",
  "category": "实验用品",
  "spec": "830 孔",
  "unit": "块",
  "supplier": "立创商城",
  "location": "F 区-01",
  "safetyStock": 8,
  "remark": "电子实验常用耗材"
}
```

物资照片上传接口使用 multipart/form-data 格式，请求参数名为 photo。上传成功后，后端返回图片访问路径。

6. 入库管理接口

入库管理接口由 inbound-controller 提供，主要用于物资入库登记、入库记录查询和入库记录删除。新增入库时，系统会在同一事务中保存入库记录并增加对应物资库存。

请求方式	接口路径	功能说明	权限要求
GET	/api/inbound	分页查询入库记录	登录用户
POST	/api/inbound	新增入库记录并增加库存	登录用户
DELETE	/api/inbound/{id}	删除入库记录并回滚库存	管理员

参数名	类型	是否必填	说明
keyword	String	否	按物资编号、名称、来源或操作人查询
materialId	Long	否	按物资 ID 查询
from	LocalDate	否	开始日期

to	LocalDate	否	结束日期
page	int	否	当前页码，默认 1
pageSize	int	否	每页数量，默认 10

新增入库请求示例：

```
{
  "materialId": 1,
  "quantity": 20,
  "unitPrice": 5.5,
  "source": "采购入库",
  "remark": "实验课补充耗材"
}
```

入库业务规则：入库数量必须大于 0，单价不能小于 0，入库物资必须存在。入库成功后，系统自动执行“当前库存 = 原库存 + 入库数量”。

7. 出库管理接口

出库管理接口由 `outbound-controller` 提供，主要用于出库记录查询、出库申请提交、管理员审核和出库记录删除。系统在出库模块中加入管理员审核机制。普通操作员提交出库后生成待审核申请，库存暂不扣减；管理员审核通过后，系统才正式扣减库存；管理员审核驳回后，库存保持不变。

请求方式	接口路径	功能说明	权限要求
GET	/api/outbound	分页查询出库记录	登录用户
POST	/api/outbound	提交出库申请或管理员直接出库	登录用户
POST	/api/outbound/{id}/approve	管理员审核通过出库申请	管理员
POST	/api/outbound/{id}/reject	管理员驳回出库申请	管理员
DELETE	/api/outbound/{id}	删除出库记录	管理员

参数名	类型	是否必填	说明
keyword	String	否	按物资编号、名称、领用人、用途或操作人查询
materialId	Long	否	按物资 ID 查询
from	LocalDate	否	开始日期
to	LocalDate	否	结束日期
page	int	否	当前页码，默认 1
pageSize	int	否	每页数量，默认 10

驳回出库请求示例：

```
{
  "remark": "领用数量过多，请重新提交"
}
```

状态值	中文含义	库存是否扣减
pending	待审批	否
approved	已通过	是
rejected	已驳回	否

出库业务规则：提交出库时，系统首先校验出库数量是否大于 0，并判断当前库存是否充足。如果库存不足，系统拒绝提交出库申请。普通操作员提交成功后，记录状态为 `pending`，库存不变；管理员审核通过后，记录状态变为 `approved`，并扣减库存；管理员审核驳回后，记录状态变为 `rejected`，库存不变。删除已通过出库记录时，系统会将原出库数量加回库存；删除待审批或已驳回记录时，库存不发生变化。

8. 统计报表接口

统计报表接口由 `report-controller` 提供，主要用于工作台和分析报表页面展示库存统计数据。统计数据来源于物资表、入库记录表和出库记录表，其中出库统计只统计已通过的出库记录。

请求方式	接口路径	功能说明	是否需要登录
GET	/api/reports/dashboard	获取工作台综合统计数据	是
GET	/api/reports/movement	获取出入库趋势数据	是
GET	/api/reports/stock-by-category	获取分类库存统计数据	是
GET	/api/reports/low-stock	获取低库存物资列表	是

参数名	类型	是否必填	说明
days	int	否	统计最近多少天，默认 14 天

工作台统计接口返回内容主要包括：物资总数、库存总量、低库存数量、今日入库数量、今日出库数量、分类库存统计、出入库趋势、库存排行和低库存预警列表。

9. 系统健康检查与接口文档接口

系统还提供健康检查和接口文档访问接口，用于确认后端服务运行状态和查看 Swagger 页面。

请求方式	接口路径	功能说明	是否需要登录
GET	/api/health	后端服务健康检查	否
GET	/api/docs	访问 Swagger 接口文档页面	否
GET	/api/docs/	访问 Swagger 接口文档页面	否
GET	/v3/api-docs	OpenAPI 原始接口文档数据	否

健康检查接口返回示例：

```
{
  "status": "ok",
  "time": "2026-06-27T10:00:00"
}
```

10. AI 智能助手接口

为提高系统易用性，本系统增加 AI 智能助手接口，用于回答用户在系统使用过程中遇到的问题，例如物资如何新增、如何提交出库申请、管理员如何审核出库申请、当前有哪些低库存物资等。前端负责收集用户输入的问题，后端负责调用智谱 API，并在需要时查询数据库中的实时库存数据后返回结果。

AI 智能助手接口设计如下：

请求方式	接口路径	功能说明	是否需要登录
POST	/api/assistant/chat	接收用户问题，返回 AI 助手回答	是

参数名	类型	是否必填	说明
message	String	是	用户输入的问题内容

请求示例：

```
{
  "message": "当前有哪些低库存物资？"
}
```

返回示例：

```
{
  "success": true,
  "reply": "当前共有 3 种低库存物资，分别是 STM32 开发板、万用表和电烙铁，建议管理员及时补充库存。"
}
```

11. 接口设计小结

本系统接口按照 RESTful 风格进行设计，使用 GET 表示查询，POST 表示新增、登录、提交和审核操作，PUT 表示修改数据，DELETE 表示删除数据。接口模块划分清晰，包括认证接口、用户接口、物资接口、入库接口、出库接口、报表接口和健康检查接口。系统通过 JWT 实现接口鉴权，通过角色权限控制限制管理员功能，通过统一异常处理返回错误信息，使前后端联调、接口测试和系统维护更加规范。

二、数据库设计文档

本系统数据库采用 MySQL 8.0，数据库名为 inventory_db。后端使用 Spring Data JPA + Hibernate 完成实体类与数据库表之间的映射，并通过 ddl-auto: update 自动维护表结构。系统核心数据表包括用户表 users、物资表 materials、入库记录表 inbound_records 和出库记录表 outbound_records。

1. 数据库总体结构

系统围绕“用户—物资—入库—出库”四类核心数据进行设计，各表之间关系如下：

数据表	中文名称	主要作用
users	用户表	保存系统用户账号、密码密文、姓名、角色、状态和时间信息
materials	物资表	保存物资编号、名称、分类、规格、单位、供应商、库位、库存、安全库存、照片地址等信息
inbound_records	入库记录表	保存物资入库流水，包括入库物资、数量、单价、来源、操作人和创建时间
outbound_records	出库记录表	保存物资出库申请和审核流水，包括出库物资、数量、领用人、用途、审核状态、审核人和审核时间

核心关系如下：

（1）materials 与 inbound_records 是一对多关系。一种物资可以对应多条入库记录，一条入库记录只能关联一种物资。

（2）materials 与 outbound_records 是一对多关系。一种物资可以对应多条出库记录，一条出库记录只能关联一种物资。

（3）users 与 inbound_records 是一对多关系。一个用户可以创建多条入库记录，一条入库记录对应一个创建人。

（4）users 与 outbound_records 是一对多关系。一个用户可以创建多条出库记录，一条出库记录对应一个申请人或操作人。

（5）users 与 outbound_records 还通过 approved_by 字段建立审核关系。一个管理员可以审核多条出库申请，一条出库记录最多对应一个审核人。

2. 用户表 users

用户表用于保存系统登录账号和权限信息。系统通过用户名和密码完成登录认证，通过角色字段区分管理员和操作员，通过状态字段控制账号启用或停用。用户密码不以明文形式保存，而是经过 BCrypt 加密后存入 password_hash 字段。

字段名	数据类型	约束	字段说明
id	BIGINT	主键，自增	用户 ID，用于唯一标识用户
username	VARCHAR(60)	非空，唯一	登录用户名
password_hash	VARCHAR(120)	非空	用户密码密文，采用 BCrypt 加密存储
name	VARCHAR(60)	非空	用户姓名或显示名称
role	VARCHAR(20)	非空	用户角色，admin 表示管理员，operator 表示操作员
status	INT	非空	用户状态，1 表示启用，0 表示停用

created_at	DATETIME(6)	非空	用户创建时间
updated_at	DATETIME(6)	非空	用户信息更新时间

用户表 users 设计说明：

- (1) username 字段设置唯一约束，防止多个用户使用相同账号。
- (2) password_hash 字段保存加密后的密码，避免数据库中出现明文密码，提高系统安全性。
- (3) role 字段用于权限控制，管理员可以访问用户管理和出库审核功能，操作员只能访问日常业务功能。
- (4) status 字段用于账号启停控制，被停用的账号不能正常登录系统。

3. 物资表 materials

物资表是系统的基础数据表，用于保存物资档案和当前库存状态。入库、出库和统计报表都依赖物资表中的基础数据。系统要求每种物资具有唯一编号，当前库存由入库和出库业务自动维护，不能通过普通编辑随意修改。

字段名	数据类型	约束	字段说明
id	BIGINT	主键，自增	物资 ID，用于唯一标识物资记录
code	VARCHAR(80)	非空，唯一	物资编号，每种物资的唯一标识
name	VARCHAR(120)	非空	物资名称，用于页面展示和查询
category	VARCHAR(80)	非空	物资分类，用于分类管理和统计
spec	VARCHAR(160)	可空	规格型号，用于描述物资规格
unit	VARCHAR(20)	非空	计量单位，如个、块、瓶、包等
supplier	VARCHAR(120)	可空	供应商信息
location	VARCHAR(120)	可空	库位信息，用于记录物资存放位置
stock	DOUBLE	非空	当前库存，由入库和出库业务自动计算
safety_stock	DOUBLE	非空	安全库存，用于低库存预警
photo_url	VARCHAR(255)	可空	物资照片访问路径
remark	VARCHAR(500)	可空	备注信息
created_at	DATETIME(6)	非空	物资创建时间
updated_at	DATETIME(6)	非空	物资更新时间

物资表 materials 设计说明：

- (1) code 字段为物资唯一编号，系统在新增和修改时都会校验编号是否重复。
- (2) stock 字段表示当前库存，新建物资时默认为 0，后续由入库和出库业务自动维护。
- (3) safety_stock 字段表示安全库存，当 stock <= safety_stock 时，系统将该物资标记为低库存。
- (4) photo_url 字段保存物资照片路径，前端根据该路径显示物资图片。

4. 入库记录表 inbound_records

入库记录表用于保存物资入库流水。当采购、补货或退还物资进入仓库时，系统会生成入库记录，并在同一事务中增加对应物资库存。

字段名	数据类型	约束	字段说明
id	BIGINT	主键，自增	入库记录 ID
material_id	BIGINT	外键，非空	入库物资 ID，关联 materials 表
quantity	DOUBLE	非空	入库数量，必须大于 0
unit_price	DOUBLE	非空	入库单价，不能小于 0
source	VARCHAR(120)	可空	入库来源，如采购入库、退还入库等
operator	VARCHAR(60)	非空	操作人姓名
remark	VARCHAR(500)	可空	备注信息
created_by	BIGINT	外键，可空	创建人 ID，关联 users 表
created_at	DATETIME(6)	非空	入库时间

入库记录表 inbound_records 设计说明：

- (1) material_id 字段关联物资表，用于确定本次入库对应的物资。
- (2) quantity 字段表示入库数量，系统要求入库数量必须大于 0。
- (3) created_by 字段关联用户表，用于记录是谁执行了入库操作。
- (4) 新增入库记录时，系统执行“当前库存 = 原库存 + 入库数量”。

5. 出库记录表 outbound_records

出库记录表用于保存物资出库申请、出库审核和出库流水。系统在出库模块中增加管理员审核机制，操作员提交出库申请后，记录状态为待审批，库存不立即扣减；管理员审核通过后，系统扣减库存；管理员审核驳回后，库存不发生变化。

字段名	数据类型	约束	字段说明
id	BIGINT	主键，自增	出库记录 ID
material_id	BIGINT	外键，非空	出库物资 ID，关联 materials 表
quantity	DOUBLE	非空	出库数量，必须大于 0
recipient	VARCHAR(120)	非空	领用人或领用部门
purpose	VARCHAR(160)	可空	出库用途
operator	VARCHAR(60)	非空	操作人姓名
remark	VARCHAR(500)	可空	出库备注
status	VARCHAR(20)	可空	出库审核状态，pending、approved、rejected
approval_remark	VARCHAR(500)	可空	审核备注或驳回原因
created_by	BIGINT	外键，可空	申请人 ID，关联 users 表
approved_by	BIGINT	外键，可空	审核人 ID，关联 users 表
created_at	DATETIME(6)	非空	出库申请时间
approved_at	DATETIME(6)	可空	审核时间

出库记录表 outbound_records 设计说明：

- (1) material_id 字段关联物资表，用于确定出库物资。

- (2) quantity 字段表示出库数量，系统要求数量必须大于 0，并且不能超过当前库存。
- (3) status 字段用于记录审核状态。普通操作员提交出库申请时，状态默认为 pending；管理员审核通过后，状态变为 approved；管理员审核驳回后，状态变为 rejected。
- (4) 只有状态为 approved 的出库记录才会影响库存统计。待审批和已驳回记录不扣减库存。

6. 出库状态说明

状态值	中文含义	说明
pending	待审批	操作员提交申请后生成，库存不变
approved	已通过	管理员审核通过后生成，库存已扣减
rejected	已驳回	管理员驳回申请后生成，库存不变

7. 数据表关系设计

关系	说明
materials.id → inbound_records.material_id	一种物资可以对应多条入库记录
materials.id → outbound_records.material_id	一种物资可以对应多条出库记录
users.id → inbound_records.created_by	一个用户可以创建多条入库记录
users.id → outbound_records.created_by	一个用户可以提交多条出库申请
users.id → outbound_records.approved_by	一个管理员可以审核多条出库申请

8. 关键约束与业务规则

- 物资编号 materials.code 必须唯一，避免同一种物资重复录入。
- 用户名 users.username 必须唯一，避免多个用户使用同一登录账号。
- 入库数量和出库数量必须大于 0，不能为 0、负数或空值。
- 物资当前库存 materials.stock 由入库和出库业务自动维护，不允许用户随意手动修改。
- 入库成功后，系统自动增加库存；入库失败时，库存不发生变化。
- 提交出库申请时，系统必须判断库存是否充足，库存不足时拒绝提交申请。
- 操作员提交出库申请后，状态为 pending，库存不发生变化。
- 管理员审核通过后，状态变为 approved，系统扣减库存。
- 管理员审核驳回后，状态变为 rejected，库存保持不变。
- 低库存判断规则为：当 stock <= safety_stock 时，系统将物资标记为低库存。
- 对于已经存在入库或出库记录的物资，删除时需要谨慎处理，防止破坏历史记录。

9. 数据库设计小结

本系统数据库设计围绕物资库存管理业务展开，核心表结构清晰，能够支持用户登录认证、角色权限控制、物资基础信息维护、入库自动增加库存、出库审核、出库通过后扣减库存、低库存预警和统计报表分析等功能。通过物资表、用户表、入库记录表和出库记录表之间的关联关系，系统能够完整记录库存变化过程，实现物资流转的可追溯管理。同时，通过唯一约束、外键关联、事务控制和业务校验，系统能够有效防止重复数据、负库存和库存记录不一致等问题，提高了库存数据的准确性和系统可靠性。